Lecture 7
Graph Algorithms: Shortest Path Problem

EEE 121 THVW1

Steven S. Sison

Exam Results

Summary of Scores

Problem 1

- Average: 8.64
- Min: 2
- Max: 16

Problem 2

- Average: 11.48
- Min: 6
- Max: 17

Problem 3

- Average: Still checking
- Min: Still checking
- Max: Still checking

Problem 4

- Average: 10.28
- Min: 0
- Max: 20

Problem 5

- Average: 14.466
- Min: 1
- Max: 21

Total

- Average: 43.866
- Min: 15.4
- Max: 70

Summary of Scores

Problem 1

- Average: 8.64
- Min: 2
- Max: 16

Problem 2

- Average: 11.48
- Min: 6
- Max: 17

Problem 3

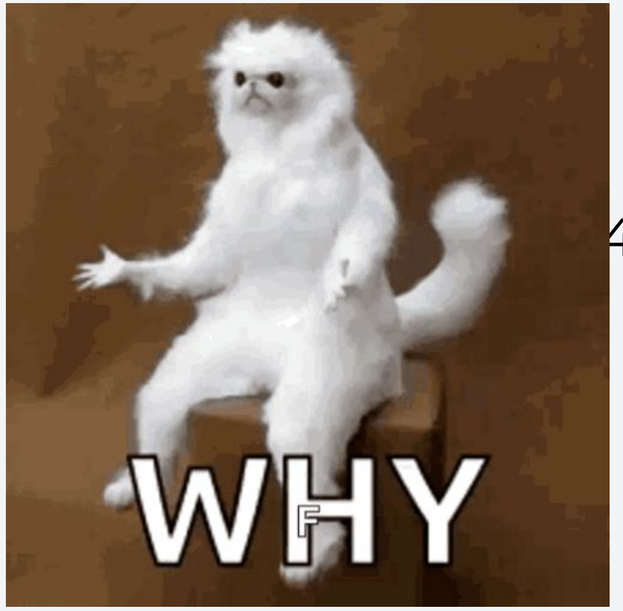
- Average: Still checking
- Min: Still checking
- Max: Still checking

Problem 4

- Average: 10.28
- Min: 0
- Max: 20

Problem 5

- Average: 14.466
- Min: 1
- Max: 21



43.866

- Path-finding Algorithms
- DFS and BFS for Path-finding
- Dijkstra’s Algorithm
- Bellman-Ford Algorithm

- **Path-finding Algorithms**
- DFS and BFS for Path-finding
- Dijkstra’s Algorithm
- Bellman-Ford Algorithm

Recall: Problems solved by Graph Algorithms

A lot of real-life problems are solvable by using graph algorithms. For example,

1. In a computer network, you want to make sure that all computers are connected.
    a. Represent the network as a graph and traverse the graph. This will verify if a certain node is reachable by all the nodes of the network.
2. In a road network, you want to close a certain road for maintenance purposes. You want to ensure that all areas are still accessible.
    a. Similar concept to item 1.
3. In a navigation system, you want to return the shortest path possible.
    a. Represent the road network as a graph with a weighted edge representing the travel time. Traverse the graph to find the shortest path possible using Dijkstra's algorithm.

Example: Is the graph connected?

Example: In a computer network, you want to make sure that all computers are connected.

How do we solve this?

- Represent the network as a graph.
- Perform BFS traversal on the graph.
- If all nodes are visited, the graph is connected indicating that all computers in the network are connected.

But, can graphs find paths from point A to point B?

Example: Using a navigation system, you want to return the shortest path possible from point A to point B.

How do we solve this?

But, can graphs find paths from point A to point B?

Example: Using a navigation system, you want to return the shortest path possible from point A to point B.

Can we use graphs?

But, can graphs find paths from point A to point B?

Example: Using a navigation system, you want to return the shortest path possible from point A to point B.

Of course we can use graphs!

Using Graphs for Path-finding problems

When used for path-finding problems, graph algorithms effectively convert graphs into a search tree. This allows us to look for paths from point A to any point in the graph.

Common path-finding algorithms include:

1. Depth First Search and Breadth First Search
2. Dijkstra’s Algorithm
3. Bellman-Ford Algorithm
4. Many more!

Of course, not all of them are created equal. Each has their own advantages and disadvantages. We’ll explore them bit-by-bit.

Before we start, remember these terms...

Solution: A solution is a path from the starting point to the desired node.

The effectiveness of a solution is measured by the following:

1. **Completeness** - does the algorithm find a solution if there is one?
2. **Optimality** - does the algorithm find the shortest path possible?
3. **Time Complexity** - how long does it take for the algorithm to find the solution?
4. **Space Complexity** - how much memory is needed to find the solution?

We'll compare the different graph algorithms in terms of these metrics.

- Path-finding Algorithms
- **DFS and BFS for Path-finding**
- Dijkstra’s Algorithm
- Bellman-Ford Algorithm

Let’s start with using DFS

Depth-First Search: Find a path from one node to another by exploring each possible path as deep as possible and use backtracking whenever necessary.

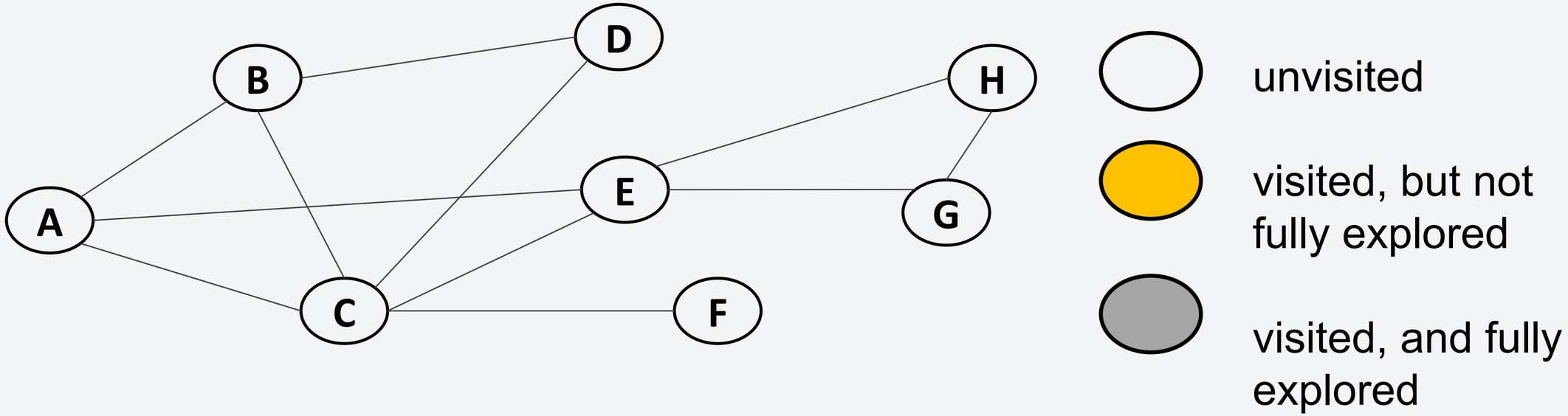
Pseudocode:

```
DFS(start, goal):
   if start is goal:
      return true
   mark start as visited and add it to path
   for each neighbor of start:
      if neighbor is not visited:
         if DFS(neighbor, goal) is true:
               return true
   return false and remove the start node from the path
```

- Algorithm is implemented recursively.
- We store the path using a list to keep track of the current path.
- For each recursive call, we add the node to the path and remove it if the DFS fails from that node.

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

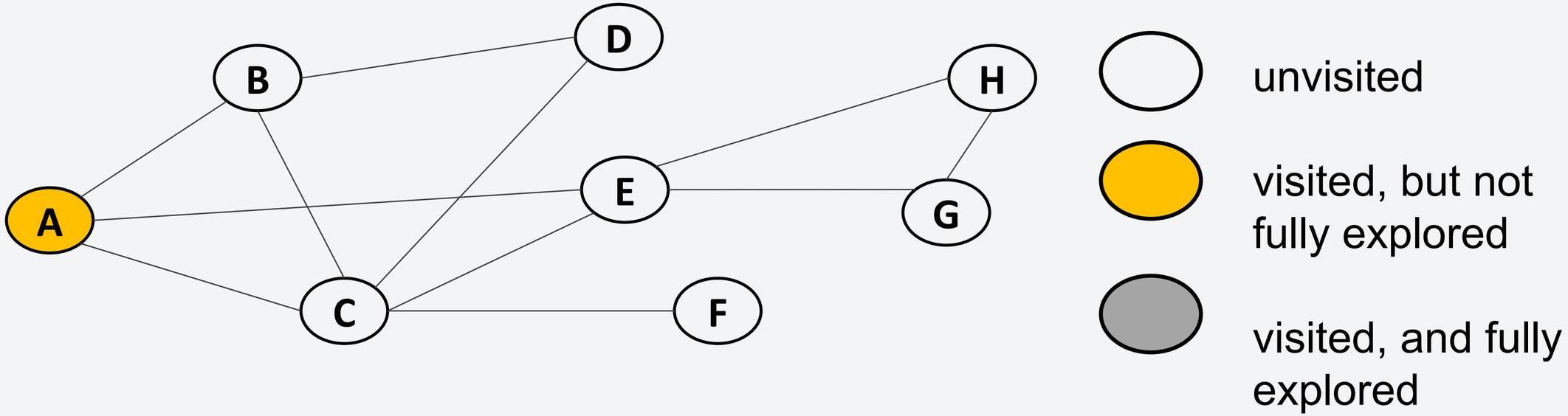


Visited:

Path:

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

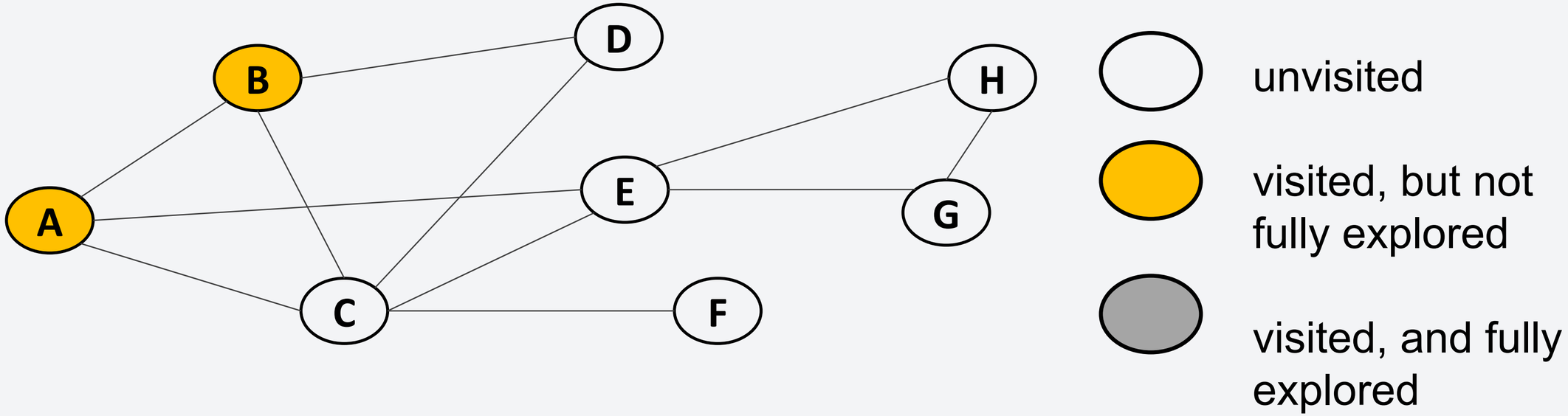


Visited: A

Path: A

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

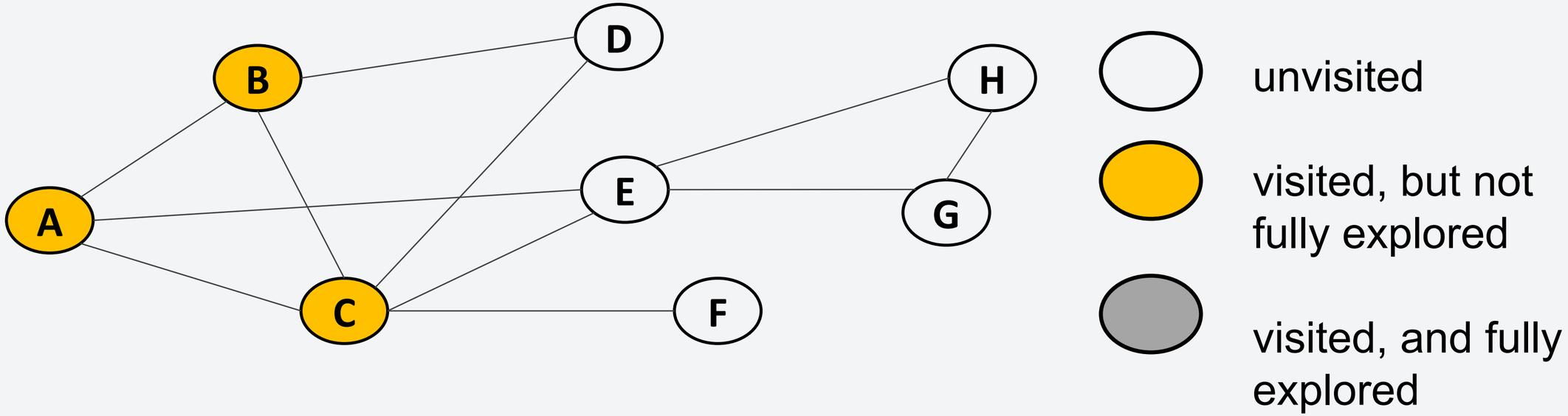


Visited: A B

Path: A B

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

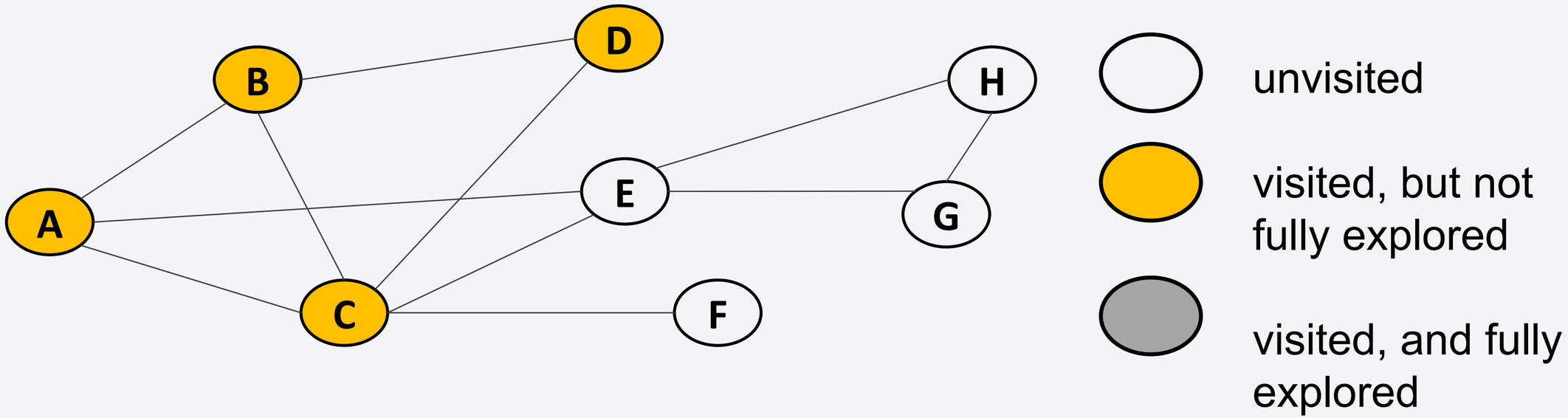


Visited: A B C

Path: A B C

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

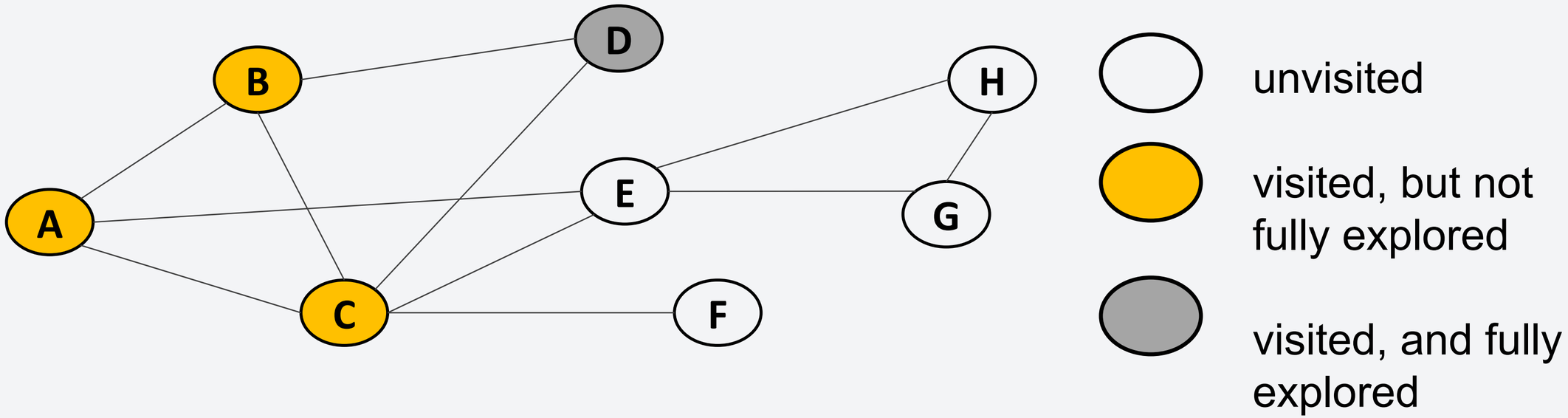


Visited: A B C D

Path: A B C D

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

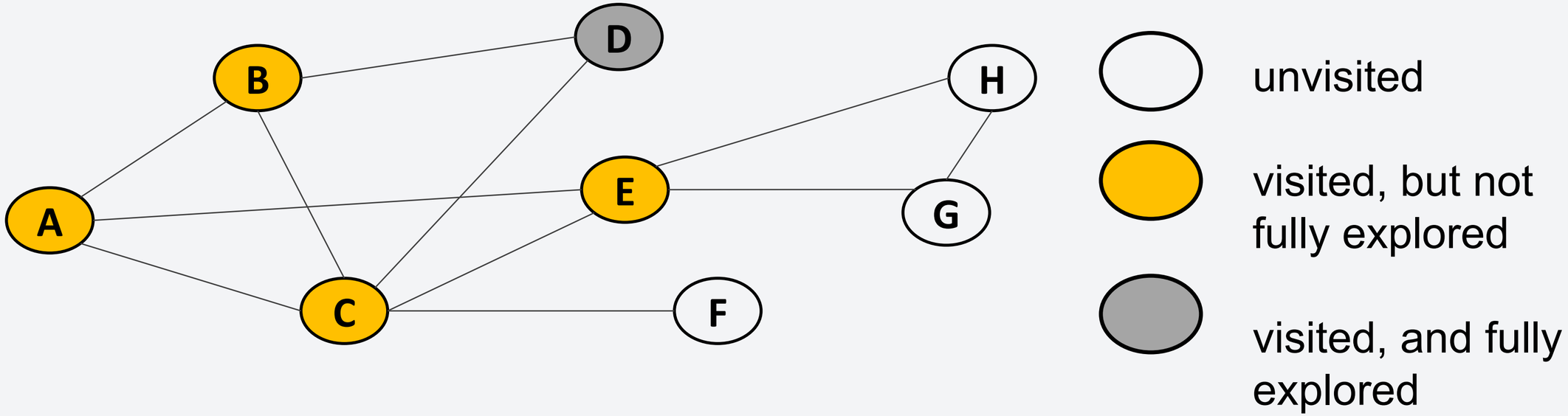


Visited: A B C D

Path: A B C

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

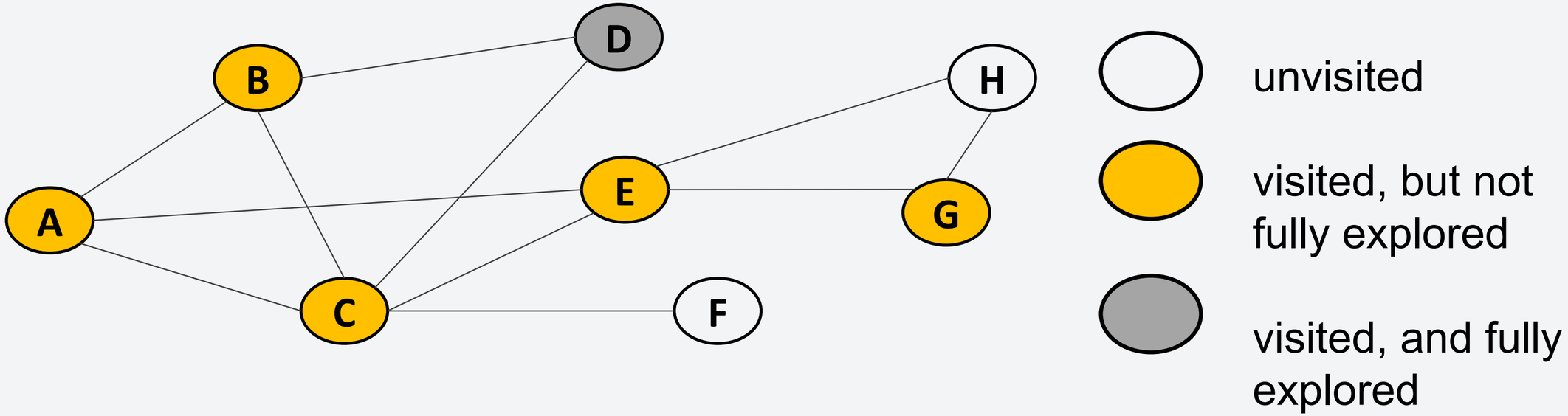


Visited: A B C D E

Path: A B C E

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.

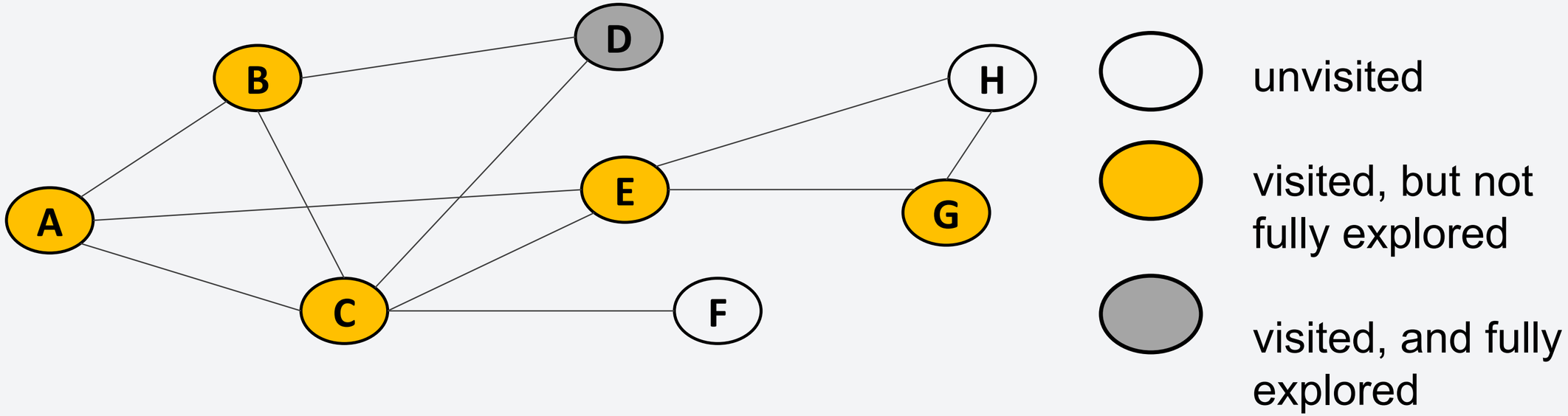


Visited: A B C D E G

Path: A B C E G

Example: Path-finding with DFS

Consider the following unweighted graph. Suppose we want to find the path from A to G.



Visited: A B C D E G

Path: A B C E G

FOUND IT!

Example: Path-finding with DFS

Now what's the problem with DFS? **DFS is neither complete nor optimal**.

- DFS may never terminate if the graph is very complex, having depths that are very large.
- DFS returns a solution, but is not guaranteed to be the shortest path possible.
- The time complexity of DFS is $O(b^m)$ where b is the branching factor of the graph and m is the maximum depth of the graph.
- The space complexity of DFS is $O(b^m)$ as well.

Because of these, we must avoid using DFS especially for large and complex graphs.

What about BFS?

Breadth-First Search: Find a path from one node to another by exploring neighboring nodes of the same depth first before proceeding to the next level.

Compared to DFS, using BFS for path-finding allows us to find the shortest path possible from point A to another point in the graph.

- We explore lower levels first before going to deeper levels ensuring that we can find a shortest path.

What about BFS?

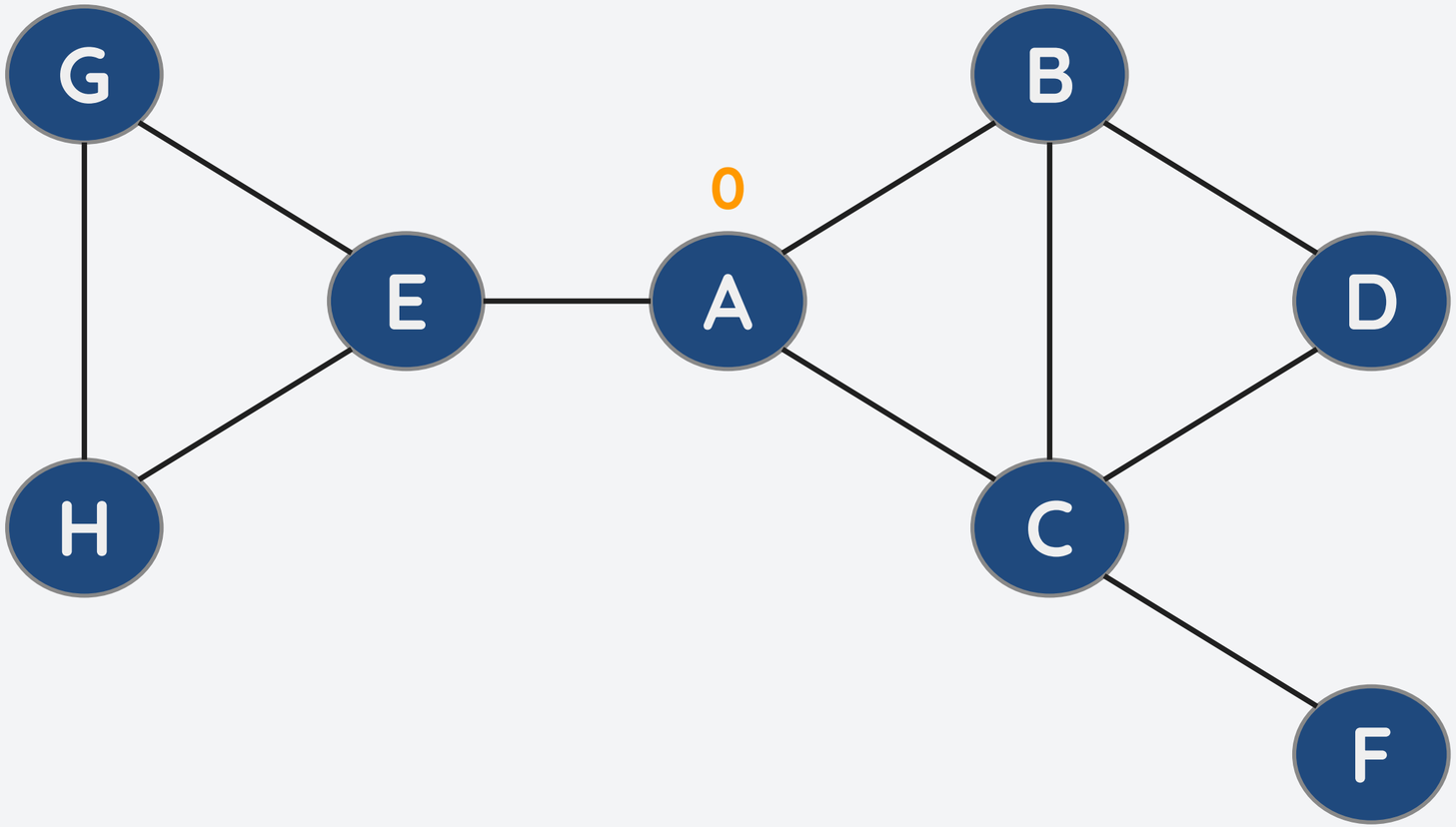
Pseudocode:

```
BFS(start, goal):
   create a queue
   add start to queue
   mark start as visited
   while queue is not empty:
      node = remove from queue
      if node is goal:
         return true
      for each neighbor of node:
         if neighbor is not visited:
               add neighbor to queue
               mark neighbor as visited
   return false
```

- Algorithm is implemented using a queue.
- We already know this from Lecture 6.
- Question is: How do we get the path from the starting point to end point?
  - We use what we call a **predecessor array**

Example: Path-finding with BFS

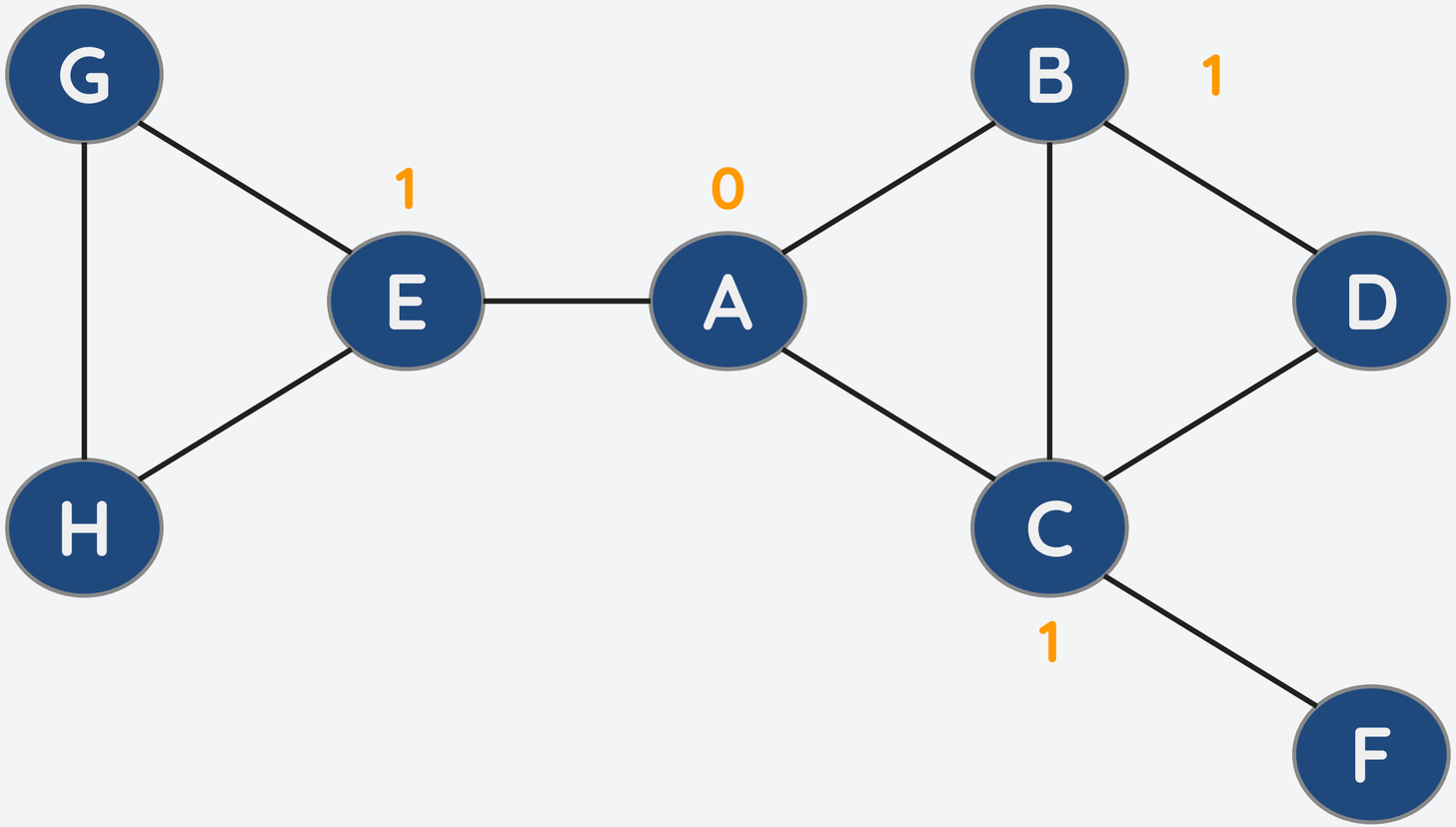
Consider the following unweighted graph. Suppose we want to find the path from A to F.



- We can use a BFS traversal to find the shortest path.
- For each iteration, we store the information about how many edges we need to go from node A to each of the nodes.

Example: Path-finding with BFS

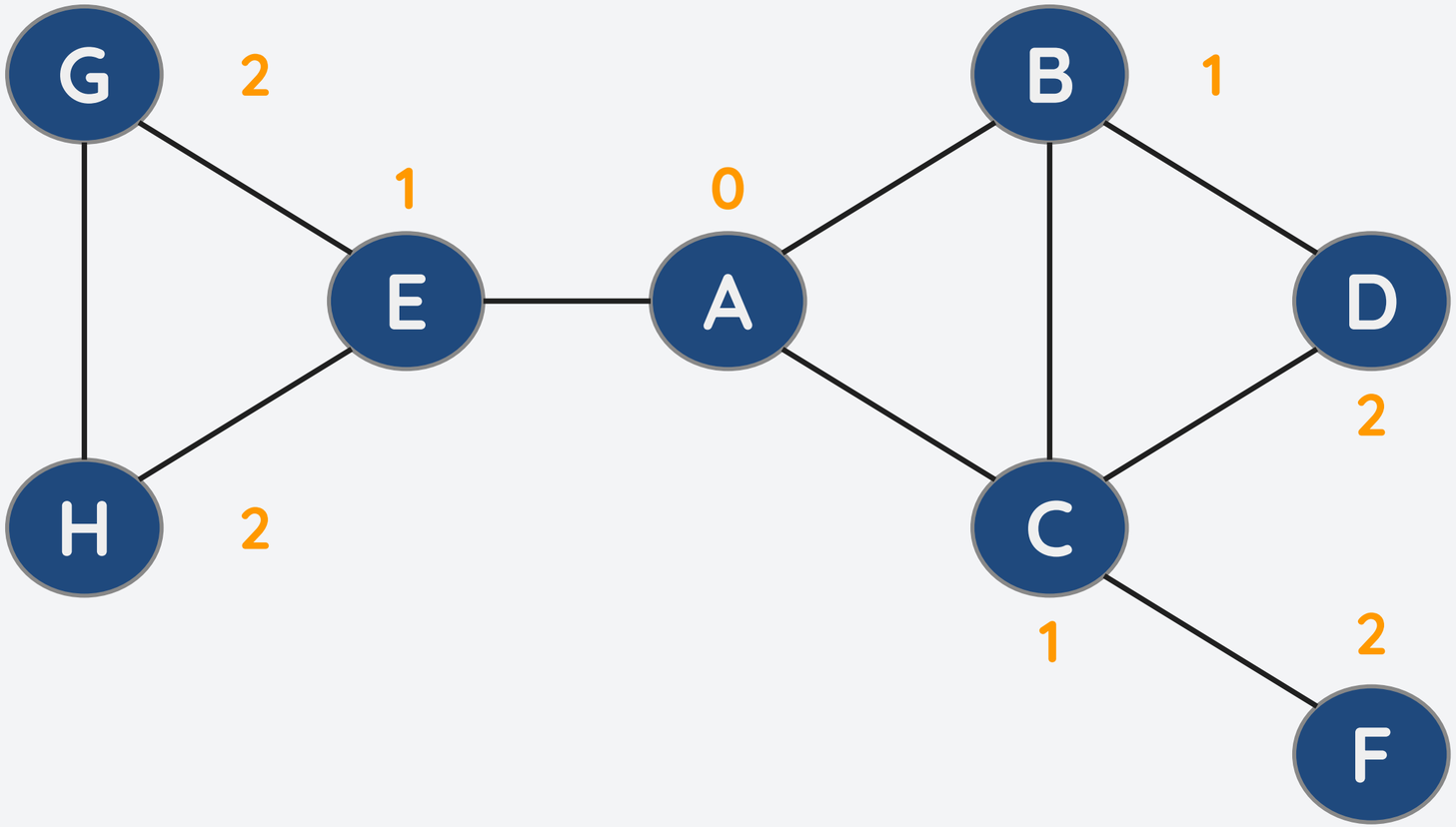
Consider the following unweighted graph. Suppose we want to find the path from A to F.



- We can use a BFS traversal to find the shortest path.
- For each iteration, we store the information about how many edges we need to go from node A to each of the nodes.

Example: Path-finding with BFS

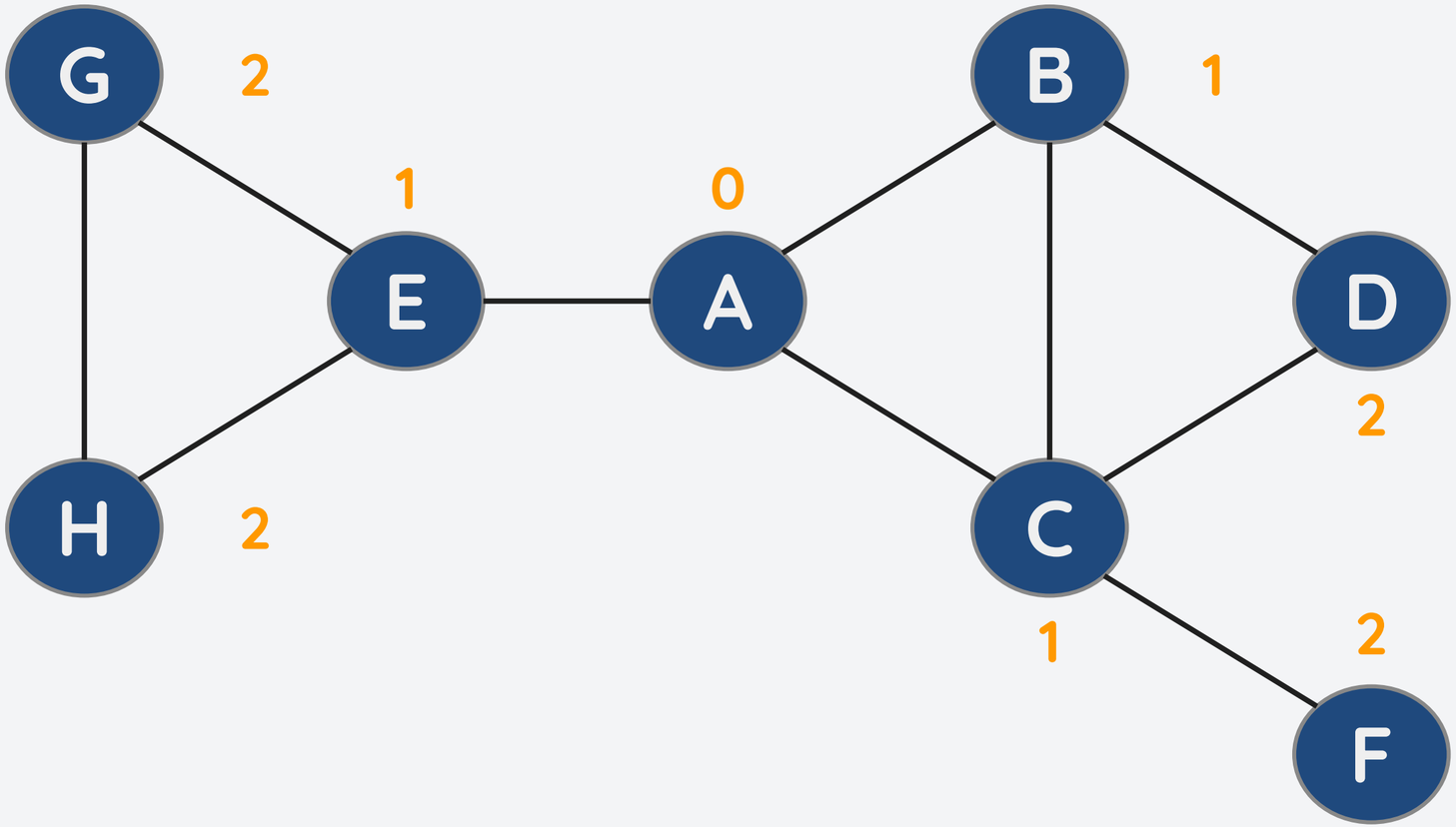
Consider the following unweighted graph. Suppose we want to find the path from A to F.



- We can use a BFS traversal to find the shortest path.
- For each iteration, we store the information about how many edges we need to go from node A to each of the nodes.
- After all the iterations, we now have a BFS tree with A as the root node.

Example: Path-finding with BFS

Consider the following unweighted graph. Suppose we want to find the path from A to F.

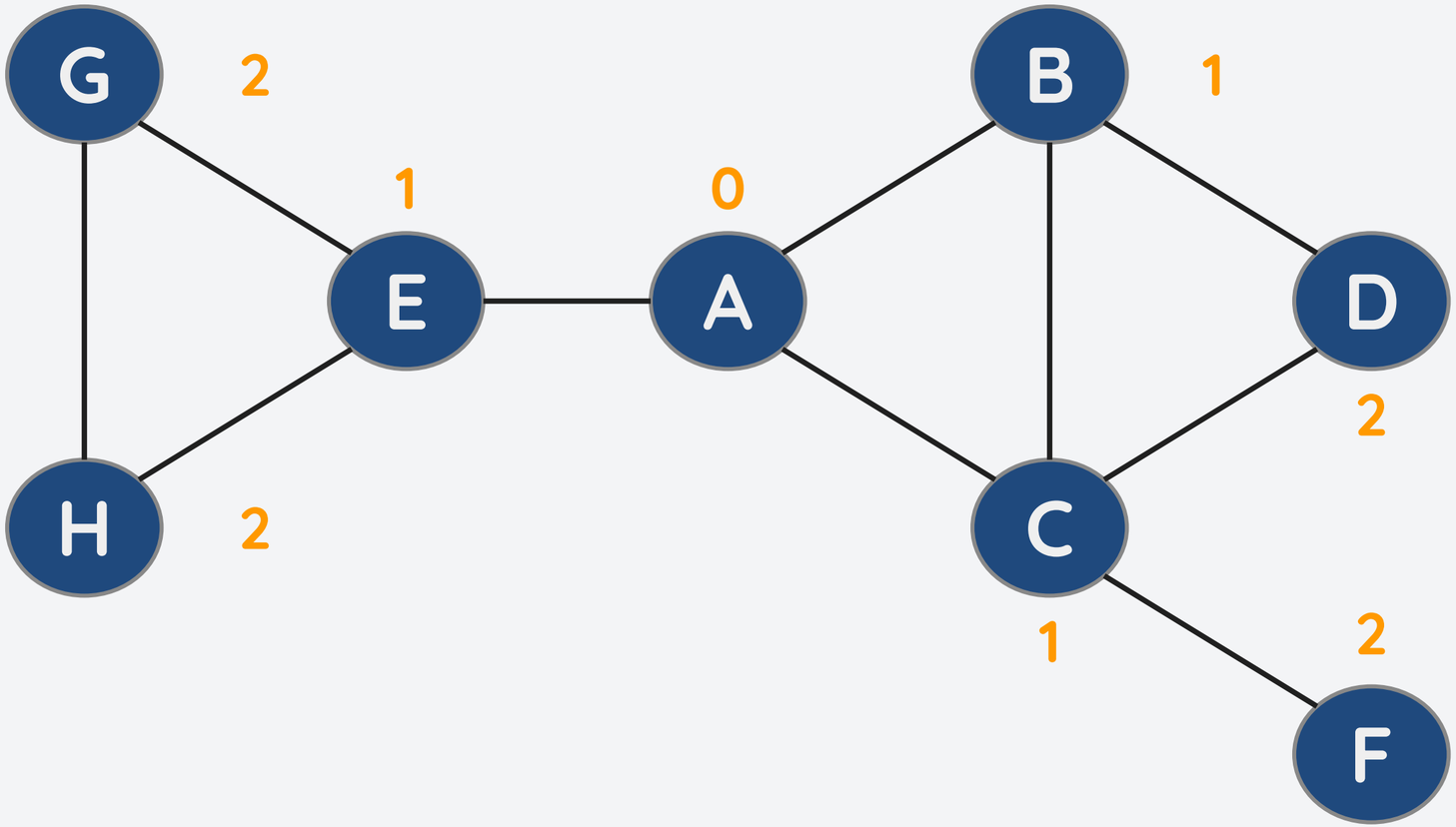


- After obtaining the BFS tree, we can find the shortest path by traversing this tree.

But how?

Example: Path-finding with BFS

Consider the following unweighted graph. Suppose we want to find the path from A to F.



- After obtaining the BFS tree, we can find the shortest path by traversing this tree.

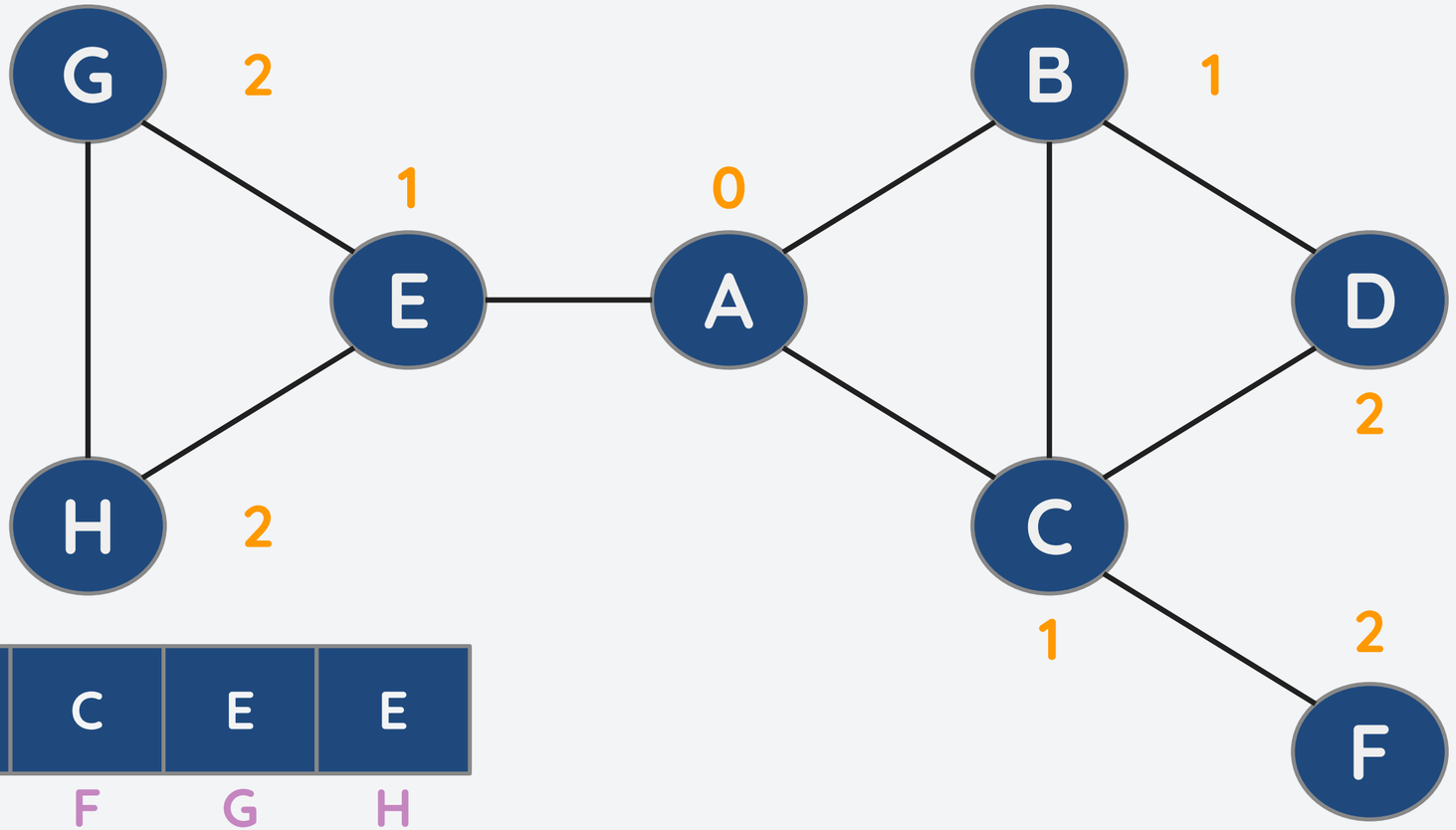
But how?

We use the Predecessor Array

Example: Path-finding with BFS

Consider the following unweighted graph. Suppose we want to find the path from A to F.

- Each index of the array corresponds to the vertex.
- Starting from the destination node, look at the predecessor array and traverse backwards until the source node is reached.

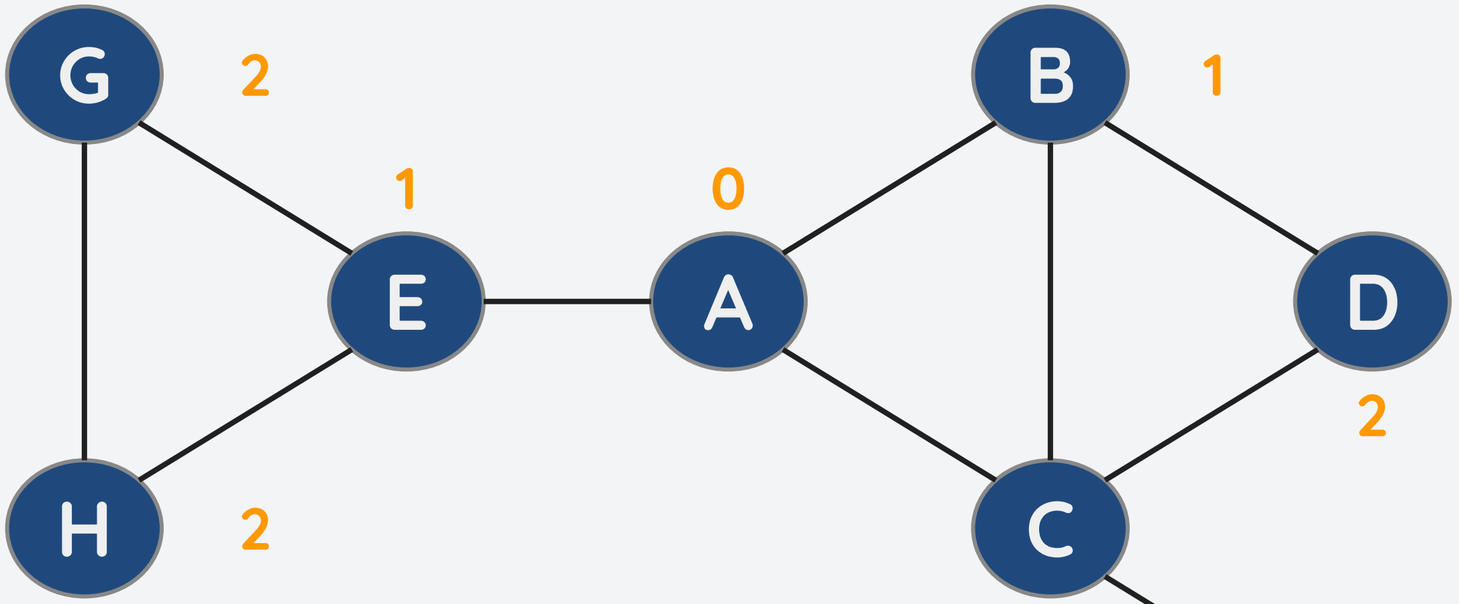


pre[v]	-	A	A	B	A	C	E	E
v	A	B	C	D	E	F	G	H

Example: Path-finding with BFS

Consider the following unweighted graph. Suppose we want to find the path from A to F.

- At index of F, pre[F] contains C.
- At index of C, pre[C] contains A.
- Thus, path = A -> C -> F



v	A	B	C	D	E	F	G	H
pre[v]	-	A	A	B	A	C	E	E

Some caveats with BFS

BFS is slightly better than DFS. How?

- If there is a solution, BFS is guaranteed to find it. Thus, BFS is **complete**.
- If there are several paths, the shallowest path will always be found first. Thus, the BFS is **optimal**.

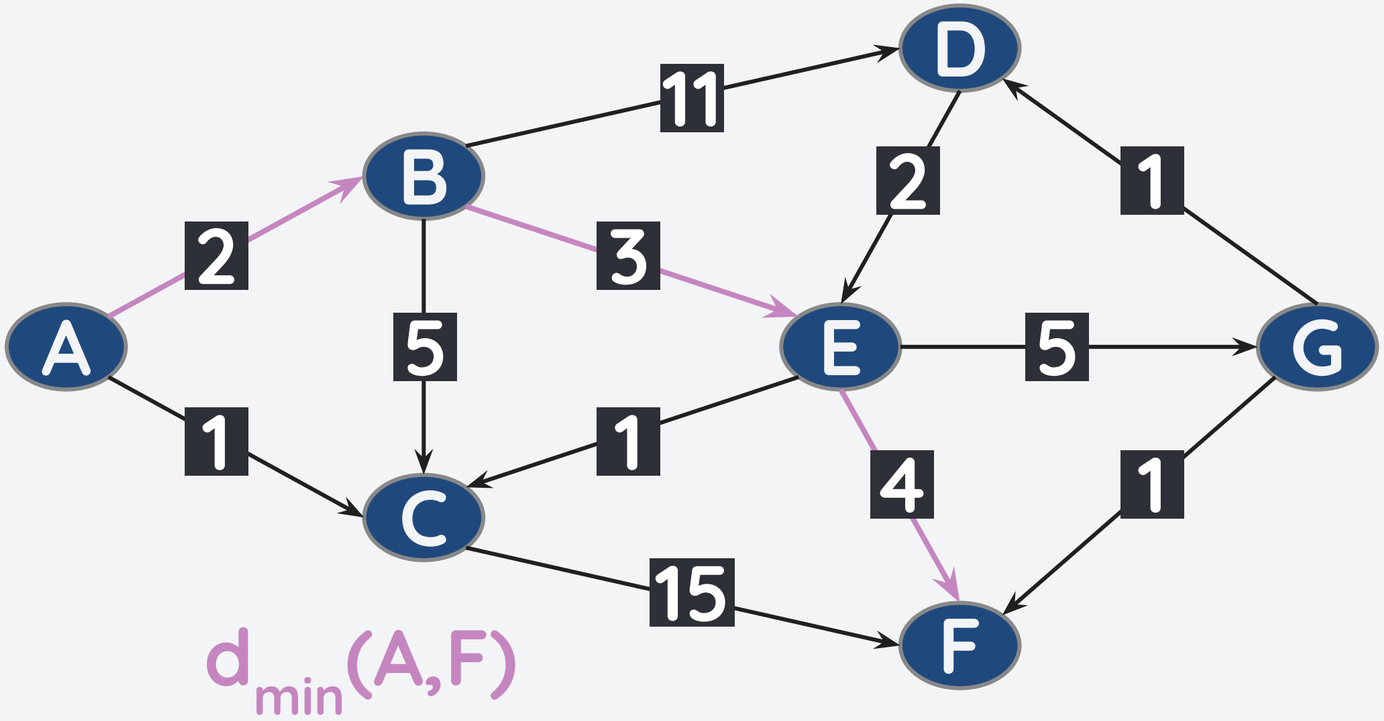
However, **BFS is slower and consumes more memory than DFS** most of the time. But let’s not delve towards that for now.

Additionally, DFS and BFS can only determine the shortest path in terms of the numbers of edges i.e. **usable only for unweighted graphs**. Most real life applications use weighted graphs.

- Path-finding Algorithms
- DFS and BFS for Path-finding
- **Dijkstra’s Algorithm**
- Bellman-Ford Algorithm

Single Source Single Target Shortest Path

In reality, problems represented as graphs use weighted edges. But how can we then find the shortest path from one node to another?

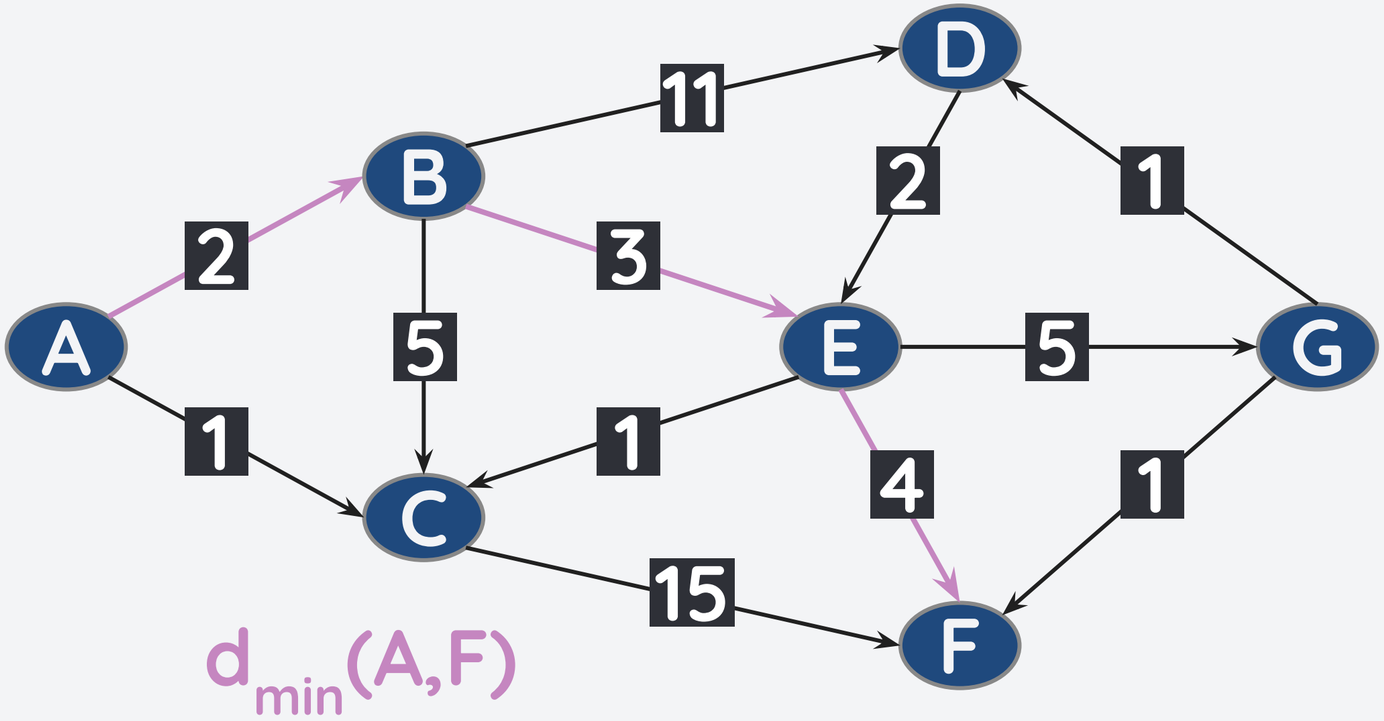


v	d(s,v)	pre[v]
A	0	-
B	2	A
C	-	-
D	-	-
E	5	B
F	9	E
G	-	-

Intuitively, we can try all possible paths. But this might be too long to compute.

Single Source Single Target Shortest Path

In reality, problems represented as graphs use weighted edges. But how can we then find the shortest path from one node to another?

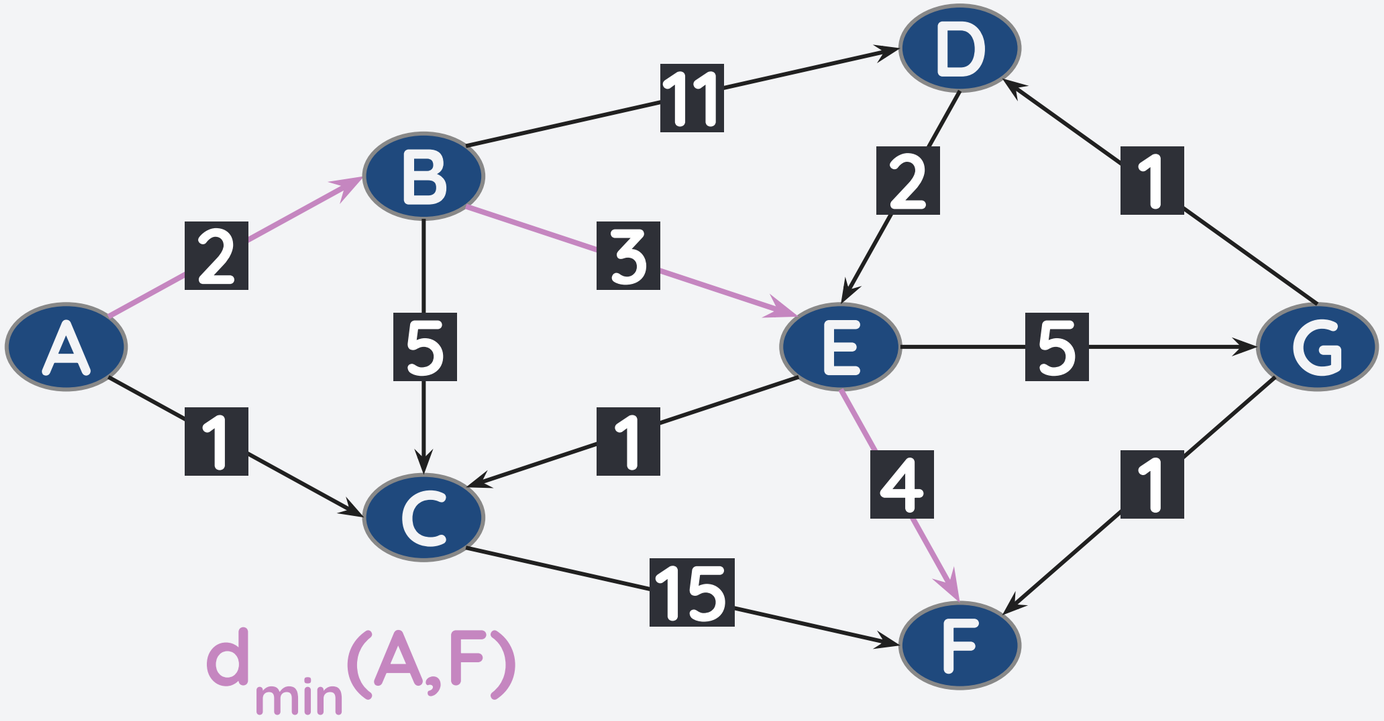


v	d(s,v)	pre[v]
A	0	-
B	2	A
C	-	-
D	-	-
E	5	B
F	9	E
G	-	-

However, one key observation is that **the shortest path have no cycles**.

Single Source Single Target Shortest Path

In reality, problems represented as graphs use weighted edges. But how can we then find the shortest path from one node to another?

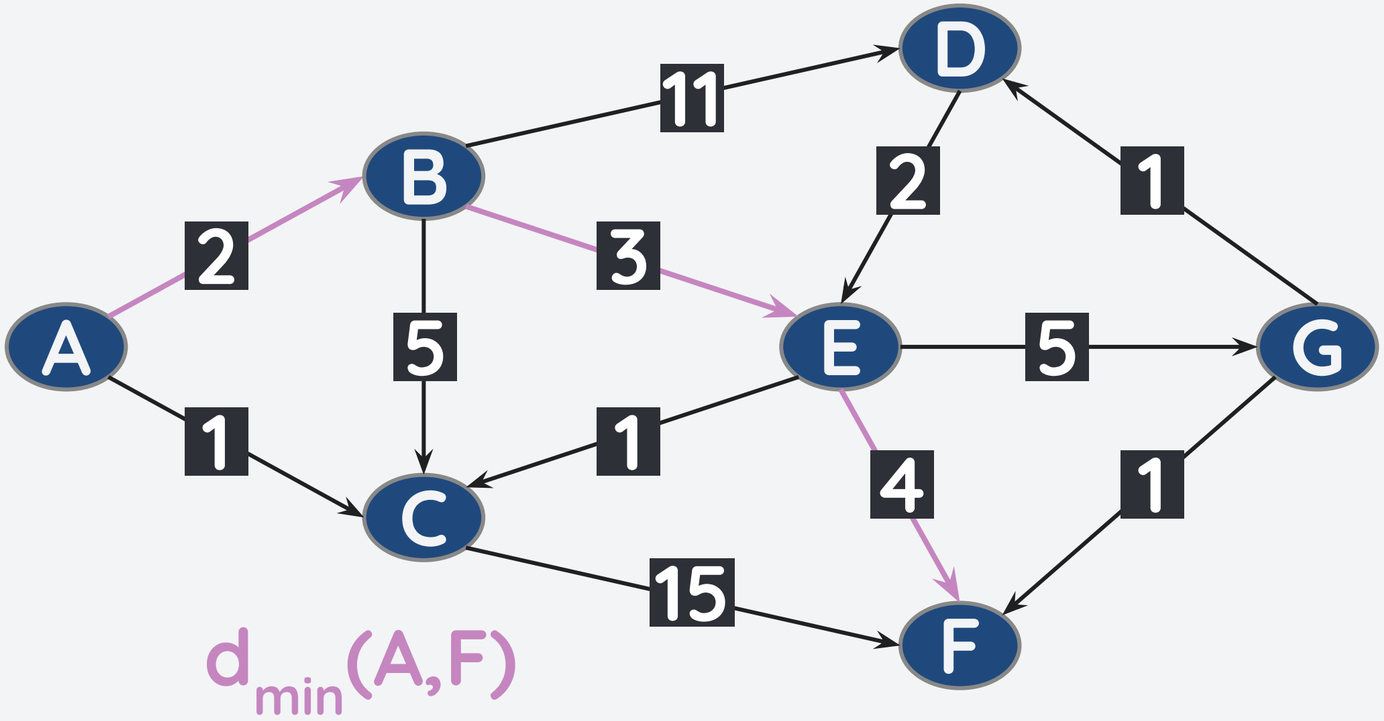


v	d(s,v)	pre[v]
A	0	-
B	2	A
C	-	-
D	-	-
E	5	B
F	9	E
G	-	-

When traversing a graph, a **walk is a sequence of consecutive vertices and edges**.

Single Source Single Target Shortest Path

In reality, problems represented as graphs use weighted edges. But how can we then find the shortest path from one node to another?

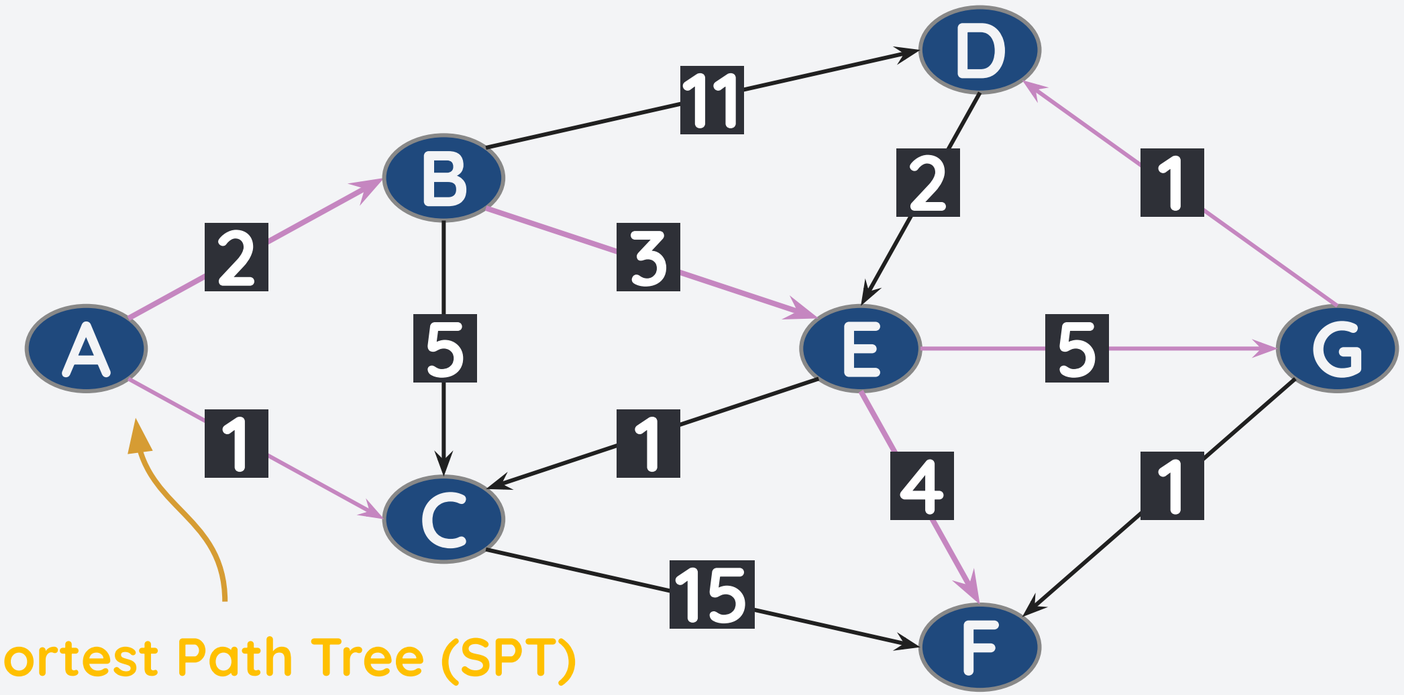


v	d(s,v)	pre[v]
A	0	-
B	2	A
C	-	-
D	-	-
E	5	B
F	9	E
G	-	-

On the other hand, a **path is a walk where no repeated vertices and edges occur.**

Single Source Shortest Path

What about if we want to find the shortest path for all nodes given a starting point?



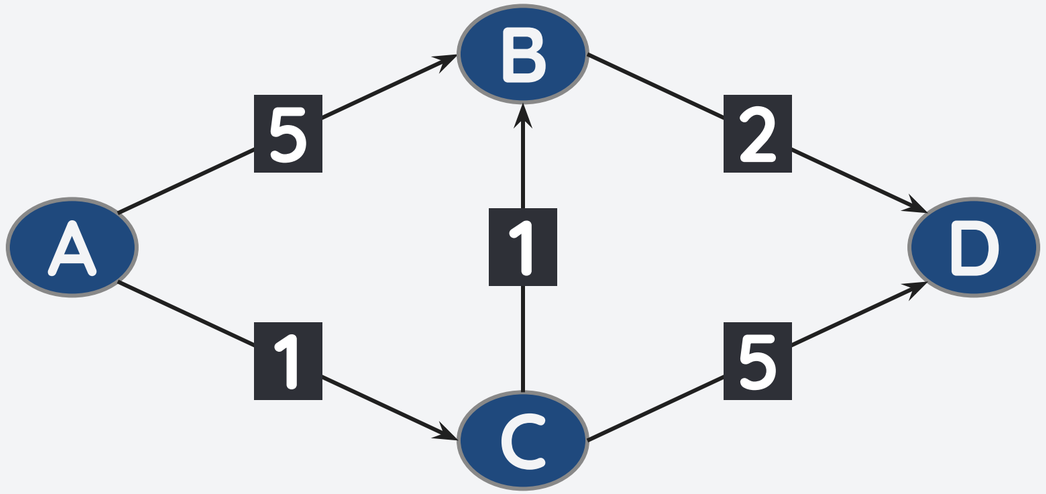
Shortest Path Tree (SPT)

v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E

If we try to find the shortest path for all nodes in a graph given a starting point, we see that the shortest paths form a tree.

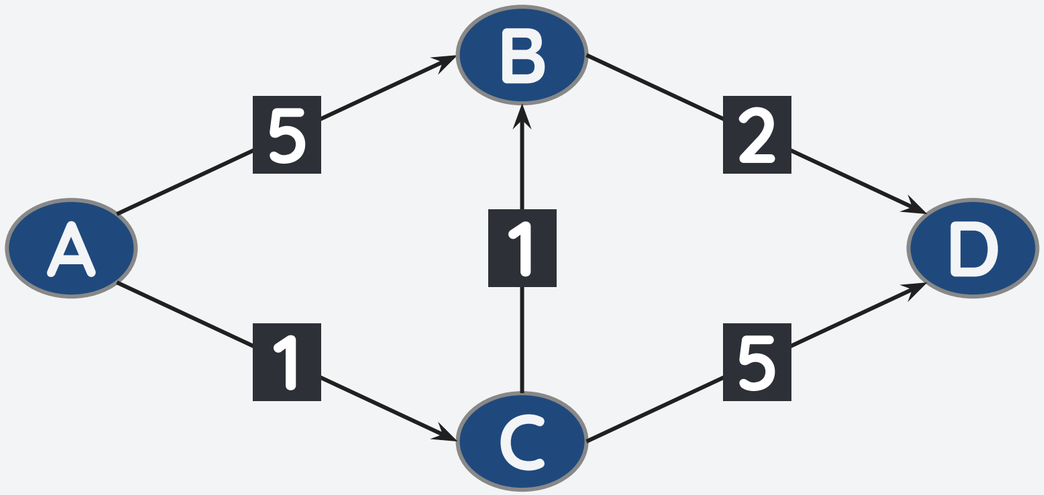
But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?



But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?



Formally, we define the shortest path as:

$$d(s,t) = \min(d(s,x)+w(x,t)),\ x \in V \setminus \{s,t\}$$

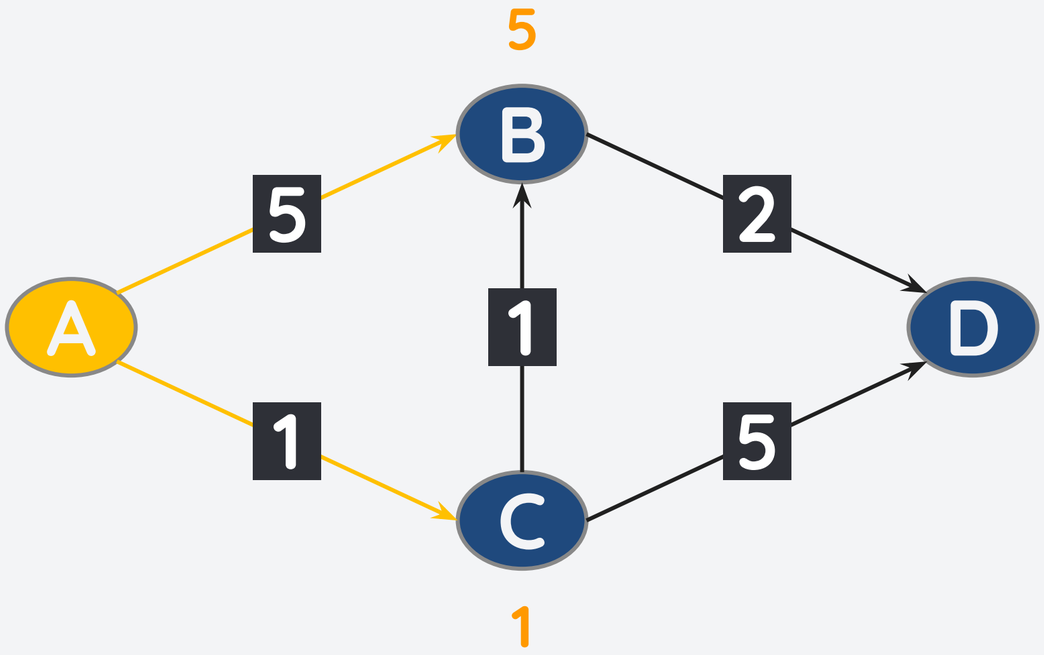
$$d(s,s) = 0$$

Interpretation:

- d(s,x) + w(x,t) is the distance from node s to t where x is a node between s and t.
- We want to find the minimum distance by looking at all the paths between s and t.

But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?

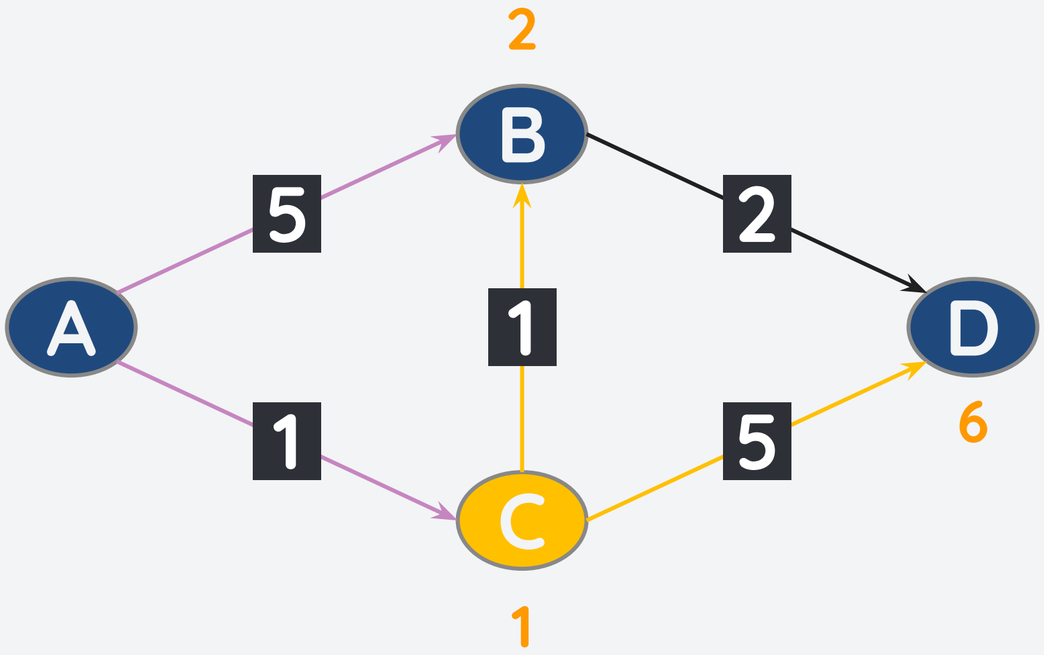


We can visit the node with the best known distances first:

- From A, we try to visit B and C.
- d(A,A) = 0
- d(A,B) = 5
- d(A,C) = 1

But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?

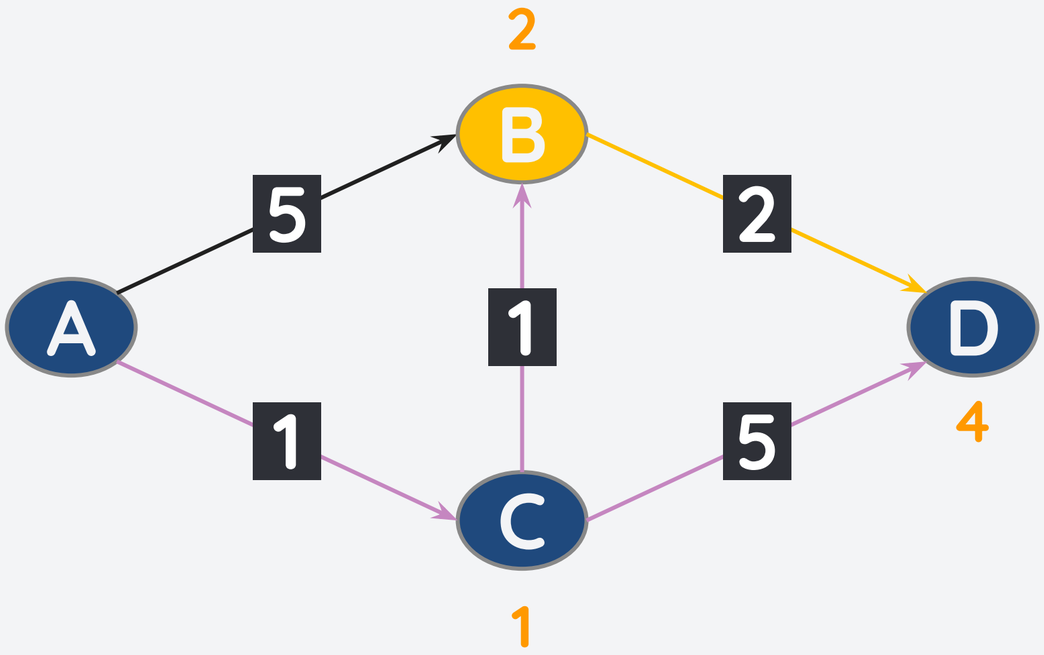


We can visit the node with the best known distances first:

- Since C has the current shortest path, we visit C.
- d(A,A) = 0
- **d(A,B) = min(d(A,B), d(A,C) + w(C,B)) = 2**
- d(A,C) = 1
- d(A,D) = d(A,C) + w(C,D) = 1+5 = 6

But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?

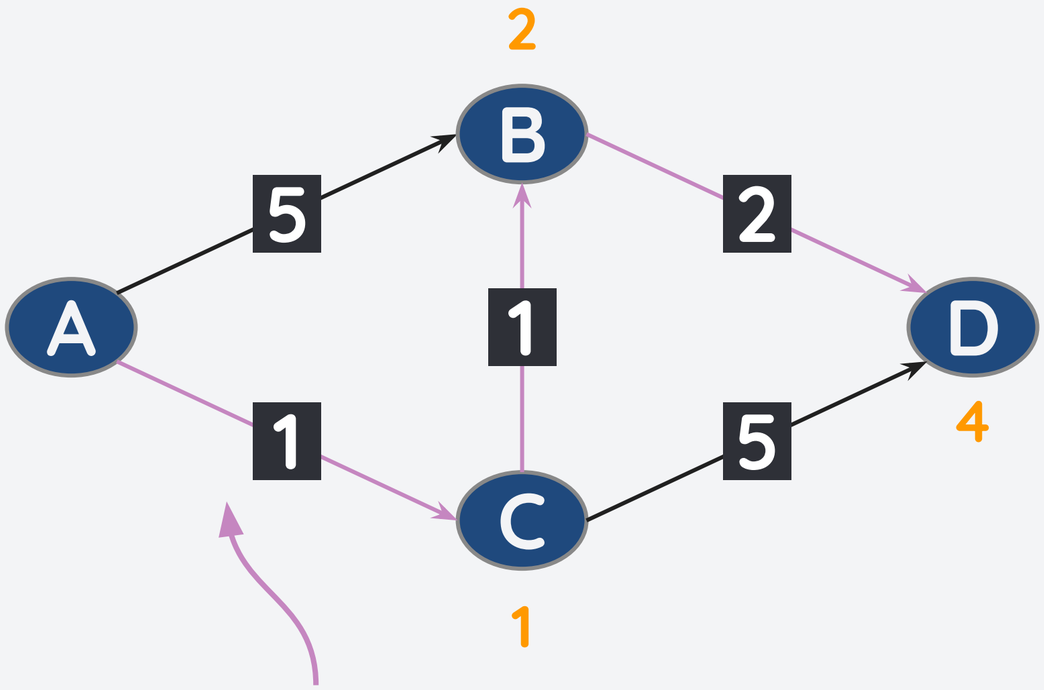


We can visit the node with the best known distances first:

- From C, we can then visit B.
- d(A,A) = 0
- d(A,B) = 2
- d(A,C) = 1
- **d(A,D) = min(d(A,C) + w(C,D), d(A,B) + w(B,D)) = 2+2 = 4**

But how do we find the Shortest Path Tree (SPT)?

Consider the graph below. What is the shortest path tree with A as the source?



We can visit the node with the best known distances first:

- d(A,A) = 0
- d(A,B) = 2
- d(A,C) = 1
- d(A,D) = 4

This is the Dijkstra's Algorithm!

Dijkstra’s Algorithm for the Shortest Path Problem

Dijkstra’s algorithm uses a priority queue to enqueue each path (compared to a queue as used by BFS). This allows us to choose the one with the shortest cost first.

Pseudocode:

For each node in the graph:

- create distance array from start node **d_s[v]** initially all ∞ except node **s** (**0**)
- push the start node to a priority queue

While the min priority queue is not empty:

- pop the node **x** with min priority and **mark it as visited**
- for each **unvisited** neighbor **y** of the popped node:
  - if $d_s[x] + w(x,y) < d_s[y]$:
    - $d_s[y] = d_s[x] + w(x,y)$
    - pre[y] = x
    - push y onto the priority queue with $d_s[y]$ as its priority

Dijkstra’s Algorithm for the Shortest Path Problem

Dijkstra’s algorithm uses a priority queue to enqueue each path (compared to a queue as used by BFS). This allows us to choose the one with the shortest cost first.

Pseudocode:

For each node in the graph:

- create distance array from start node **$d_s[v]$** initially all ∞ except node **s** (**0**)
- push the start node to a priority queue

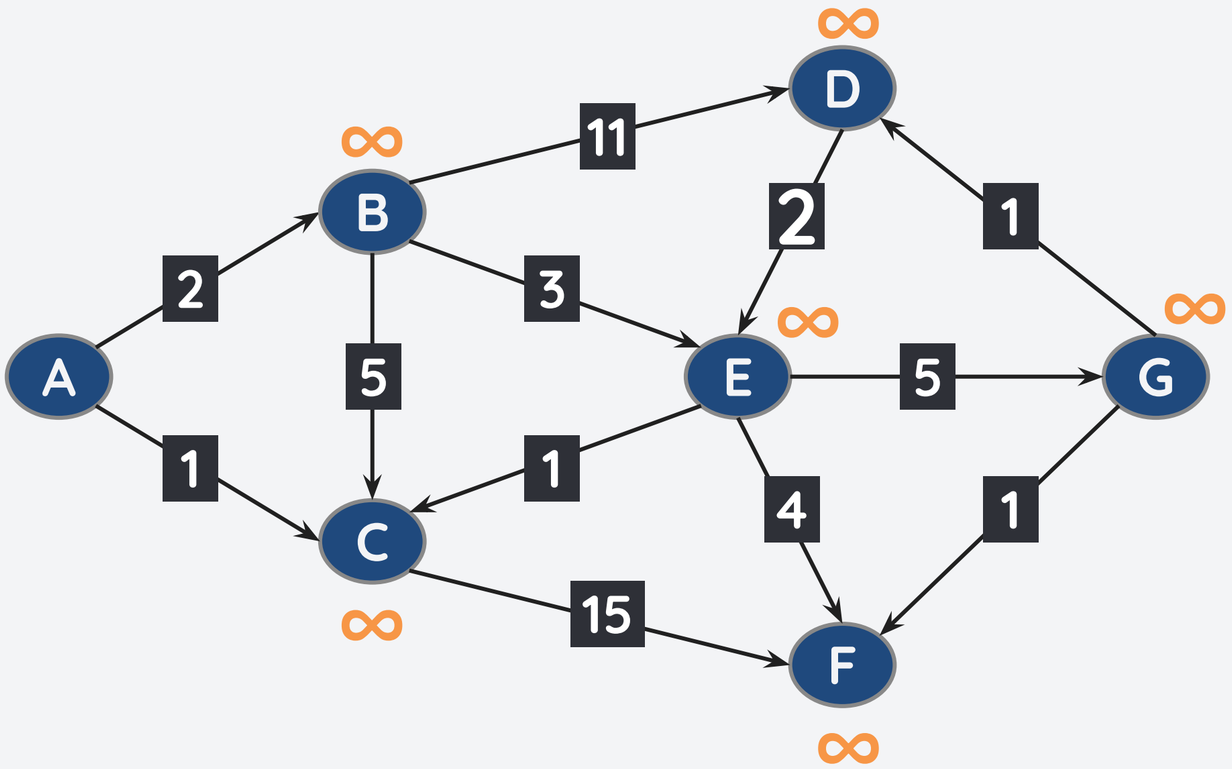
While the min priority queue is not empty:

- pop the node **x** with min priority and **mark it as visited**
- for each **unvisited** neighbor **y** of the popped node:
  - if $d_s[x] + w(x,y) < d_s[y]$:
    - $d_s[y] = d_s[x] + w(x,y)$
    - pre[y] = x
    - push y onto the priority queue with $d_s[y]$ as its priority

Edge relaxation - the process of updating the cost of a path

Example: Finding the SPT using Dijkstra's Algo

Consider the weighted graph below. Let's find the SPT with A as the root node.



v	$d_s[v]$	pre[v]
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

Example: Finding the SPT using Dijkstra's Algo

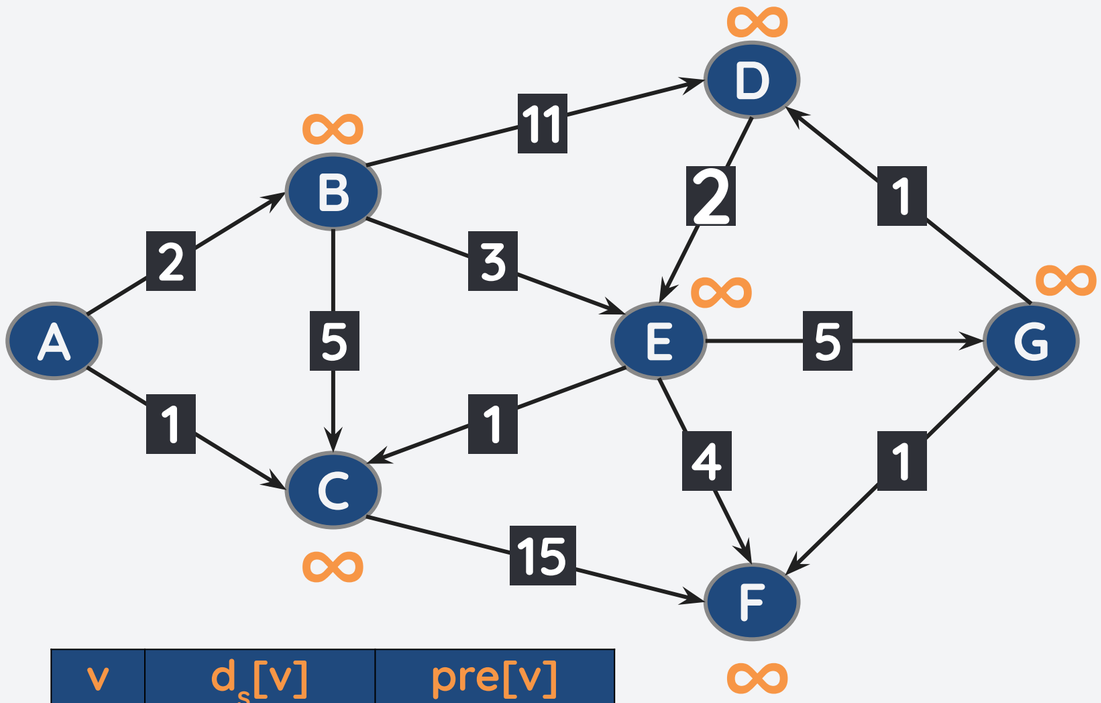
For each node in the graph:

- **create distance array from start node $d_s[v]$ initially all ∞ except node s (0)**
- **push the start node to a priority queue**

While the min priority queue is not empty:

- pop the node **x** with min priority and mark as visited
- for each **unvisited** neighbor **y** of the popped node:
  - if $d_s[x] + w(x,y) < d_s[y]$:
    - $d_s[y] = d_s[x] + w(x,y)$
    - pre[y] = x
    - push y onto the priority queue with $d_s[y]$ as its priority

PQ: [(A,0)]



v	$d_s[v]$	pre[v]
A	0	-
B	∞	-
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

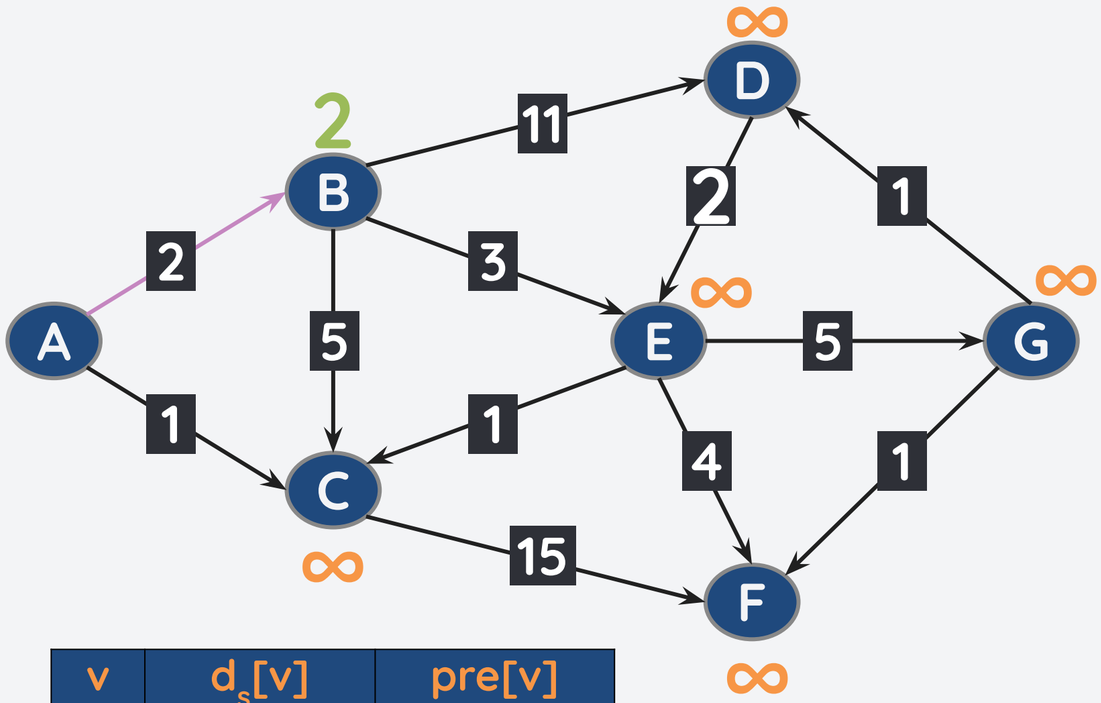
- **pop the node x with min priority and mark as visited**
- **for each unvisited neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [(B,2)]

Neighbors: B and C

$d_s[A] + w(A,B) = 0+2 = 2 < ∞$. Update $d_s[B] = 2$, pre[B] = A.

Push onto PQ.



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	∞	-
D	∞	-
E	∞	-
F	∞	-
G	∞	-

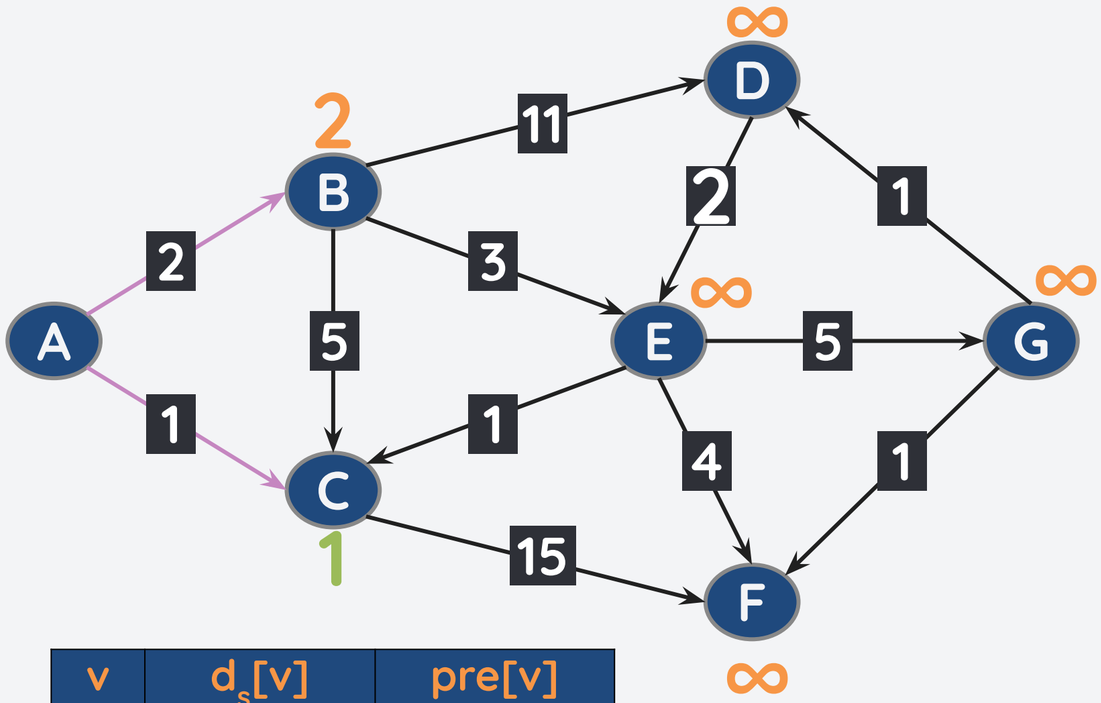
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-

PQ: [(C,1), (B,2)]

Neighbors: B and C

$d_s[A] + w(A,C) = 0+1 = 1 < ∞$. Update $d_s[C] = 1$, pre[C] = A. Push onto PQ.

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

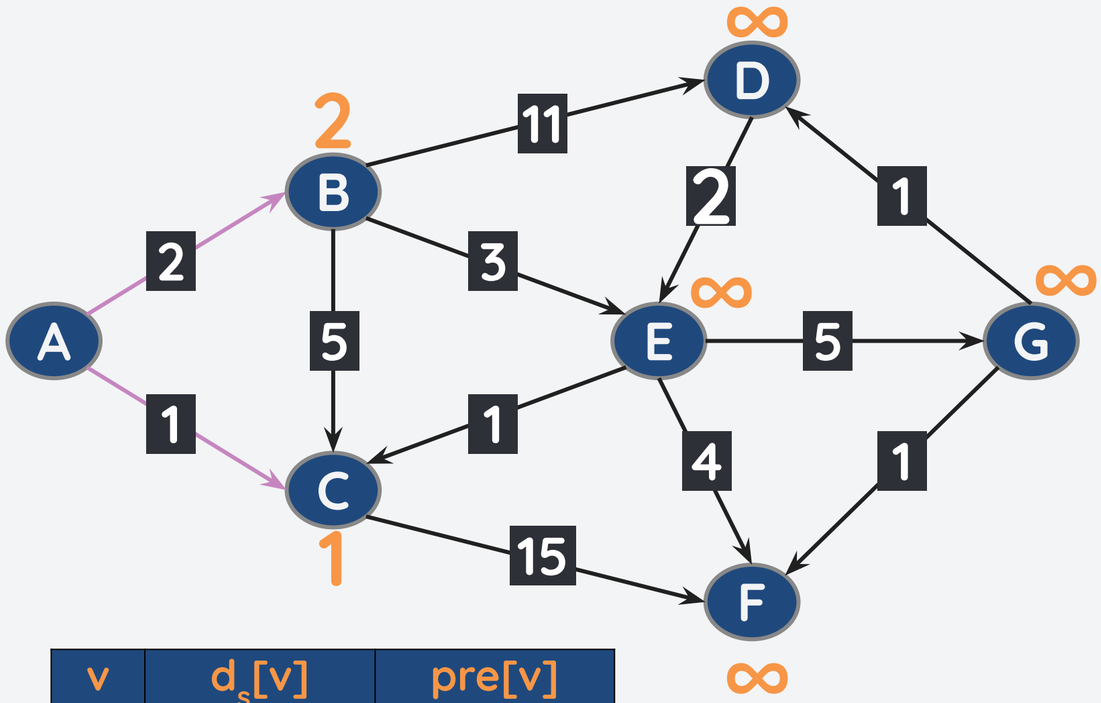
- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [**(C,1)**,(B,2)]

Pop (C,1). Neighbors of C: F



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	∞	-
G	∞	-

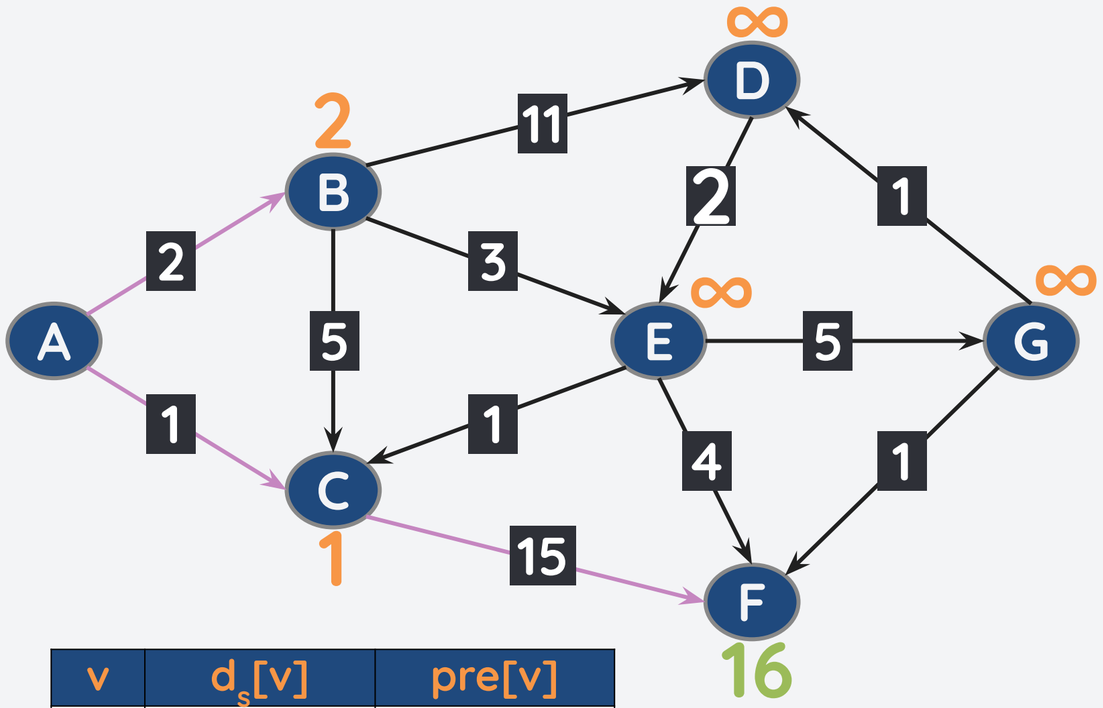
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-

PQ: [(B,2),**(F,16)**]

Pop (C,1). Neighbors of C: F

$d_s[C] + w(C,F) = 1+15 = 16 < ∞$. Update $d_s[F] = 16$, pre[F] = C.

Push onto PQ.

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

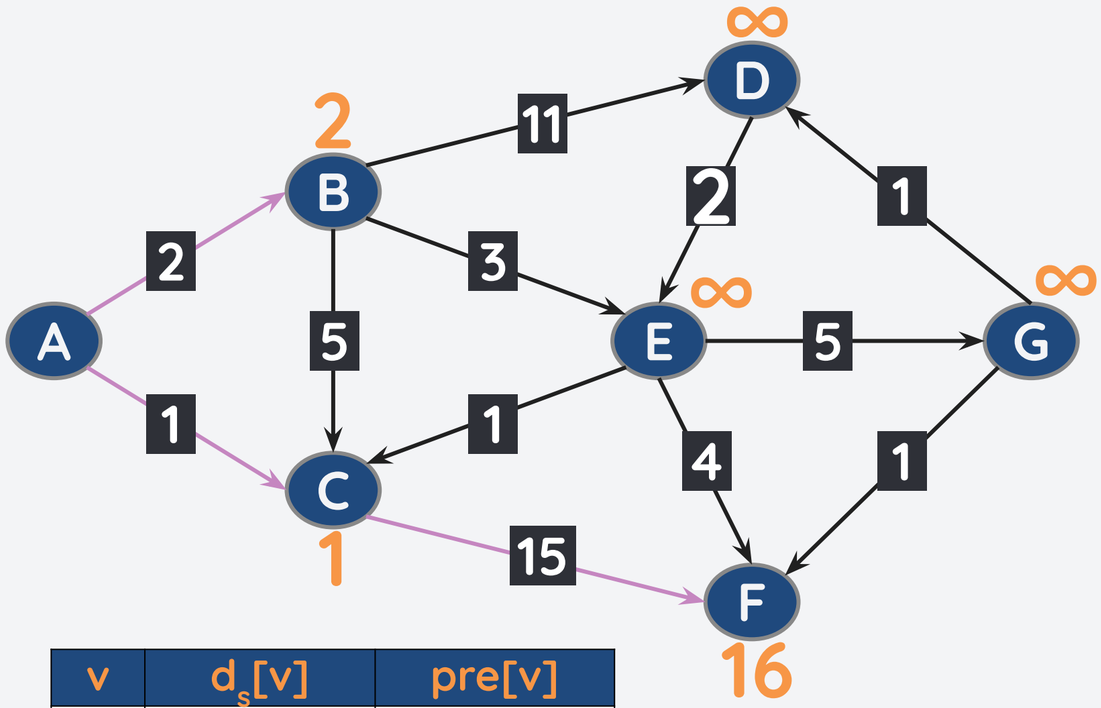
While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [**(B,2)**,(F,16)]

Pop (B,2). Neighbors of B: C, D, and E

$d_s[B] + w(B,C) = 2+5 = 7 > 1$. **Do nothing.**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	∞	-
E	∞	-
F	16	C
G	∞	-

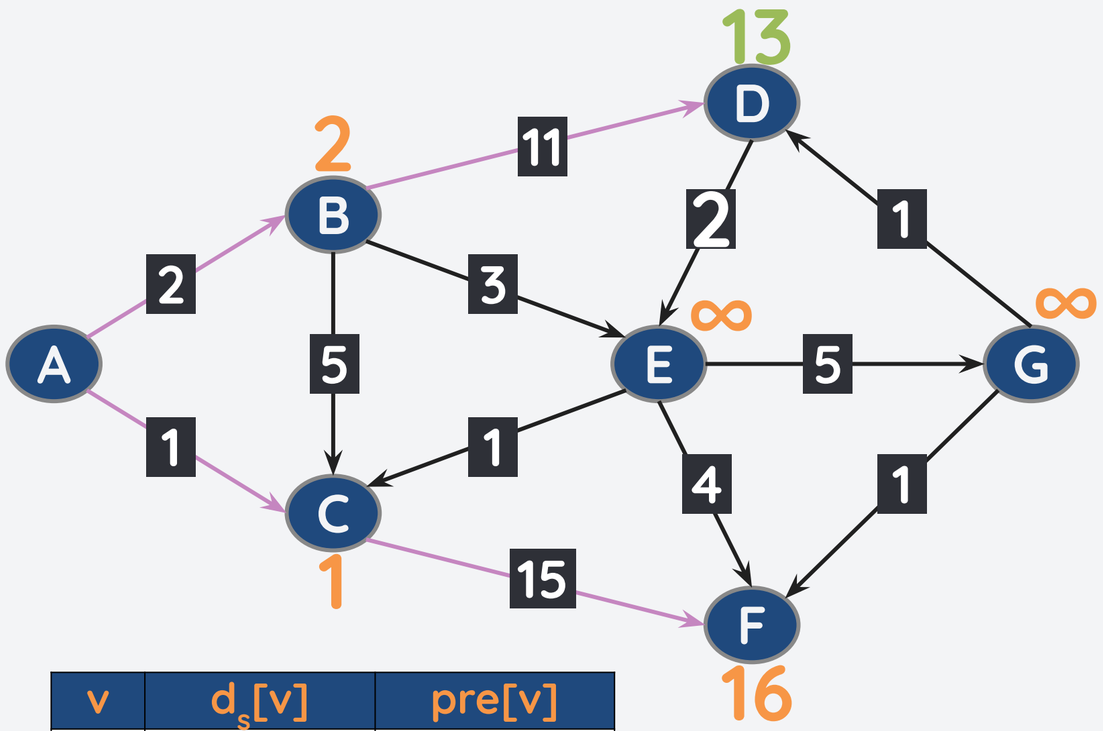
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	∞	-
F	16	C
G	∞	-

PQ: [(D,13),(F,16)]

Pop (B,2). Neighbors of B: D and E

$d_s[B] + w(B,D) = 2+11 = 13 < ∞$. Update $d_s[D] = 13$, pre[D] = B.

Push onto PQ.

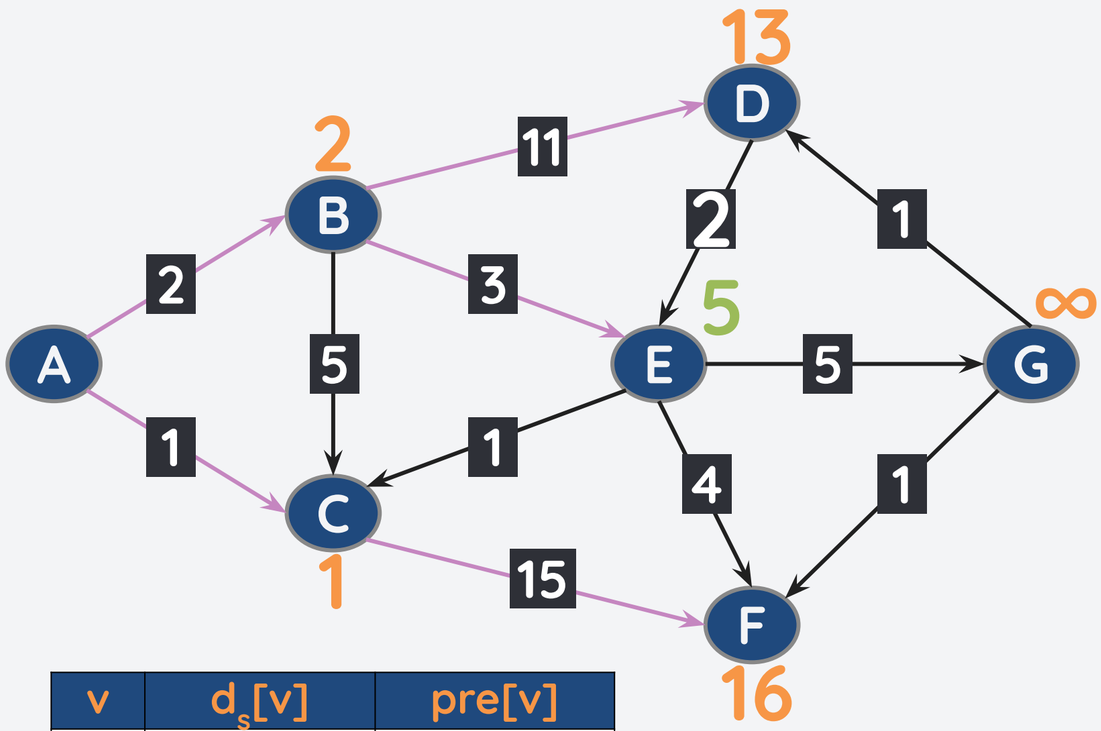
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	16	C
G	∞	-

PQ: [**(E,5)**,(D,13),(F,16)]

Pop (B,2). Neighbors of B: C, D, and E

$d_s[B] + w(B,E) = 2+3 = 5 < ∞$. Update $d_s[E] = 5$, pre[E] = B.

Push onto PQ.

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

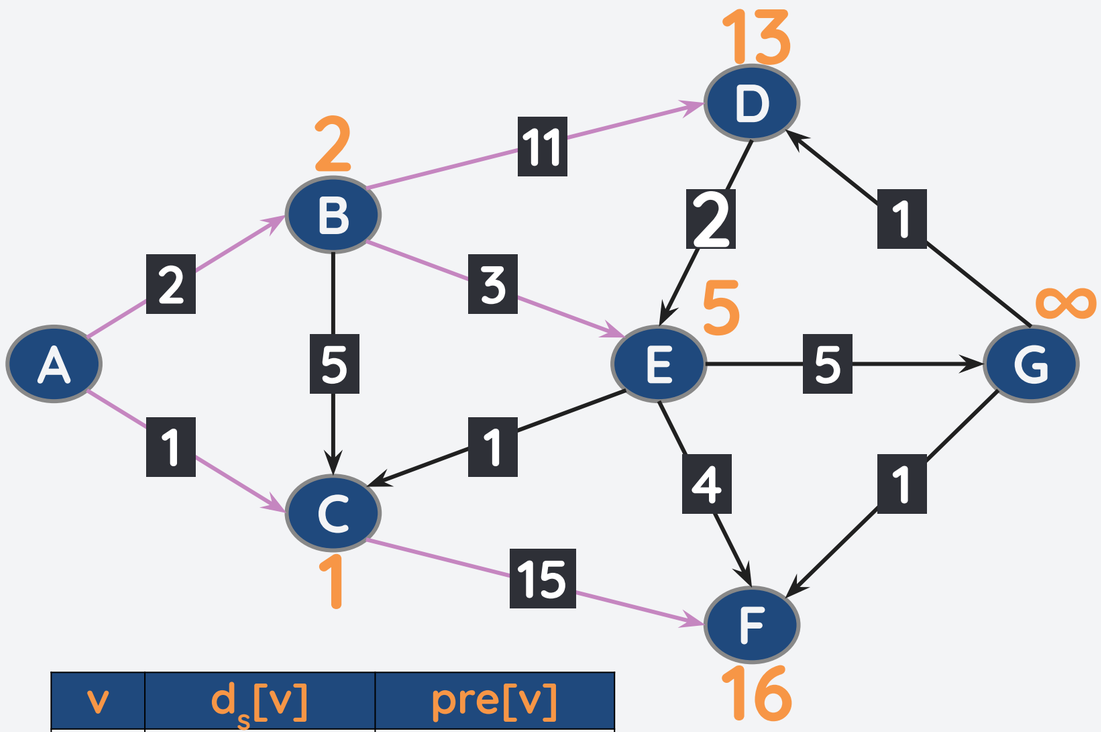
- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [**(E,5)**,(D,13),(F,16)]

Pop (E,5). Neighbors of B: F and G



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	16	C
G	∞	-

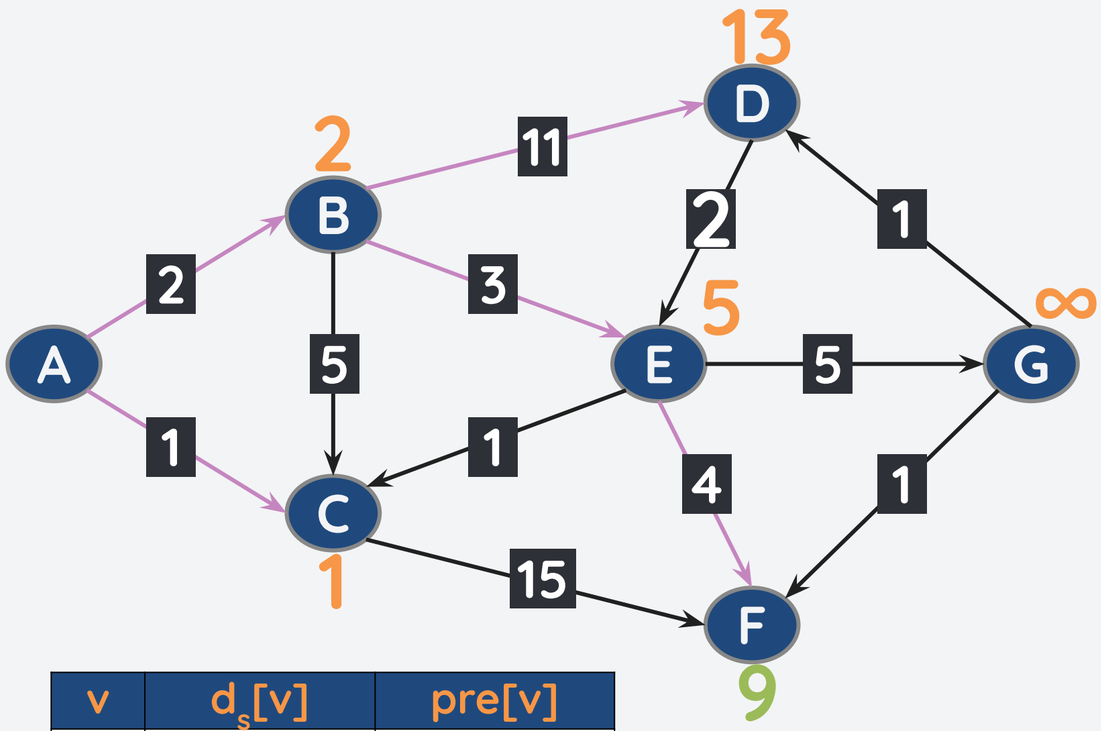
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	∞	-

PQ: [**(F,9)**,(D,13)]

Pop (E,5). Neighbors of B: F and G

$d_s[E] + w(E,F) = 5+4 = 9 < 16$. Update $d_s[F] = 9$, pre[F] = E.

Push onto PQ.

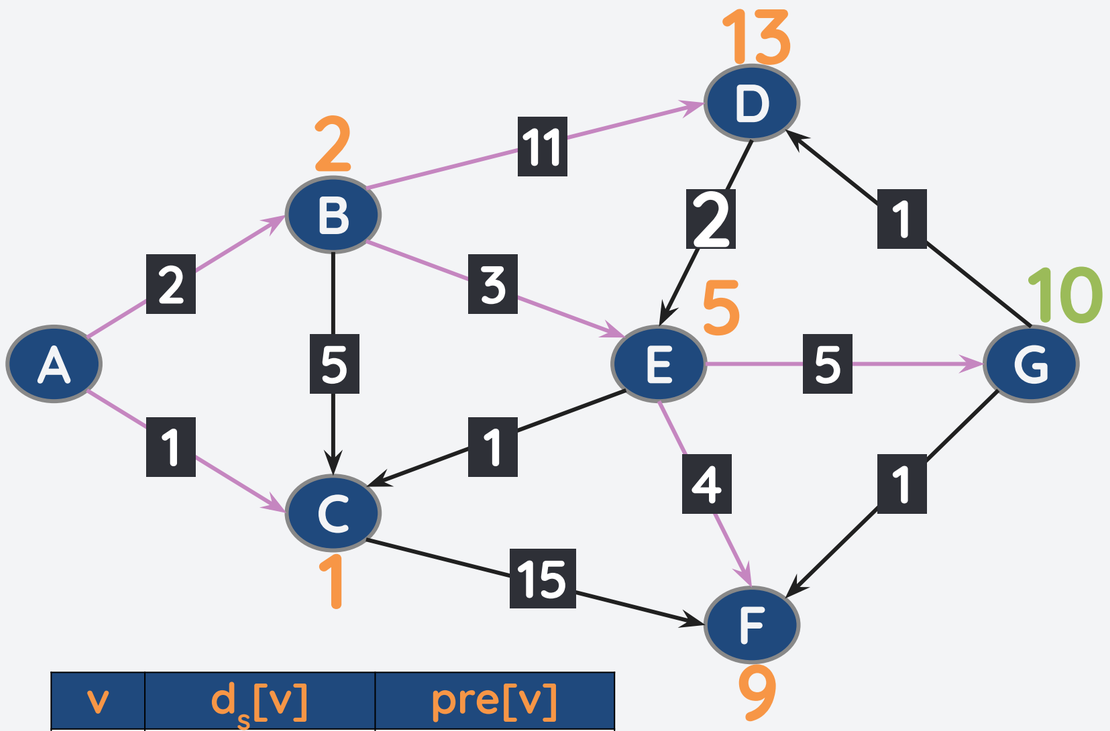
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	10	E

PQ: [(F,9),**(G,10)**,(D,13)]

Pop (E,5). Neighbors of B: F and G

$d_s[E] + w(E,G) = 5+5 = 10 < ∞$. Update $d_s[G] = 10$, pre[G] = E.

Push onto PQ.

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

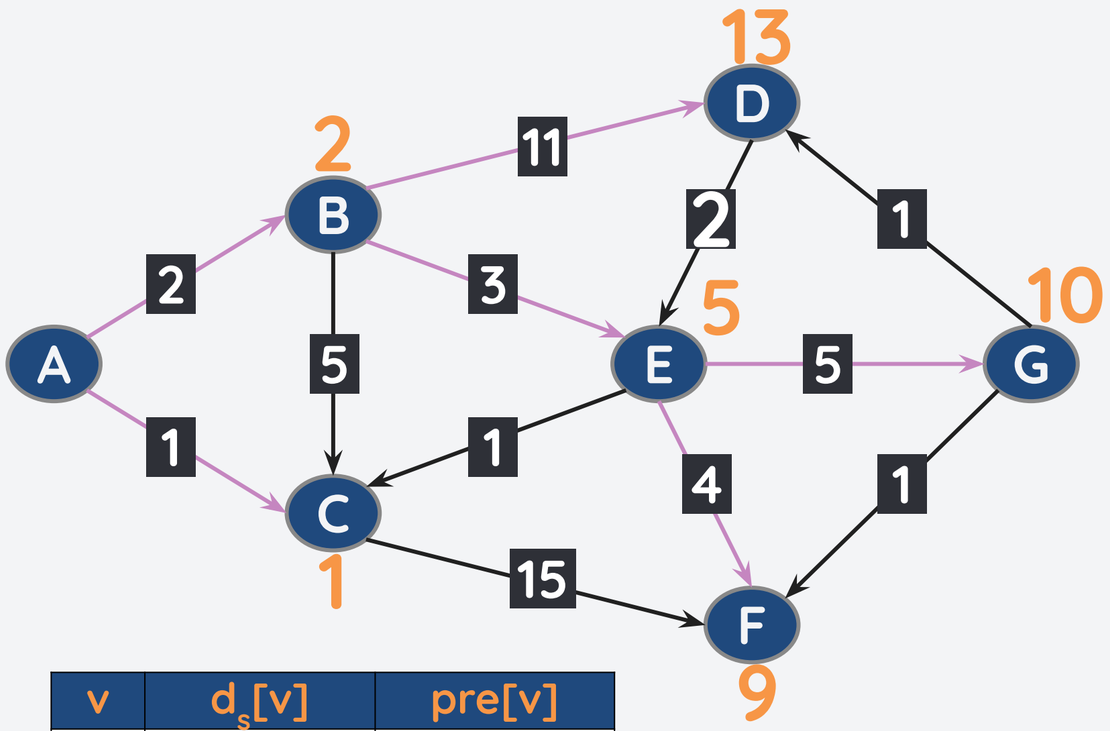
- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [**(F,9)**,(G,10),(D,13)]

Pop (F,9). No neighbors. Skip.



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	10	E

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

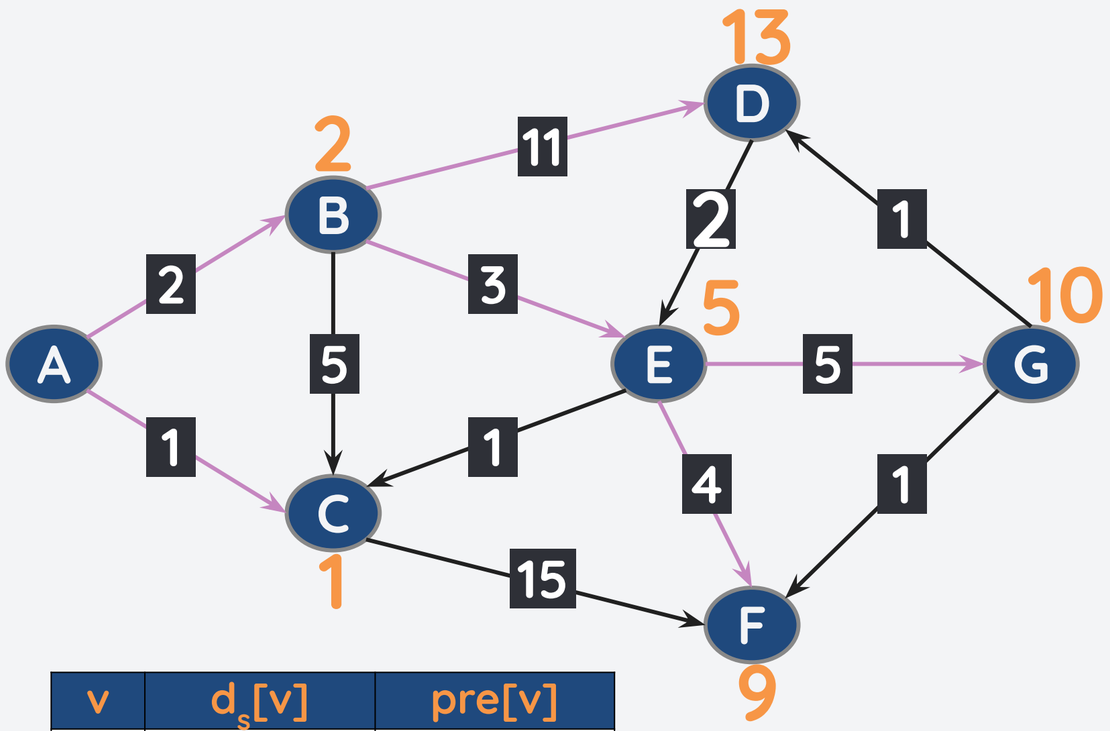
- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: [**(G,10)**,(D,13)]

Pop (G,10). Neighbors: D and F



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	13	B
E	5	B
F	9	E
G	10	E

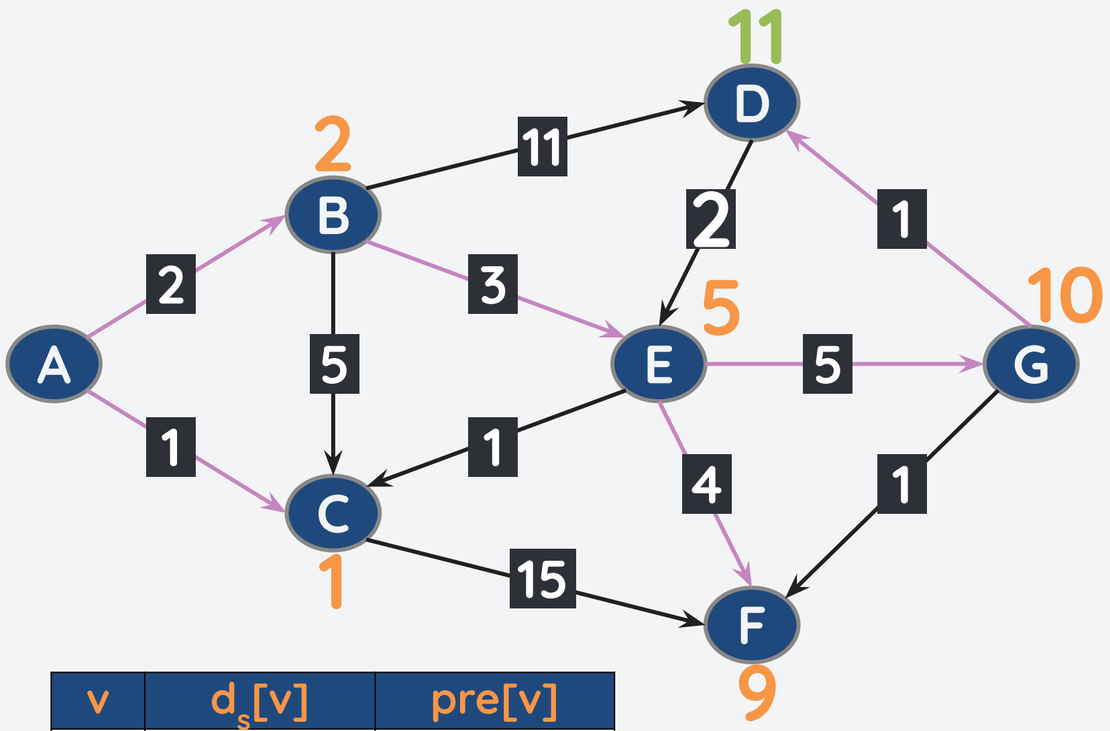
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E

PQ: [(D,11)]

Pop (G,10). Neighbors: D and F

$d_s[G] + w(G,D) = 10+1 = 11 < 13$. Update $d_s[D] = 11$, pre[D] = G.

Push onto PQ.

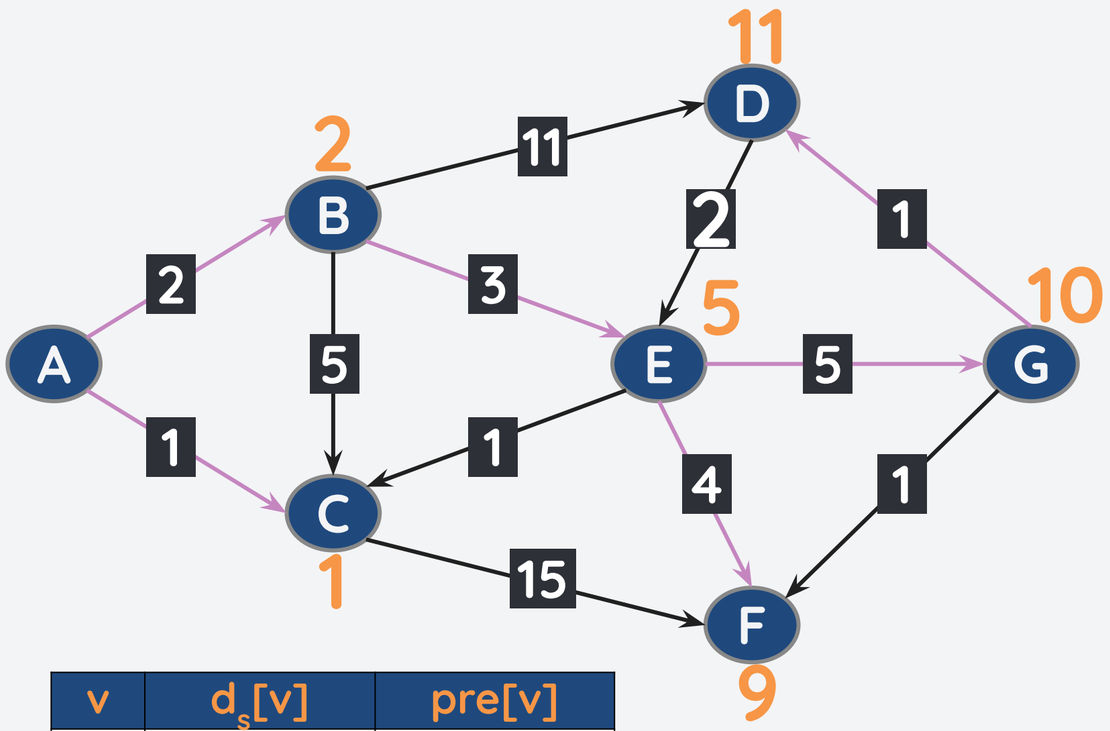
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E

PQ: [(D,11)]

Pop (G,10). Neighbors: D and F

$d_s[G] + w(G,F) = 10+1 = 11 > 9$. **Do nothing.**

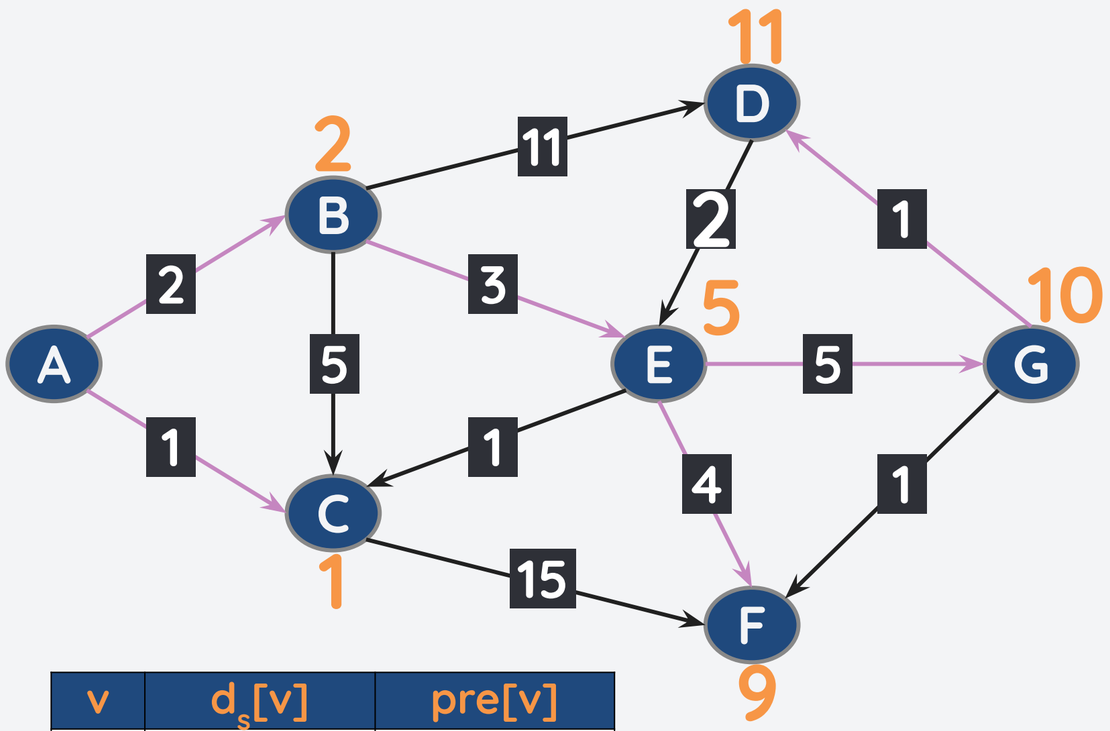
Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E

PQ: [**(D,11)**]

Pop (D,11). Neighbors: E

$d_s[D] + w(D,E) = 11+2 = 13 > 5$. **Do nothing.**

Example: Finding the SPT using Dijkstra's Algo

For each node in the graph:

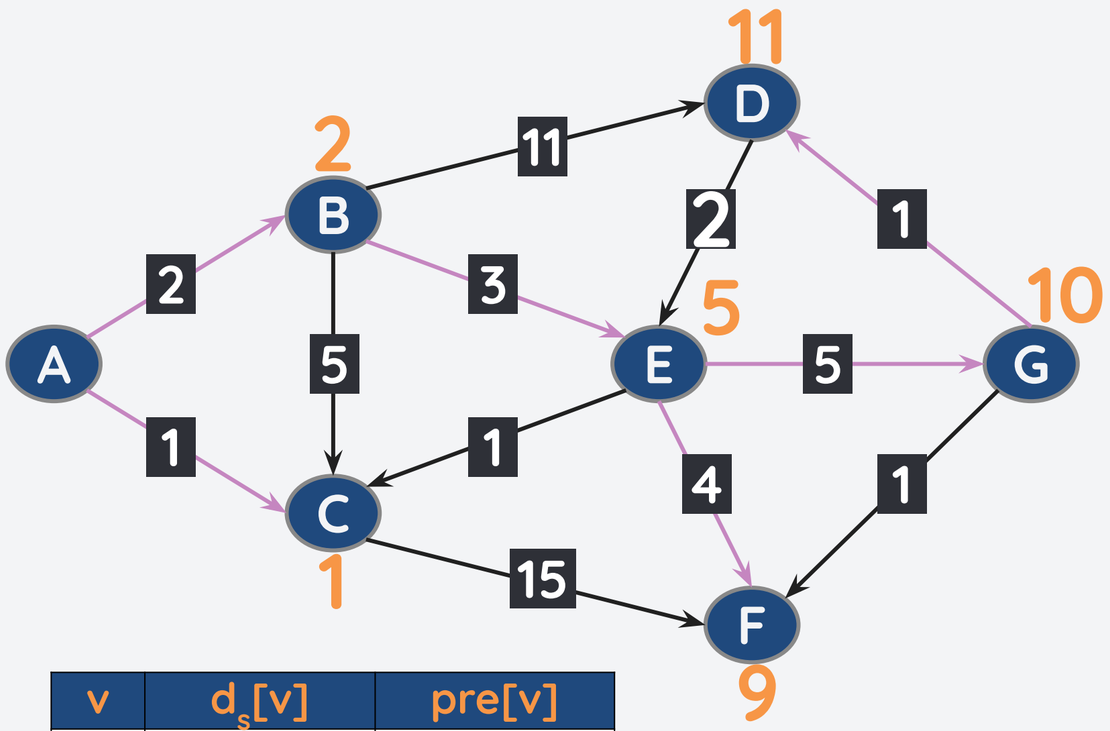
- create distance array from start node $d_s[v]$ initially all ∞ except node s (0)
- push the start node to a priority queue

While the min priority queue is not empty:

- **pop the node x with min priority**
- **for each neighbor y of the popped node:**
  - **if $d_s[x] + w(x,y) < d_s[y]$:**
    - **$d_s[y] = d_s[x] + w(x,y)$**
    - **pre[y] = x**
    - **push y onto the priority queue with $d_s[y]$ as its priority**

PQ: []

We are done! See the Smallest Path Tree?



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	11	G
E	5	B
F	9	E
G	10	E

Some comments with Dijkstra’s Algorithm

- Dijkstra’s Algorithm is **complete** as long as the graph is **connected and has non-negative edges**.
- Dijkstra’s Algorithm is **optimal** since it provides the shortest path from the start node to all the other node as long as the graph is **connected** and has **non-negative edges**.
- **Time Complexity**: **O((|V|+|E|)log|V|)**
  - |V|log(|V|) - each node is added and removed |V| times. For each addition/removal, the binary heap is fixed to maintain heap property.
  - |E|log(|V|) - if we find a shorter path, we update the heap at most |E| times.
- **Space Complexity**: **O(|V|)**
  - Stores the distance and predecessor arrays.

- Path-finding Algorithms
- DFS and BFS for Path-finding
- Dijkstra’s Algorithm
- **Bellman-Ford Algorithm**

In real life, its possible to have a negative weight

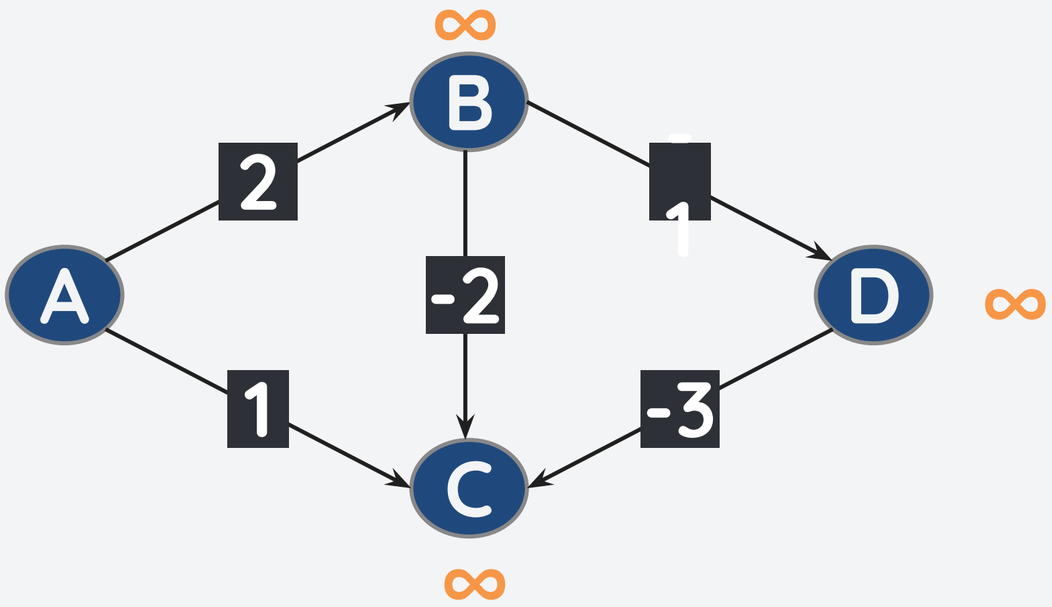
As mentioned previously, Dijkstra's algorithm works fine for unweighted and weighted graph **as long as the edges are positive**.

It might seem weird at first, but there are instances where real life applications of graphs have some edges have negative weights.

- Transmission networks in power systems can be modelled as a graph where each edge represent power loss. A positive edge indicate a system loss while a negative edge indicate a system gain.
- In communication systems, wireless networks can be modelled as a graph where each edge represent a loss in signal quality. Positive edge means signal strength degrades, negative edge means an increase in signal strength.

But why did Dijkstra's algorithm fail?

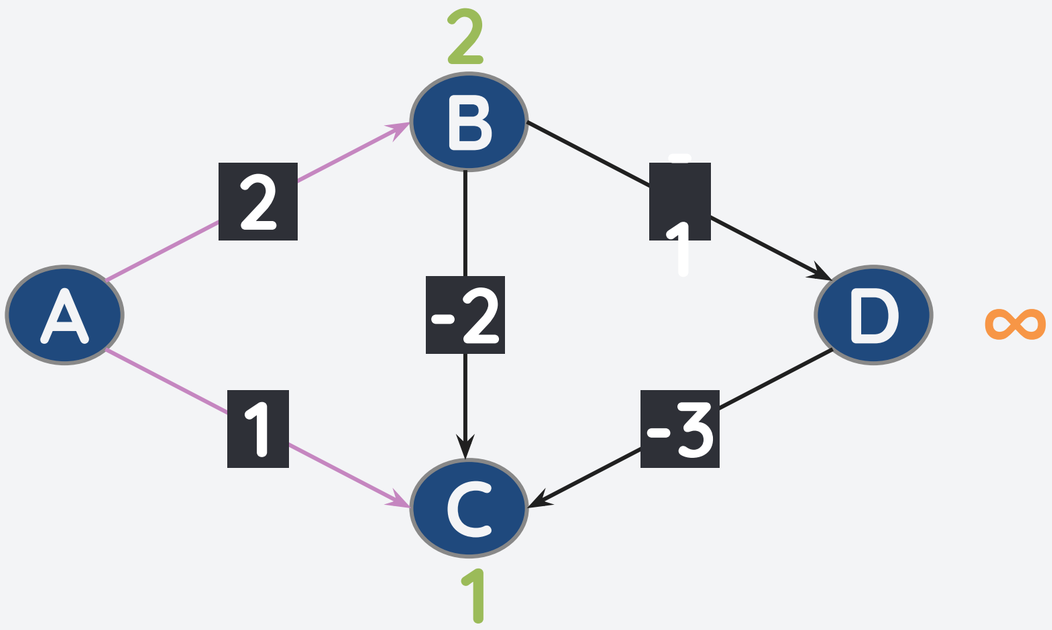
Consider the weighted graph below with positive and negative weighted edges. Let's try to create an SPT with A as the starting node.



v	$d_s[v]$	pre[v]
A	0	-
B	∞	-
C	∞	-
D	∞	-

But why did Dijkstra's algorithm fail?

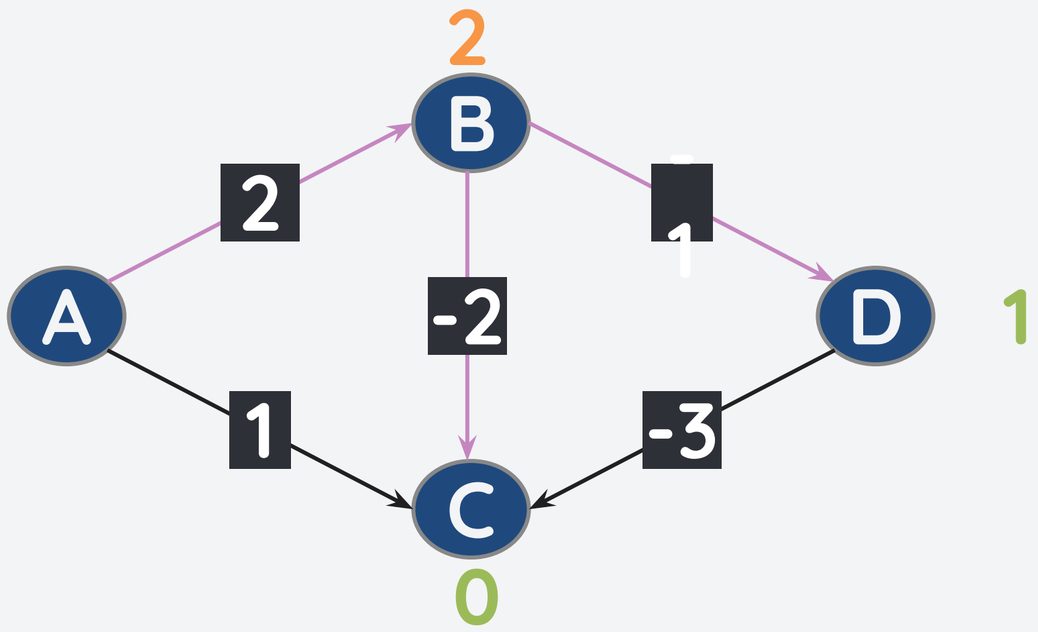
Consider the weighted graph below with positive and negative weighted edges. Let's try to create an SPT with A as the starting node.



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	1	A
D	∞	-

But why did Dijkstra's algorithm fail?

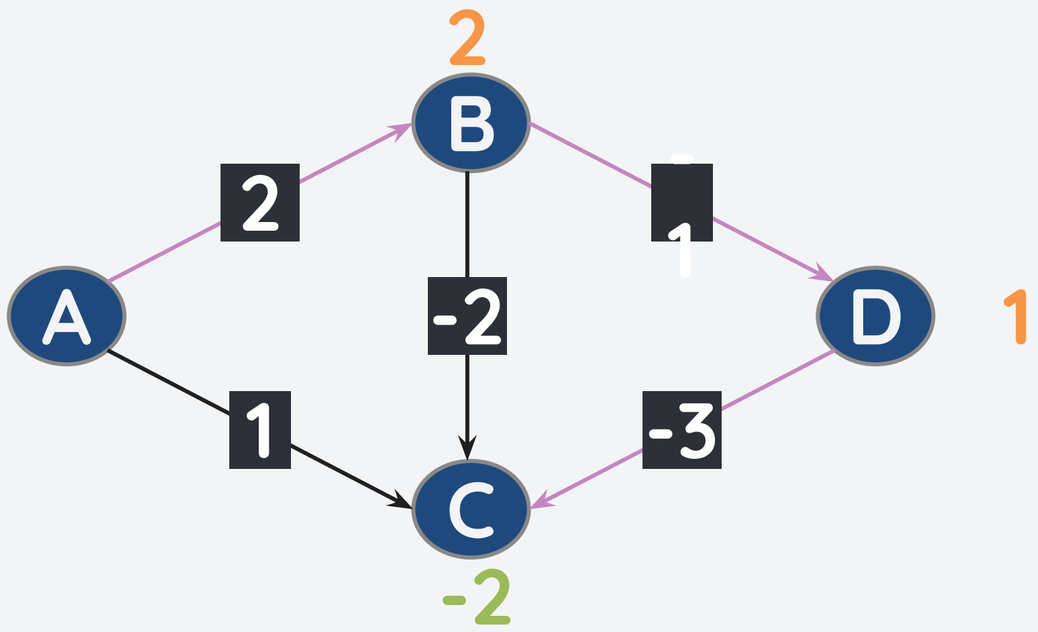
Consider the weighted graph below with positive and negative weighted edges. Let's try to create an SPT with A as the starting node.



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	B
D	1	B

But why did Dijkstra's algorithm fail?

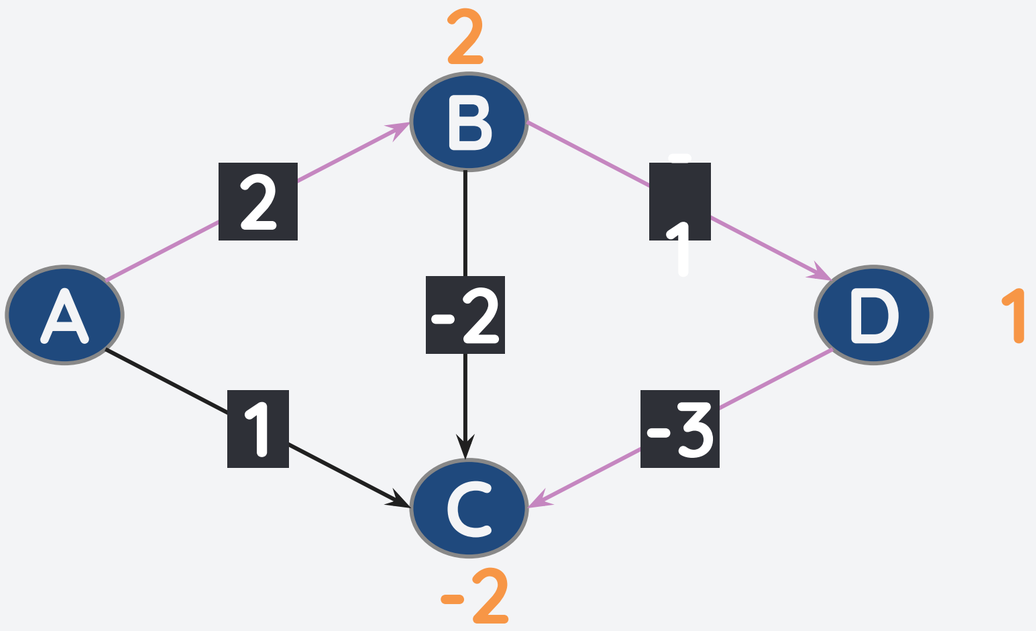
Consider the weighted graph below with positive and negative weighted edges. Let's try to create an SPT with A as the starting node.



v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	-2	D
D	1	B

But why did Dijkstra's algorithm fail?

Consider the weighted graph below with positive and negative weighted edges. Let's try to create an SPT with A as the starting node.

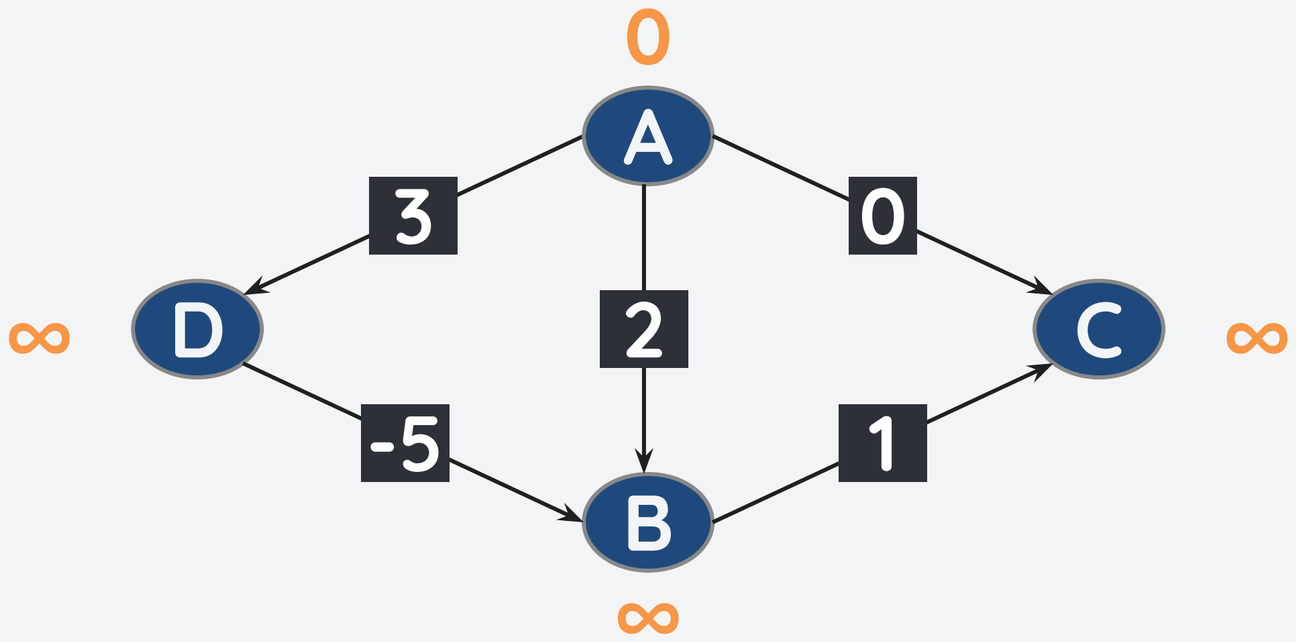


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	-2	D
D	1	B

Dijkstra's Algorithm can still work on <u>some</u> graphs!

But why did Dijkstra’s algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

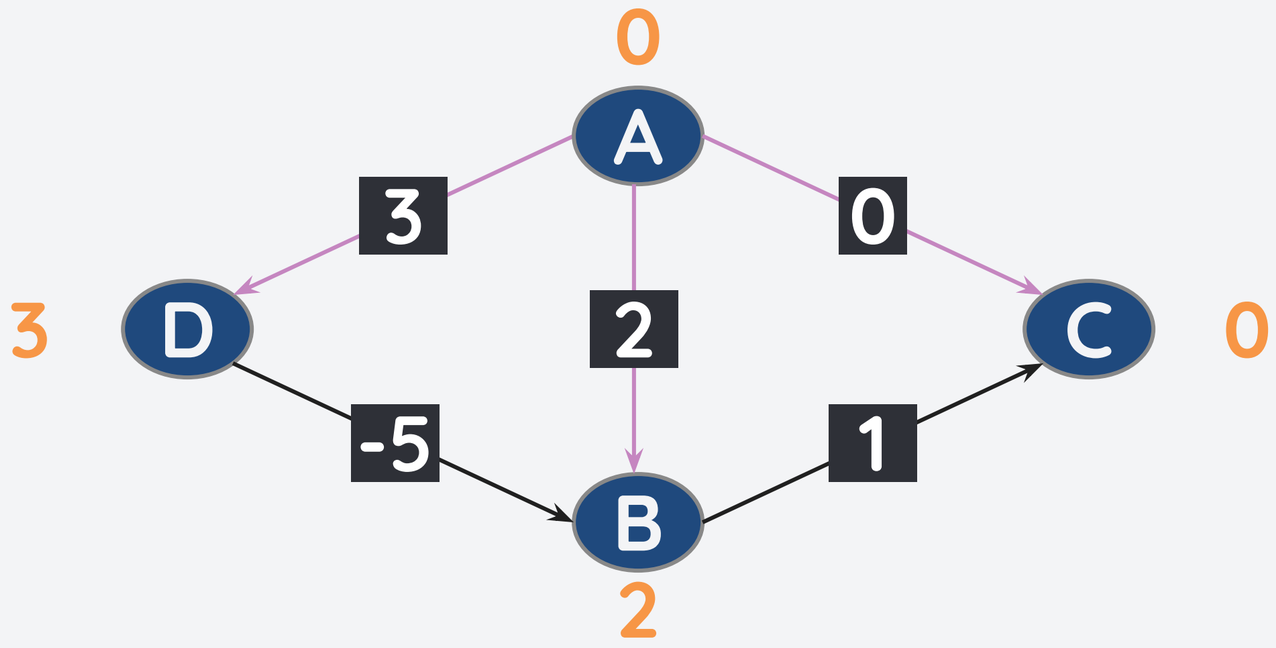


v	$d_s[v]$	pre[v]
A	0	-
B	∞	-
C	∞	-
D	∞	-

PQ: [(A,0)]

But why did Dijkstra’s algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

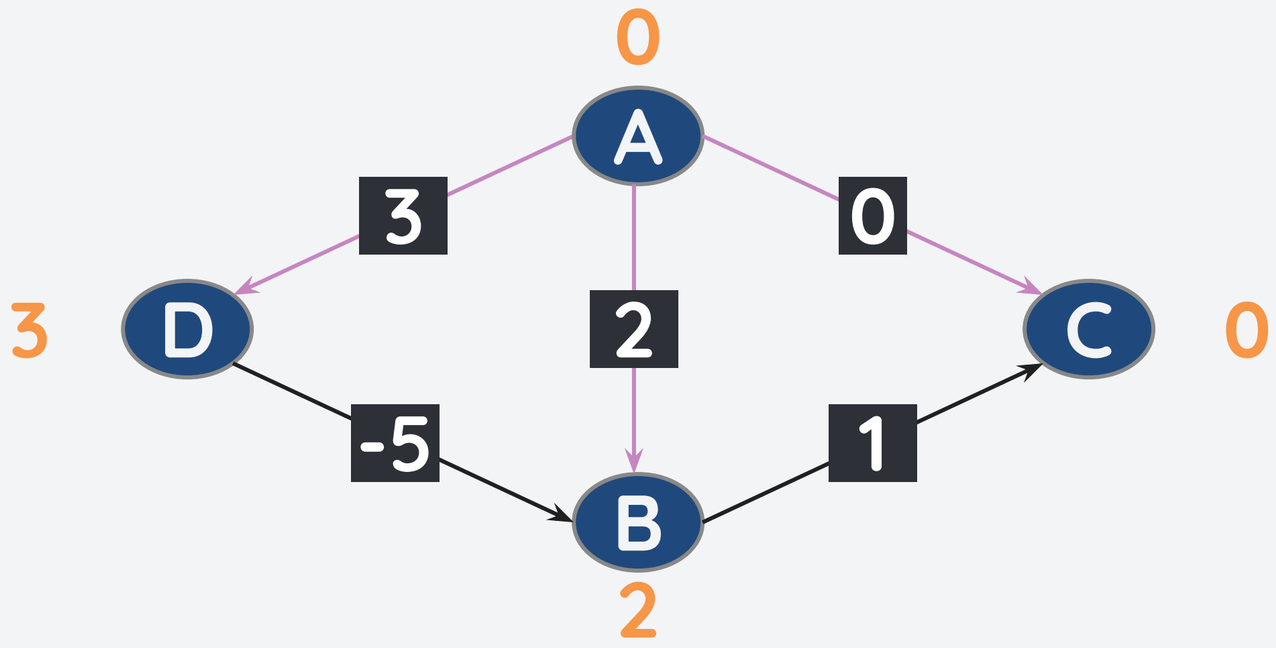


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

PQ: [(C,0),(B,2),(D,3)]

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

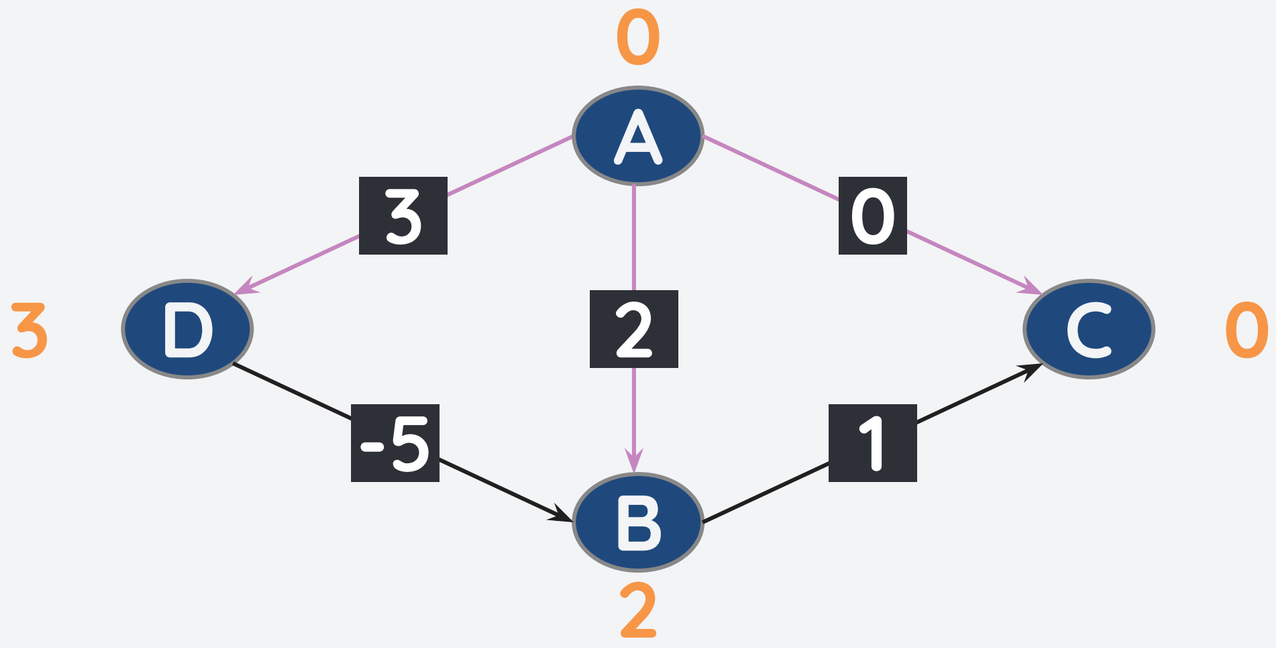


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

PQ: [(B,2),(D,3)]

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

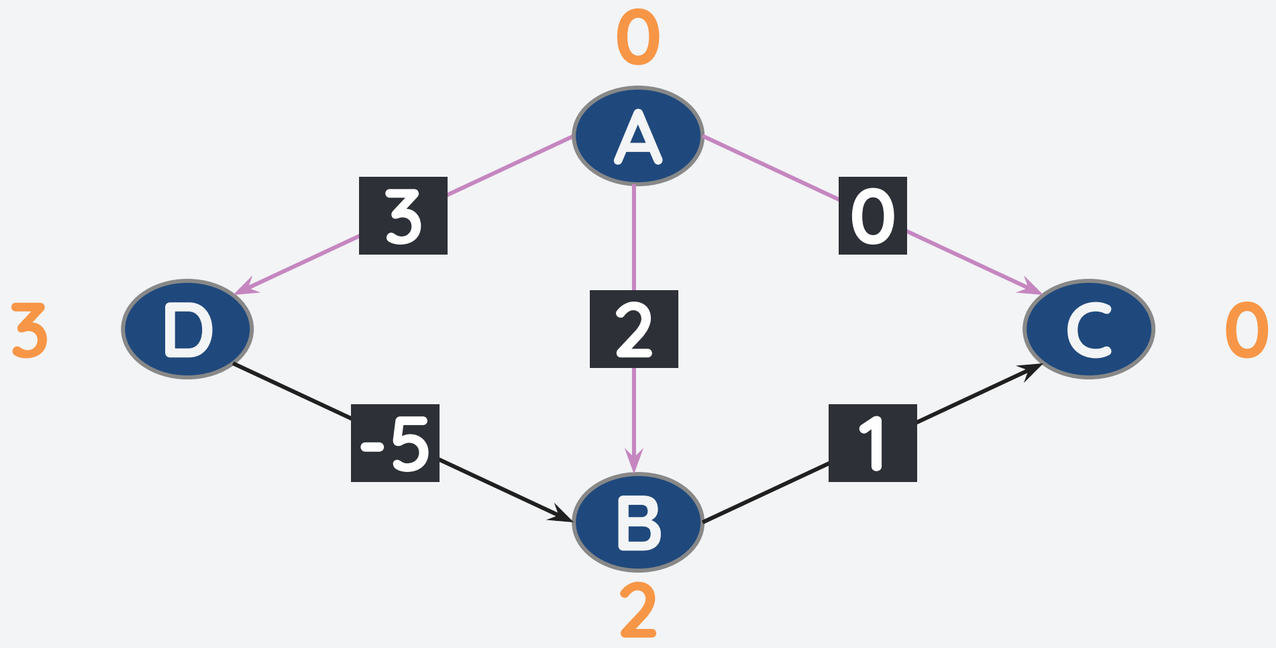


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

PQ: [(D,3)]

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

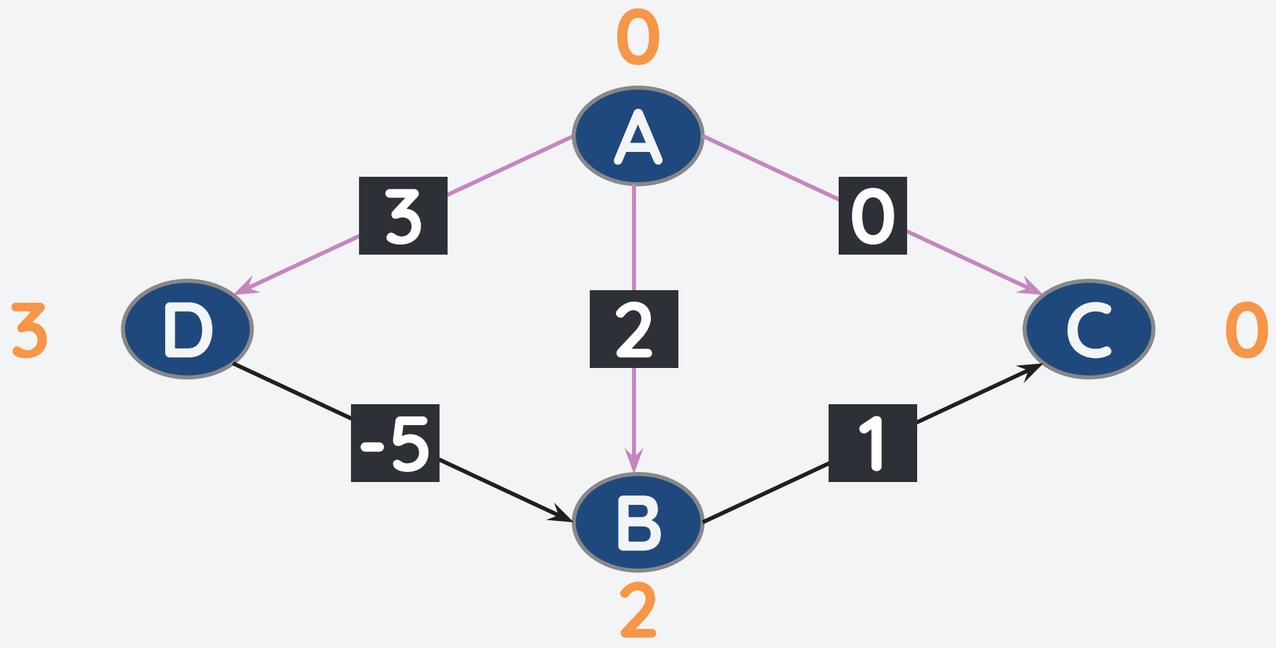


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

PQ: []

But why did Dijkstra’s algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

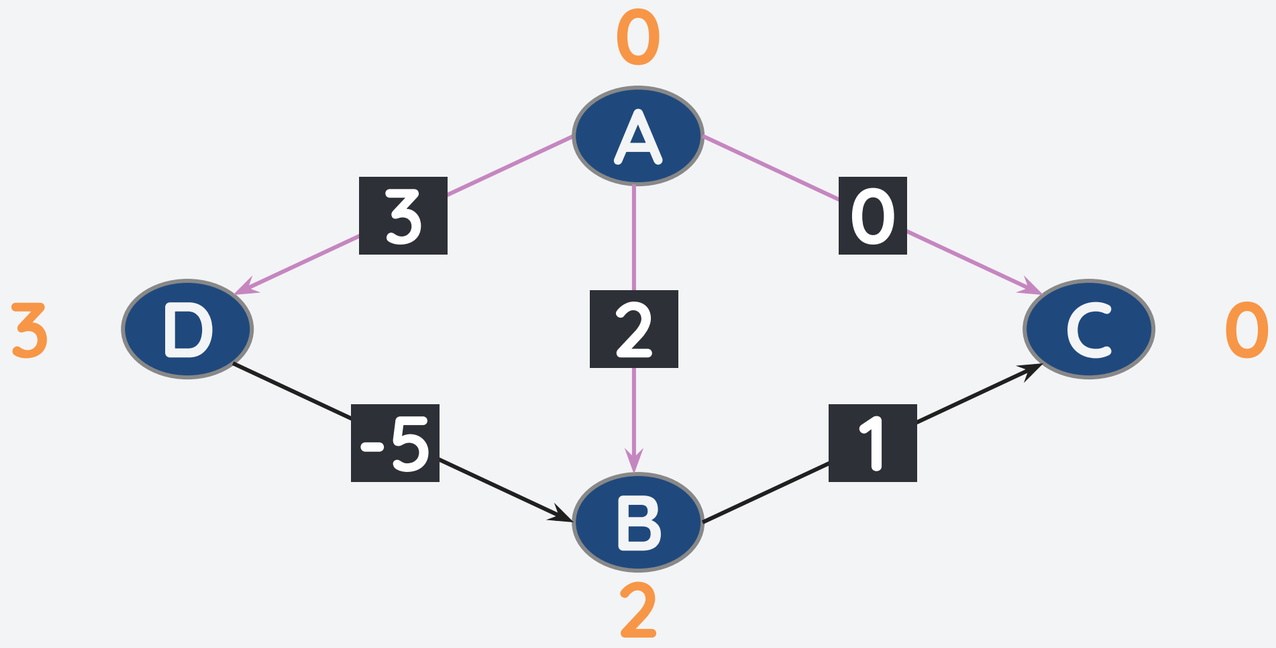


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

What’s wrong here? The shortest path from B and C is not correct.

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

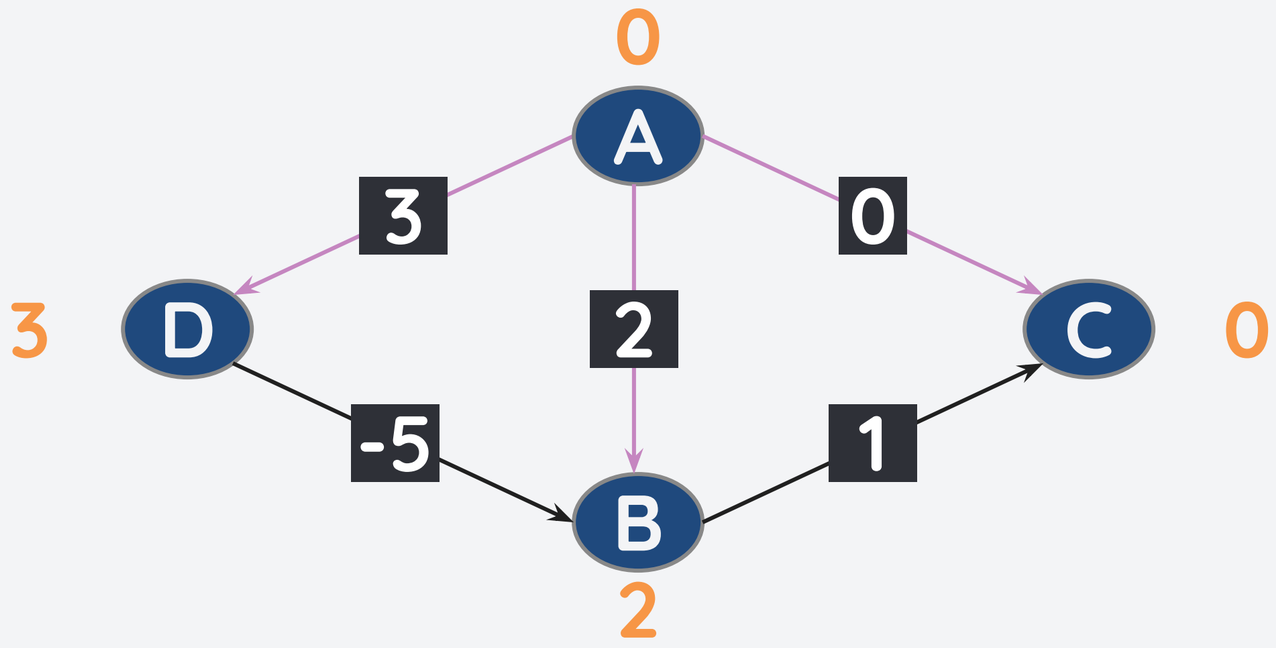


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

Why? Recall that for Dijkstra's algorithm, once a node is visited, its path is final.

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

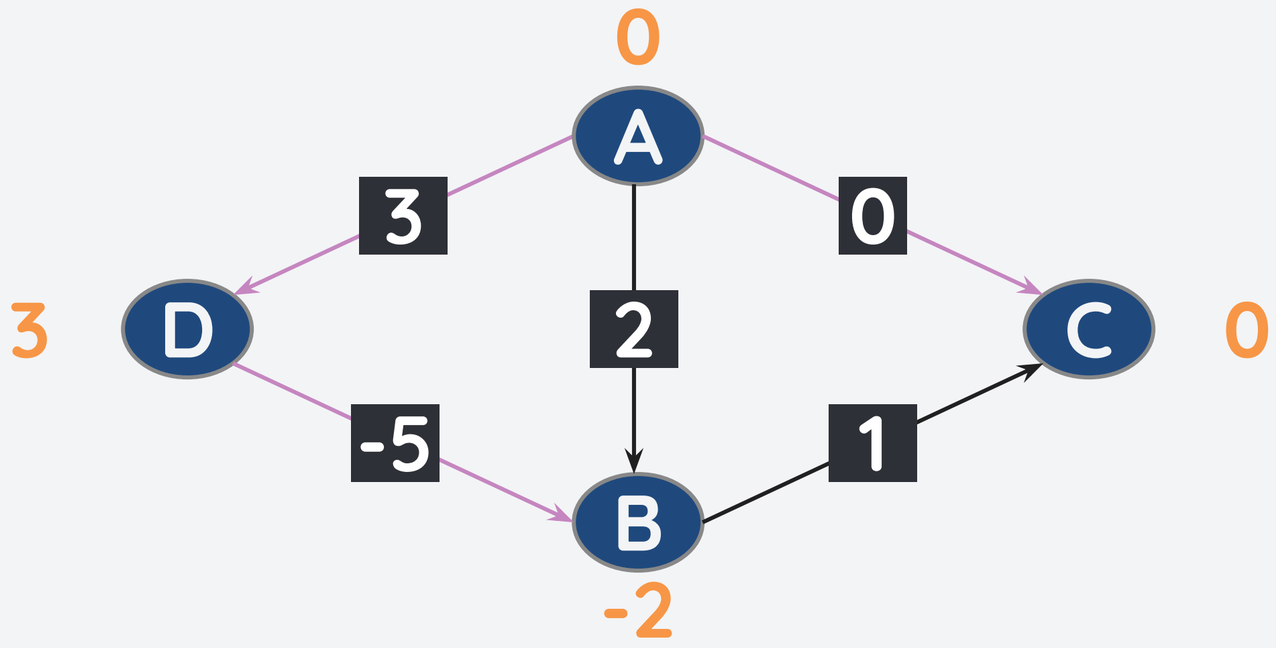


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	0	A
D	3	A

The solution? Apply the edge relaxation again even if the node is already visited.

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

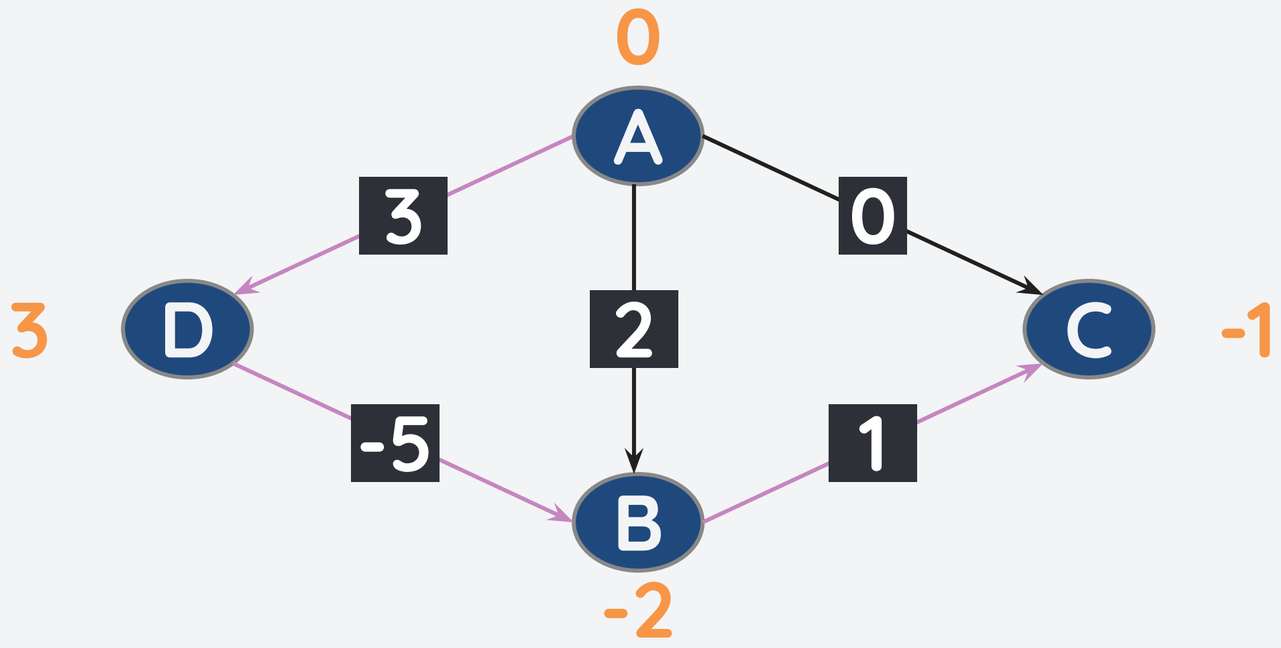


v	$d_s[v]$	pre[v]
A	0	-
B	-2	D
C	0	A
D	3	A

PQ: [(B,-2)]

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?

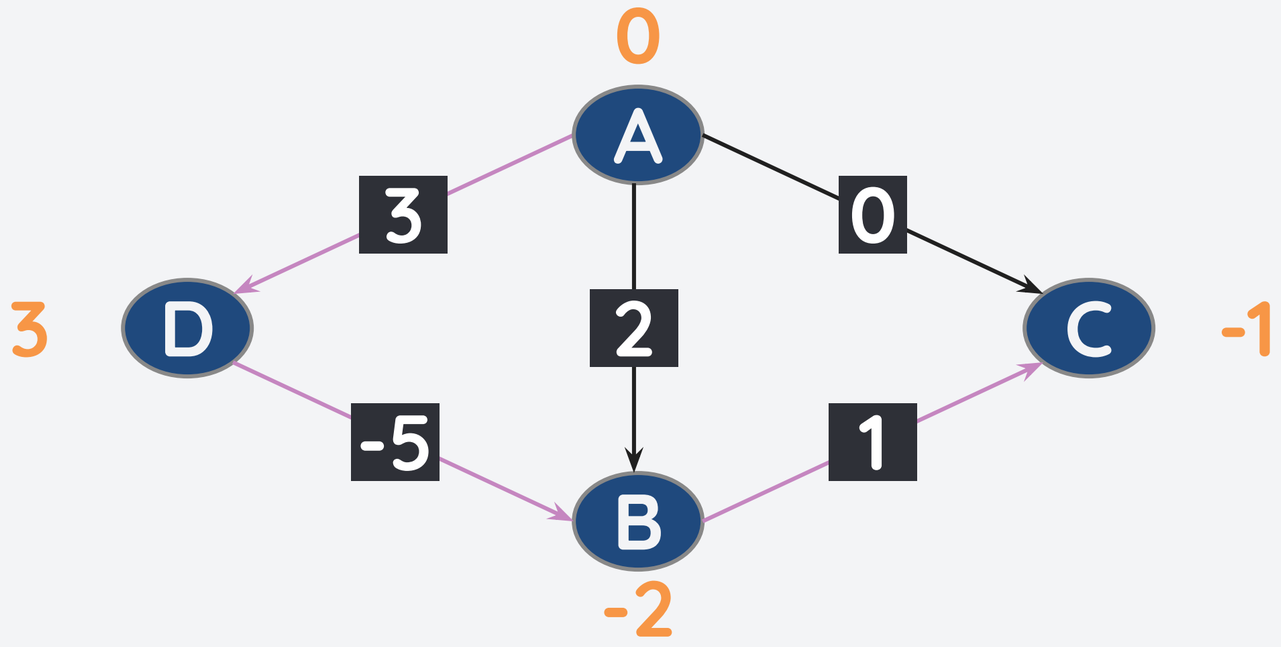


v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	-1	B
D	3	A

PQ: []

But why did Dijkstra's algorithm fail?

But how about this graph, can we find an SPT with node A as the source?



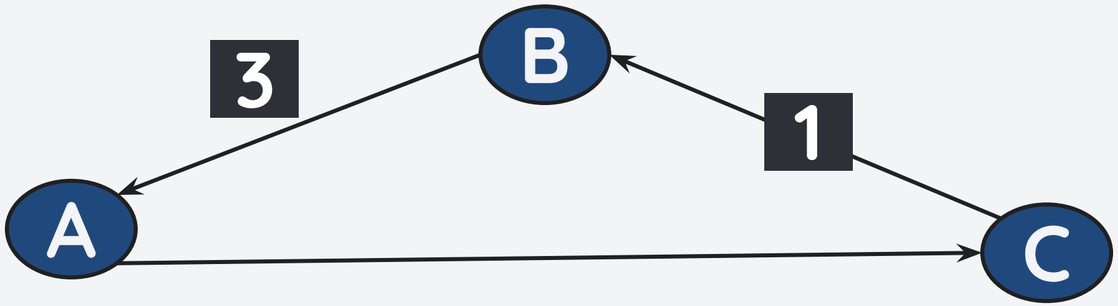
v	$d_s[v]$	pre[v]
A	0	-
B	2	A
C	-1	B
D	3	A

Much better, right? This is the main principle of the Bellman-Ford Algorithm

Principle of Edge Relaxation

For a graph with |V| vertices, all the edges e ∈ E should be relaxed |V|-1 times to compute for the single source shortest path.

However, this does not hold for negative cycles.



To detect negative cycles:

- Do edge relaxation |V|-1 times.
- Do edge relaxation one more time. If $d_s[v]$ changes, then there is a negative cycle.

The Bellman Ford Algorithm

The Bellman-Ford algorithm finds the shortest path by performing edge relaxation on every edge over-and-over again.

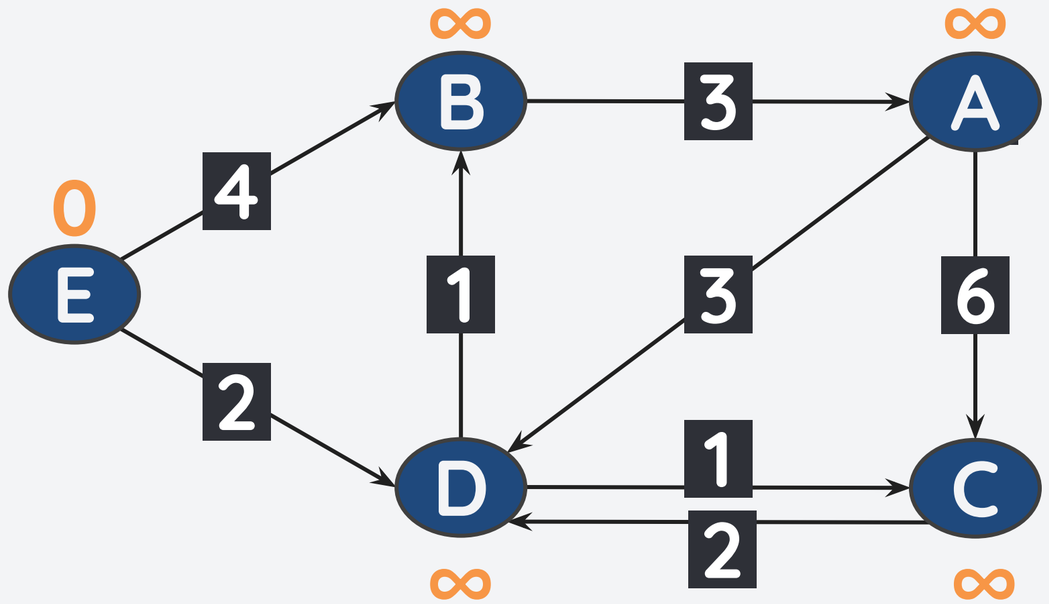
Pseudocode:

```
For number of iterations |V| - 1:
    • For each directed edge from x to y:
        ○ if d_s[x] + w(x,y) < d_s[y]:
            ■ d_s[y] = d_s[x] + w(x,y)
            ■ pre[y] = x
For each directed edge from x to y:
    • if d_s[x] + w(x,y) < d_s[y]:
        ○ negative cycle exists
```

- It does not use PQ, it just loops over all the edges.
- Effective even for graphs with negative edge weights.
- Has a mechanism that detects a negative cycle.
- If there are no changes in the current iteration, the algorithm can be stopped.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



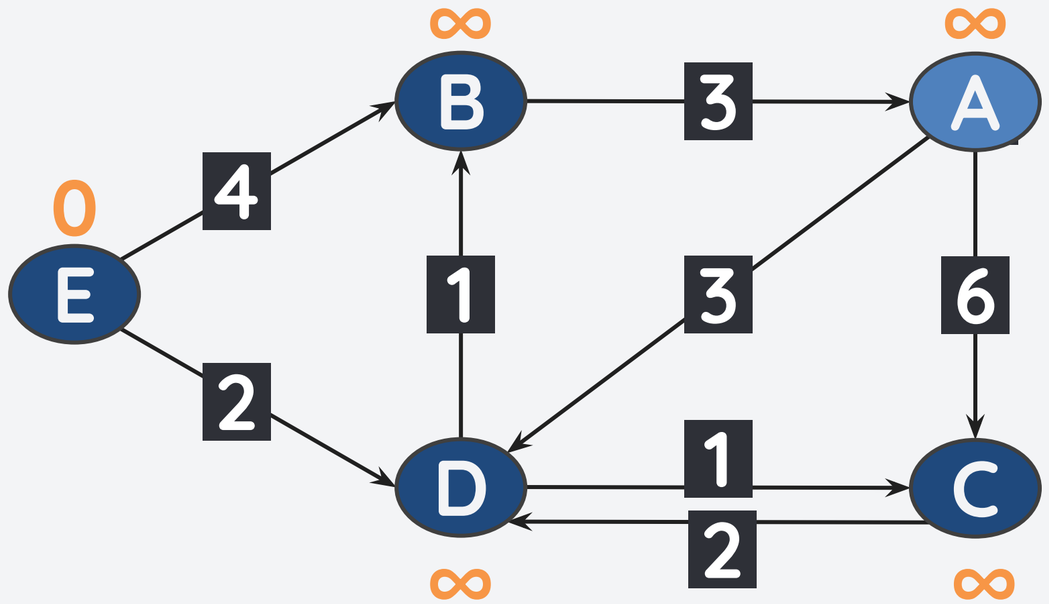
Initial Values

v	dA[v]	pre[v]
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	0	-

We have |V|=5. Thus, we need to perform |V|-1=4 iterations.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



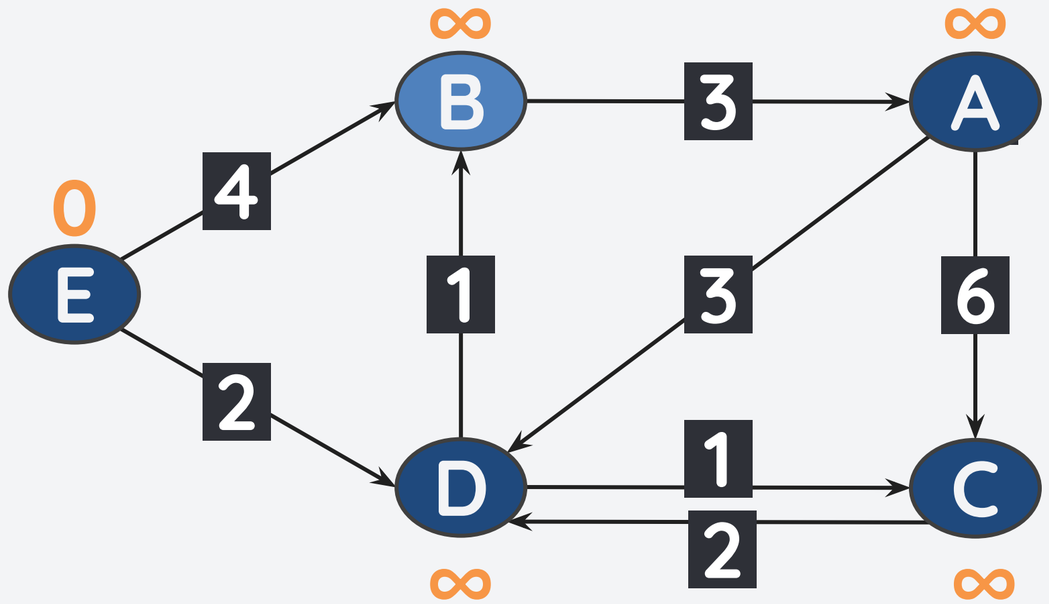
Iteration 1
Edge(s) from A

v	dA[v]	pre[v]
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	0	-

Since current value of dA[A] is still infinity, no update for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



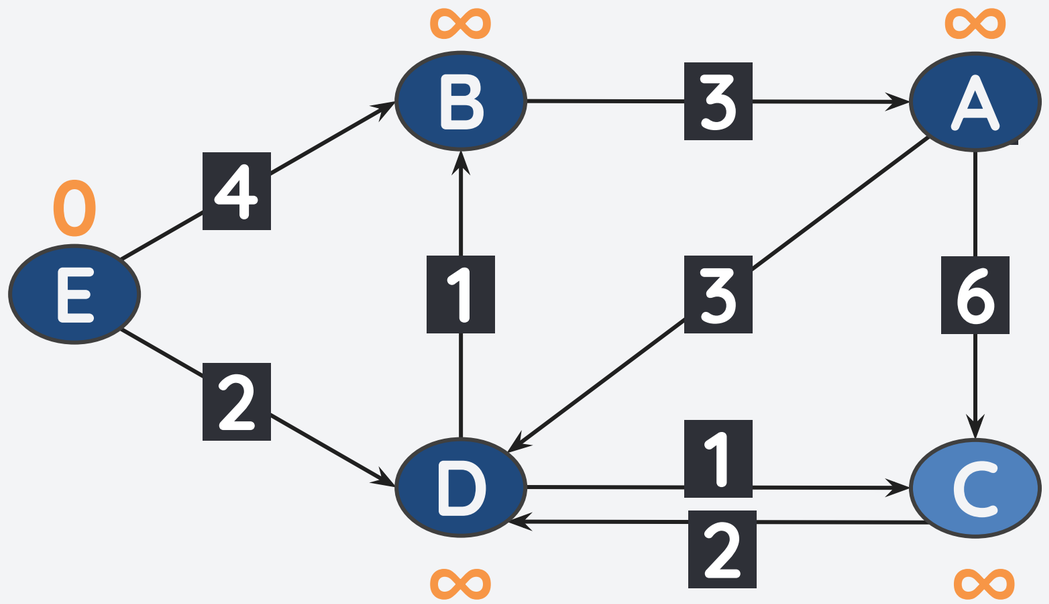
Iteration 1
Edge(s) from B

v	dA[v]	pre[v]
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	0	-

Since current value of dA[B] is still infinity, no update for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



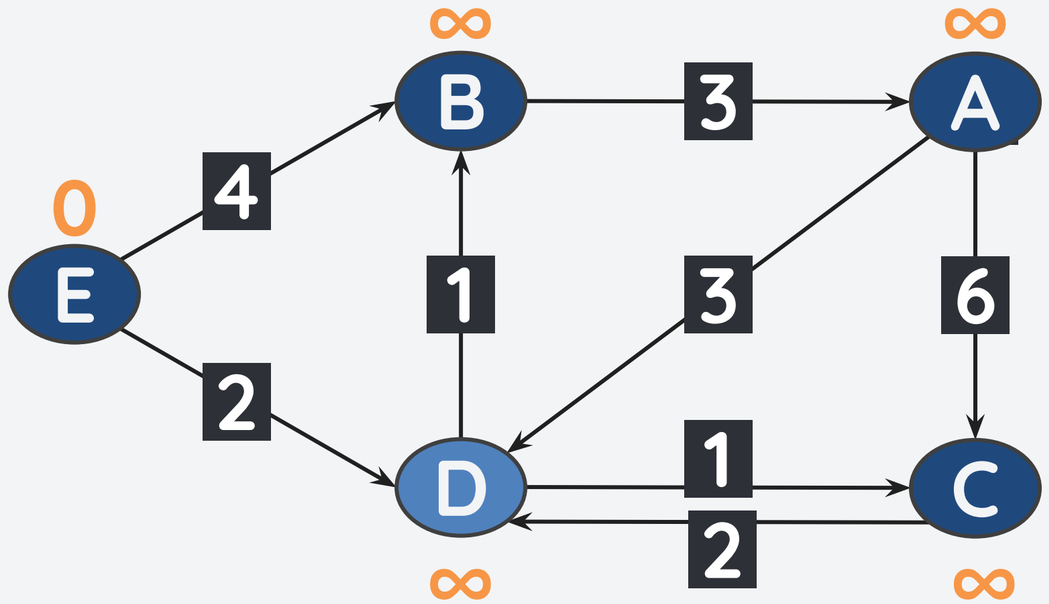
Iteration 1
Edge(s) from C

v	dA[v]	pre[v]
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	0	-

Since current value of dA[C] is still infinity, no update for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



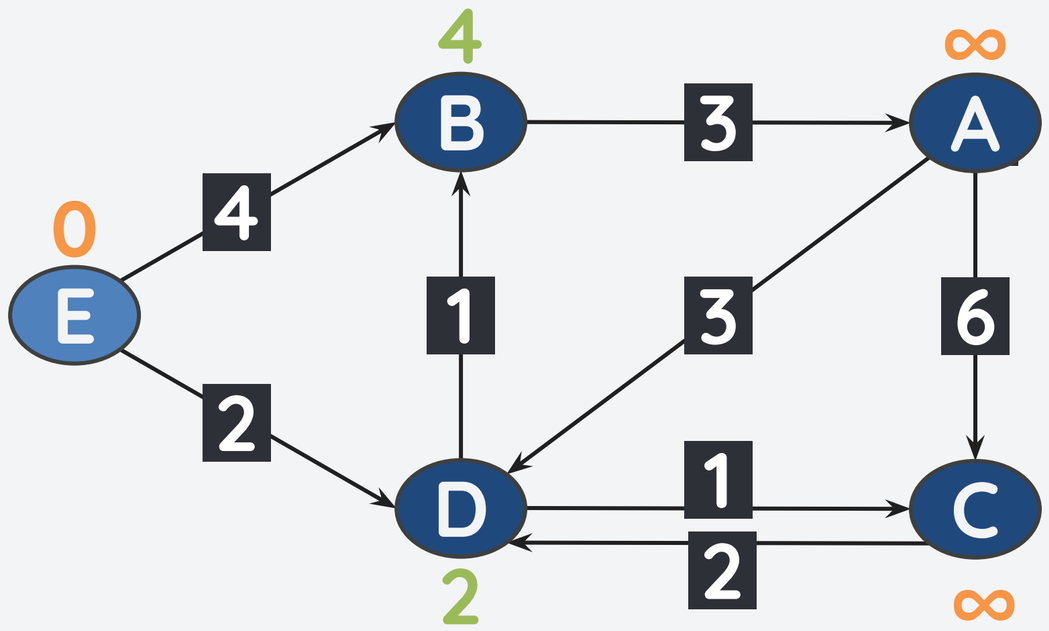
Iteration 1
Edge(s) from D

v	dA[v]	pre[v]
A	∞	-
B	∞	-
C	∞	-
D	∞	-
E	0	-

Since current value of dA[D] is still infinity, no update for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



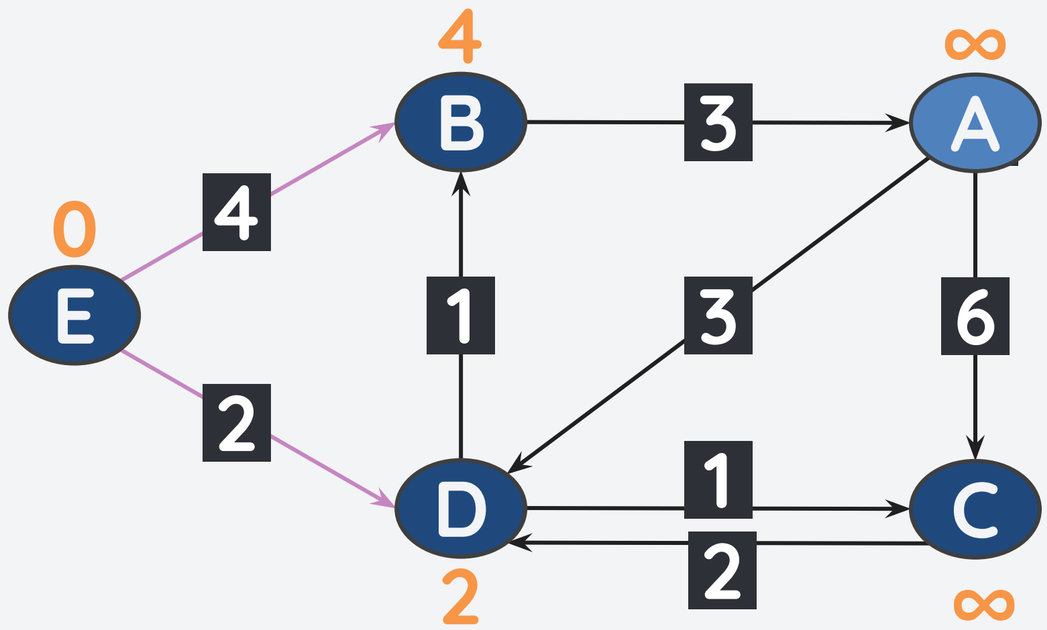
Iteration 1
Edge(s) from E

v	dA[v]	pre[v]
A	∞	-
B	4	E
C	∞	-
D	2	E
E	0	-

From E, we update the path of B and D.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



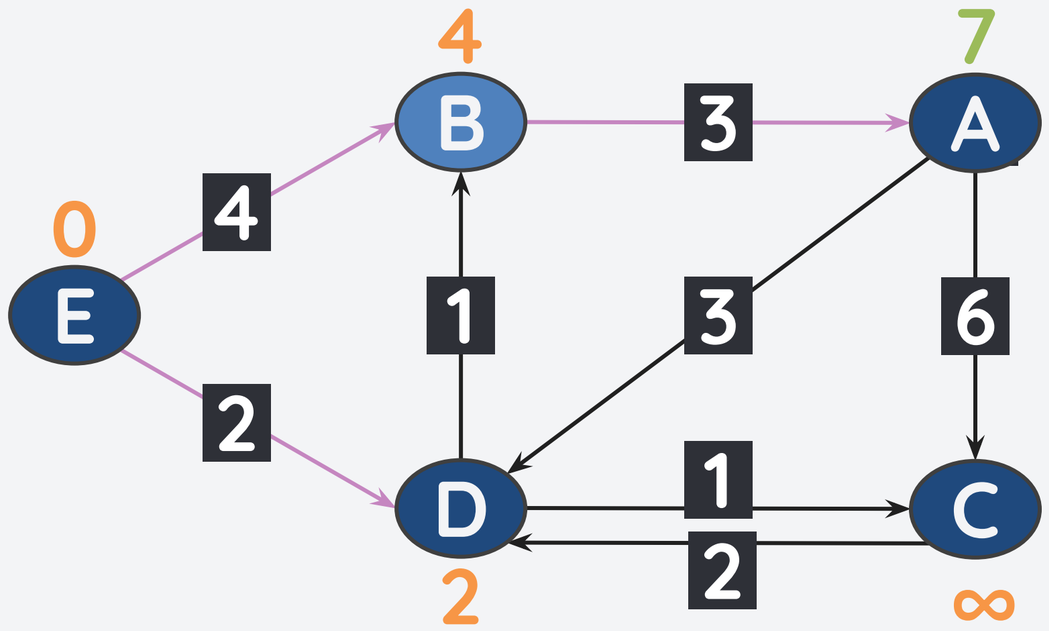
Iteration A
Edge(s) from A

v	dA[v]	pre[v]
A	∞	-
B	4	E
C	∞	-
D	2	E
E	0	-

Since current value of dA[A] is still infinity, no update for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



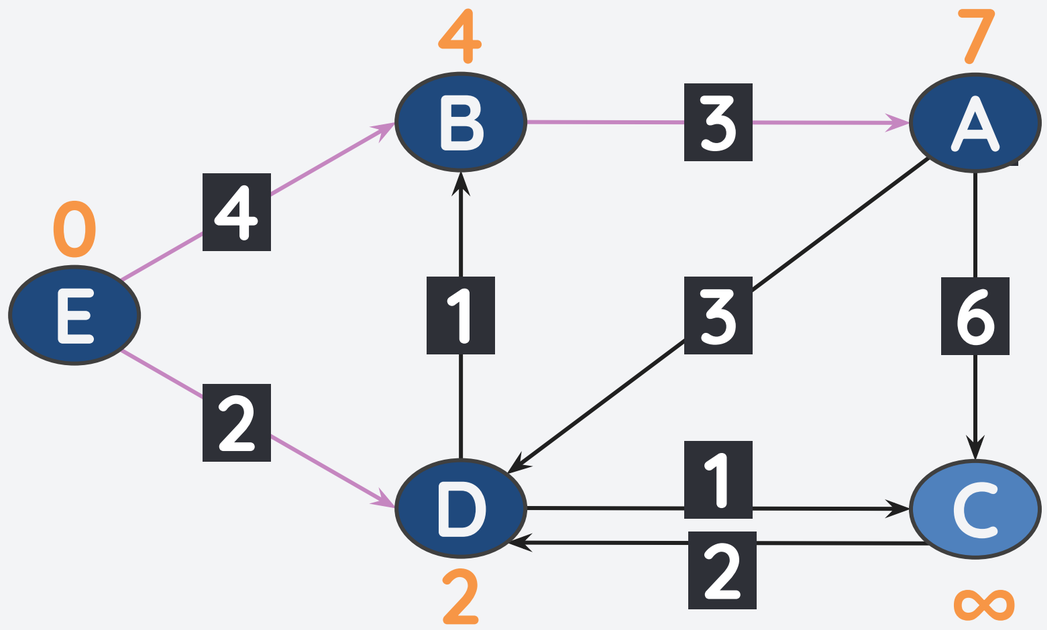
Iteration 2
Edge(s) from B

v	dA[v]	pre[v]
A	7	B
B	4	E
C	∞	-
D	2	E
E	0	-

From B, we update the path cost to A.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



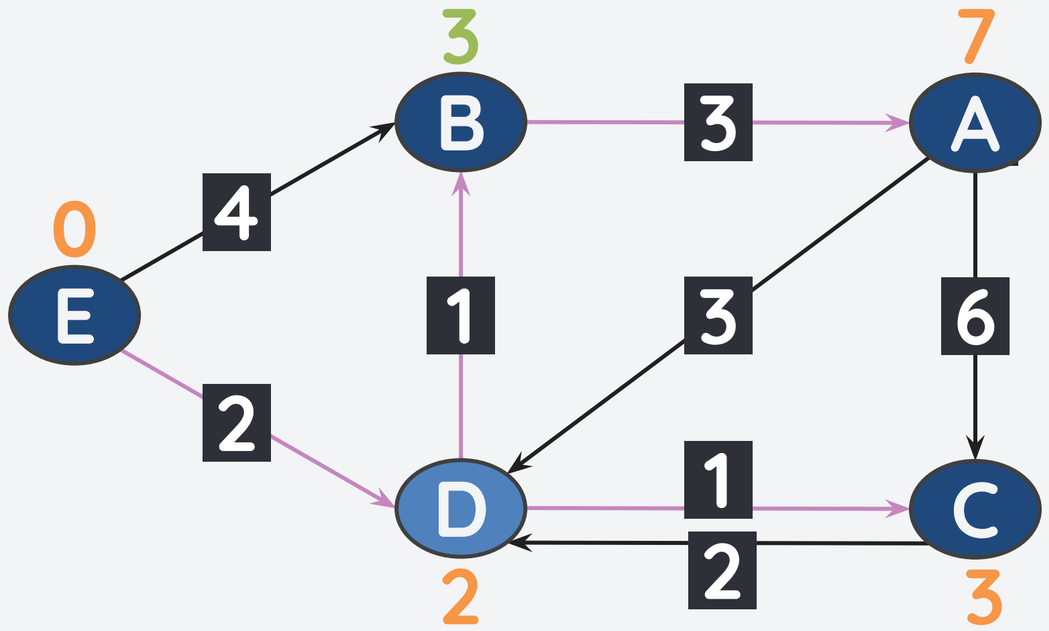
Iteration 2
Edge(s) from C

v	dA[v]	pre[v]
A	7	B
B	4	E
C	∞	-
D	2	E
E	0	-

Since current cost to dA[C] is still infinity, no updates for its neighbors.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



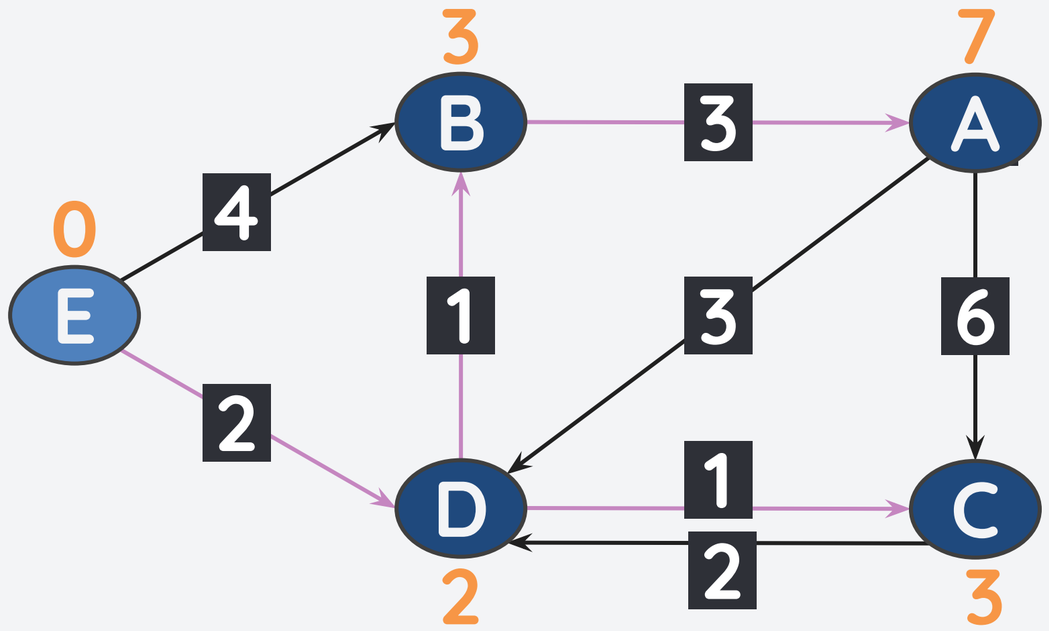
Iteration 2
Edge(s) from D

v	dA[v]	pre[v]
A	7	B
B	3	D
C	3	D
D	2	E
E	0	-

From D, we update the path cost to C and B.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



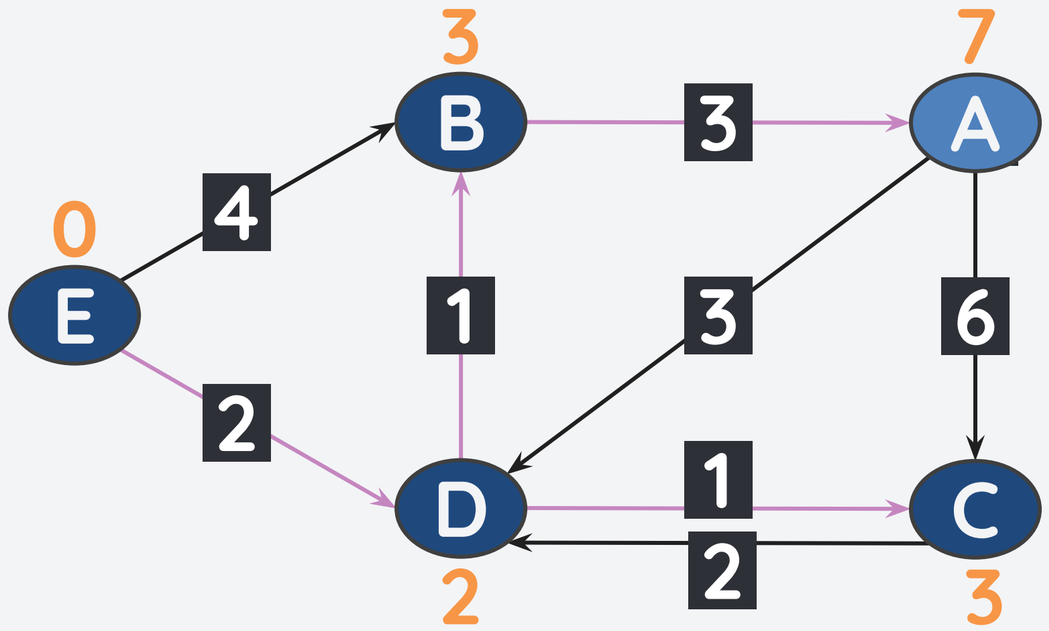
Iteration 2
Edge(s) from E

v	dA[v]	pre[v]
A	7	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the path from E to B/D has equal or higher cost than the current cost of B and D, no update.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



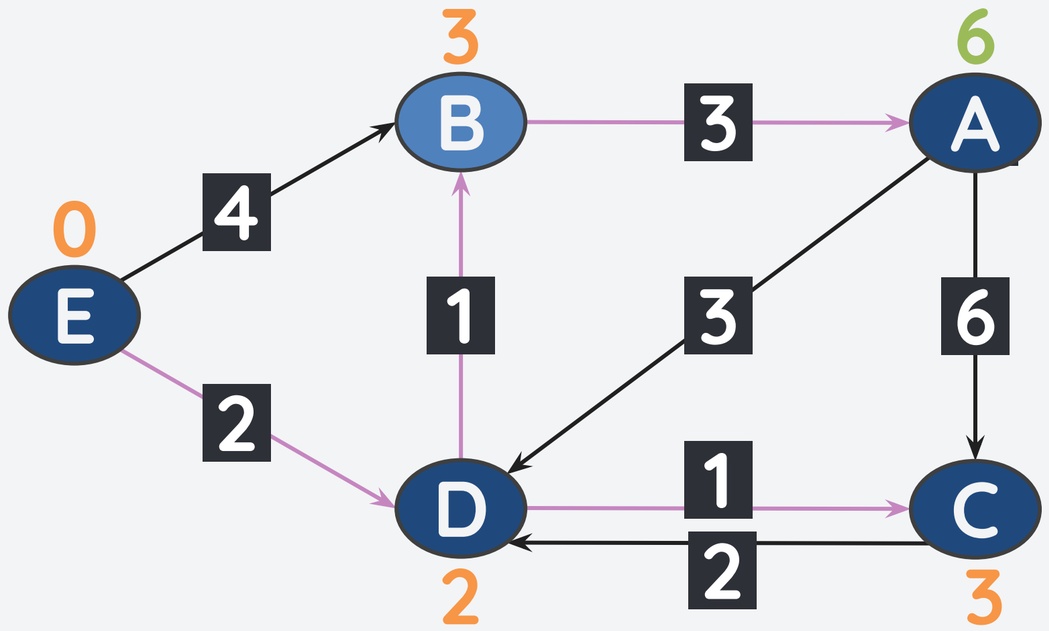
Iteration 3
Edge(s) from A

v	dA[v]	pre[v]
A	7	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the path from A to D/C has equal or higher cost than the current cost of D and C, no update.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



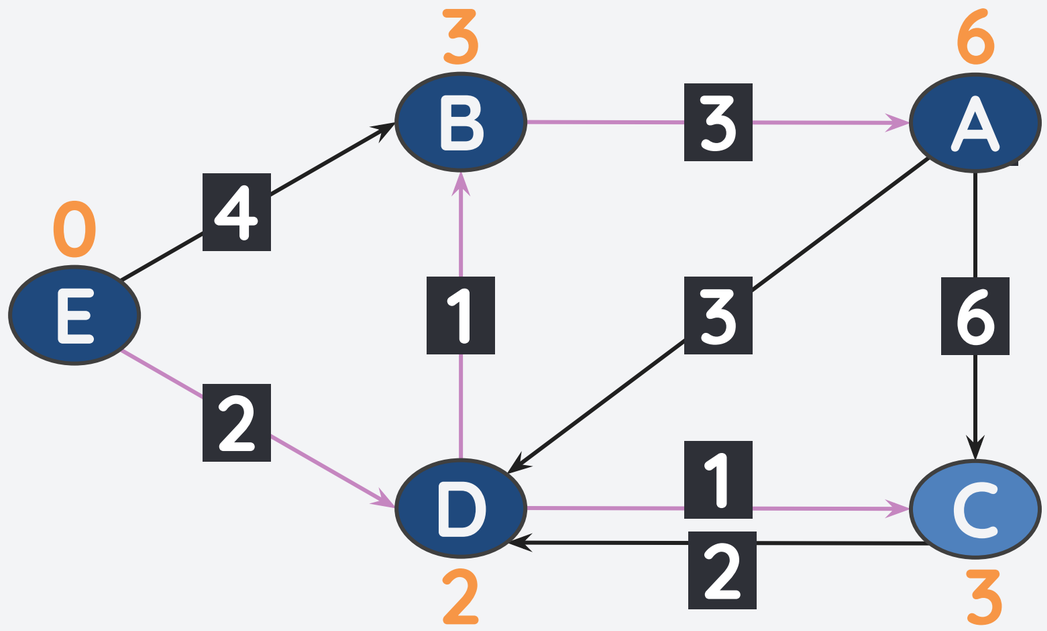
Iteration 3
Edge(s) from B

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the cost to B is updated, we also update the cost from B to A.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



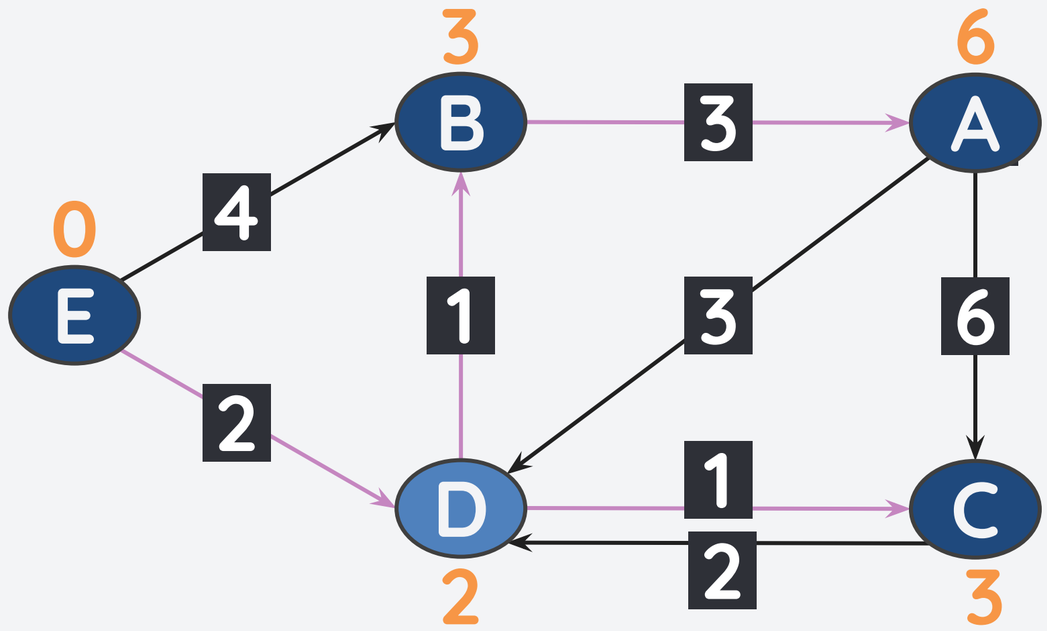
Iteration 3
Edge(s) from C

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the path from C to D has equal or higher cost than the current cost of D, no update.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



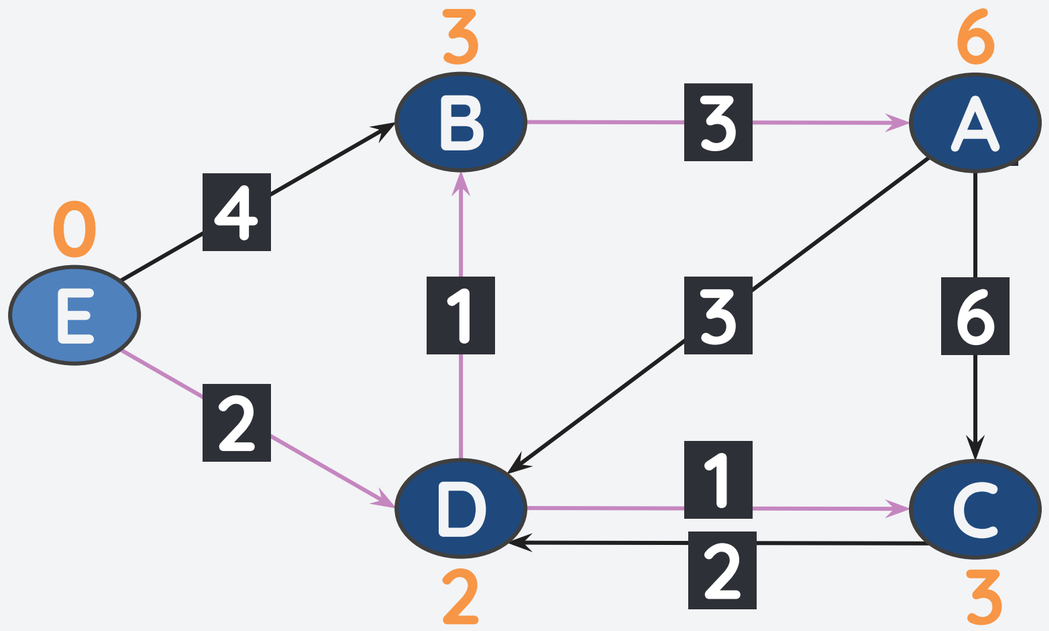
Iteration 3
Edge(s) from D

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the path from D to its neighbors has equal or higher cost than the current cost from D, no update.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



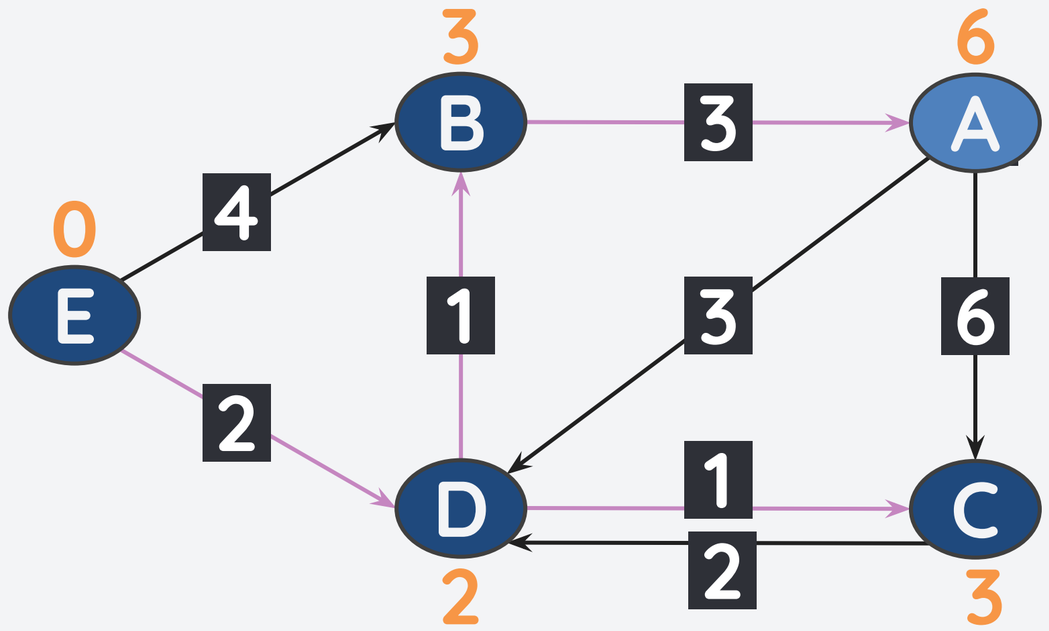
Iteration 3
Edge(s) from E

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

Since the path from E to its neighbors has equal or higher cost than the current cost from E, no update.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



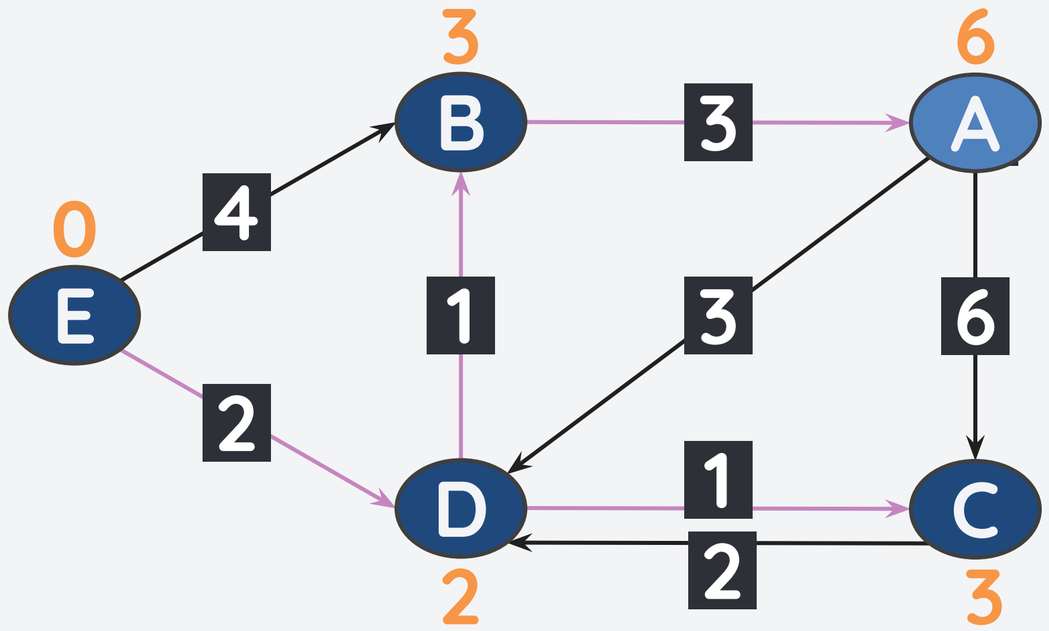
Iteration 4
Edge(s) from A

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

If you go over through all the nodes in iteration 4, you will see that there will be no more changes. Hence, we can stop the algorithm.

Bellman-Ford Algorithm for finding SPT

Consider the weighted graph shown below with positive and negative edges. Find the SPT using E as the starting node.



Final Cost Table

v	dA[v]	pre[v]
A	6	B
B	3	D
C	3	D
D	2	E
E	0	-

Final SPT

The Bellman Ford Algorithm

The Bellman-Ford algorithm finds the shortest path by performing edge relaxation on every edge over-and-over again.

- The algorithm is **complete** and can find solution even for weighted/unweighted graphs with negative edges.
- The algorithm is also **optimal** since it will find the shortest path due to its multiple iterations and edge relaxation.
- **Time Complexity: O(E*V)**
  - We iterate through all the edges at least V times.
- **Space Complexity: O(|V|)**

Summary

Algorithm	Complete?	Optimal?	Time Complexity	Space Complexity	Notes
DFS	No	No	Not important, for now.		Only for unweighted graphs.
BFS	Yes	Yes			
Dijkstra’s	Yes	Yes	O((|V|+|E|)log|V|)	O(|V|)	Can’t handle negative weights.
Bellman-Ford	Yes	Yes	O(|V||E|)	O(|V|)	Can handle negative weights and can detect cycles.

Any Questions?